

Operating Systems Principles: Notes

by Matthew Cai

Chapter 1: Introduction

"An **operating system** is the layer of software that manages a computer's resources for its users and their applications."

Roles

An operating system plays three main roles for the computer.

- **Referee.** Operating systems must fairly distribute resources between applications, and ensure the applications are isolated from each other. This includes protecting itself and other applications from malicious programs. It must also, however, allow inter-process communication in a safe, controlled manner.
- **Illusionist.** Operating systems must also provide an abstraction of physical hardware for the user applications. It also hides other inner system workings, like physical memory management and processor hardware sharing between processes. This is called *virtualization*, an illusion of resources that are not physically present.
- **Glue.** Operating systems also enable user applications to execute some common instructions and view common data (e.g. file system). They also provide a layer of abstraction between hardware I/O and applications, e.g. gluing the two together without needing applications to worry about hardware-specific operations and vice versa.

Goals

An operating system should satisfy the following criteria as best as possible.

- **Reliability and Availability.**

Reliability means that the operating system should minimize errors to the extreme, since errors in the OS can be devastating for users. This is especially true with regards to security.

Availability means the operating system should minimize crashes, since the OS should ideally be available as frequently as possible. It's typically measured by the frequency of failures (*mean time to failure (MTTF)*) and the time required to restore the system after a failure (*mean time to repair (MTTR)*). Higher MTTF or lower MTTR increases availability.

- **Security.** Self-explanatory.

- **Portability.**

A highly portable operating system is one that should change minimally between different hardware. That is, it should be easy to adapt a portable OS to run on different graphics cards, processors, etc. A portable OS should also be designed (or require minimal changes) to support applications and hardware that have not been developed yet, i.e. *forward compatible*.

This is typically accomplished by providing an **abstract virtual machine (AVM)**—an interface for system calls (API), a memory abstraction, etc. This allows applications and hardware to develop entirely independently.

Portability is also frequently enabled by a **hardware abstraction layer (HAL)**. Basically, an AVM is to applications as an HAL is to the operating system.

- **Performance.**

Performance is a surprisingly difficult concept to easily judge. There are several different measurements to consider:

- *Overhead*: the added resource cost of an abstraction for applications (less overhead = more efficient). That is, how much faster would the application be if run directly on the hardware.
- *Fairness*: how to equitably allocate resources between users/applications.
- *Response time*: how long it takes a single task to run. This is especially important for computers that people work on, as responsiveness for I/O dominated processes is extremely important.
- *Throughput*: how quickly the system completes tasks. This is often in opposition to response time, due to the overhead of context switches. (Ideally, the processor prioritizes processes that don't spend a lot of time waiting on I/O, as interactive processes typically do).
- *Predictability*: how consistent the system's performance (across the other metrics) is over time.

- **Adoption.**

This is just how many users use the OS. More users == more resources, more success.

There's also the issue of proprietary vs. open source operating systems. In other words, why Linux >> Windows/Mac.

In general, many of these criteria may oppose each other. Improvements in performance can impact security and portability negatively, for instance.

Operating Systems: Past, Present, and Future

Moore's Law

In previous decades, Moore's Law allowed operating system developers to ignore performance to some extent, as hardware got faster and faster, year by year. However, as Moore's Law slows down, OS maintainers increasingly have to consider performance, and how to best design an OS to not sacrifice the other qualities.

Multi-User Operating Systems

After the development of early operating systems, for the purpose of reducing programmer errors, came multi-user operating systems. This allowed sharing of resources between different users and applications.

A **batch operating system** worked on a queue of tasks, that could allow two tasks to "run" in tandem, one fetching from I/O devices using *direct memory access (DMA)*. Next came **multiprogramming**, which was essentially just concurrency. batch operating systems introduced challenges. though, in the lack of program isolation and the difficulty of debugging the operating system (required to turn the computer into a single-user system temporarily).

Time-Sharing Operating Systems

These were operating systems developed for interactive use, rather than processor-heavy jobs. This is your typical personal computer, which spends much of the time waiting for I/O from hardware like a keyboard or mouse. This required designing operating systems to handle bursts of activity, rather than consistent, sustained activity as seen in batch processing.

Modern Operating Systems

Lots of different types of operating systems for different applications, now. Too many to talk about.

Chapter 2: The Kernel Abstraction

Protection, i.e. isolation of applications and users from each other and the operating systems, is a critical job for the operating system kernel. A **process** is the execution of a program within restricted rights, or the abstraction for protected execution. A process needs permissions from the kernel to perform certain operations.

2.1 The Process Abstraction

The OS allocates memory to store code (.text), data (.bss, .data, etc.), the heap (grows up), and the stack (grows down). The virtual memory addresses are in that order. The OS starts the program by just setting the stack pointer and jumping to the first instruction. To run multiple copies, the OS duplicates the memory mappings. Frequently, this memory will be deduplicated to reduce overhead until write (Copy on Write). The OS keeps track of the various processes using a **process control block**.

2.2 Dual-Mode Operation

The OS executes in **supervisor/kernel mode** and all other processes execute in **user mode**. User mode has protection checks in place. This is controlled by bit 0 and 1 in the **CS** register in x86 (supports 4 privilege rings). The hardware must support at least three things to allow the OS to properly implement protections:

Privileged Instructions.

Programs executing in user mode are not allowed to execute privileged instructions that could be dangerous. Any attempt will result in a processor exception, in which the processor transfers control to a kernel exception handler.

Memory protection.

Userspace programs should not be allowed to access or modify sensitive kernel memory or the memory of other processes.

A very simplistic approach to memory protection is having two extra registers for a processor: **base** and **bound**. The memory a program accesses must be between these two registers. This has some issues with how expanding the heap/stack should work, how programs should share memory, how memory addresses change between different processes of the same program, and how bad it is for memory fragmentation.

Modern approaches utilize virtual memory paging, and the kernel ensures that only the memory required for the process is mapped into the page table. (Excluding some subset of

kernel addresses, see [KPTI](#). Flags are set to ensure access is not allowed).

Timer Interrupts.

Hardware needs to allow timer interrupts to enable the OS to interrupt a program and regain processor control, if necessary. This is known as [preemption](#).

2.3 Types of Mode Transfer

2.3.1 User → Kernel Mode

This can occur asynchronously—interrupts—or synchronously—processor exceptions and system calls. Synchronous transfers from user to kernel mode are called [traps](#).

Interrupt

This is when some hardware, software, or another processor triggers an interrupt, notifying the kernel that one of the processors needs to handle it. This causes the kernel to elevate to kernel mode to handle the interrupt before returning to user mode execution. An alternative to interrupts is [polling](#), but the kernel can't run user code while polling.

Processor Exceptions

This is a hardware event caused by some error in the user program, e.g. a page fault, division by zero, etc. These usually result in the process halting. On a multiprocessor, the kernel must also send interprocessor interrupts to other processors running the same program in parallel.

System Calls

A system call is a way for the user process to voluntarily transfer control to the kernel to request some sort of restricted operation that the user cannot themselves perform.

2.3.2 Kernel to User Mode

This can happen when a new process starts, the kernel resumes after one of the aforementioned user to kernel mode switches, the kernel switches a processor to run a different process, or a user-level upcall occurs (asynchronous event notification for user programs).

2.4 Safe Mode Transfer

For safe transitions between user and kernel mode, the OS must at least provide

- Limited entry into the kernel
- Atomic changes to processor state
- Transparent, restartable execution

Interrupt Vector Table

This is a table, whose address is held in a special register, such that it points to the addresses of various interrupt handlers.

Interrupt Stack

When some context switch from user to kernel mode occurs, the kernel will use the **interrupt stack**. Its address is held in a special hardware register, and the kernel uses it to save user registers and the return address to the user program.

The interrupt stack is necessary for reliability—the process's original stack pointer may not be valid, due to errors or malicious behavior in the process—and security—on a multiprocessor, other threads could modify the user memory during a system call and take over kernel control flow.

Most OS kernels will actually allocate a kernel interrupt stack for **every** userspace process. This makes it easier to switch to a new process inside an interrupt/syscall handler. This interrupt stack will usually contain the user CPU state, the syscall handler address, and then potentially other information (e.g. I/O driver processing triggered from the syscall).

The kernel interrupt stack can be in one of several states:

- If the process is currently running in user mode, the stack is empty
- If the kernel is running in context of the user process, the stack contains the user CPU state and current state of the kernel handler
- If the process is available to run but is waiting for an available processor, the stack contains the user CPU state
- If the process is waiting for I/O, the kernel stack contains the information for the suspended code to resume when I/O finishes

Note that multiple interrupts in a row (i.e. kernel execution is interrupted) will just all be pushed onto the interrupt stack.

Interrupt Masks

Hardware allows the kernel to temporarily *disable interrupts*. Or, more accurately, **defer** interrupts to be handled at a later time. Once a corresponding instruction to enable interrupts executes, pending interrupts are processed. Interrupts are typically disabled whenever an interrupt handler is executing. This prevents confusion, e.g. an interrupt mid-handler may overwrite the previous interrupt handler's information on the interrupt stack. Hardware does notably have limited buffering for interrupts, so interrupts may be lost if disabled for too long.

Interrupt Handlers

Interrupt handlers can trigger a large amount of processing. As aforementioned, some interrupts may be lost if disabled for too long. Thus, interrupt handlers are frequently split into a **top half** and **bottom half**. (Note: in Linux, this terminology is reversed). The interrupt's bottom half executes with interrupts masked, and is typically designed to finish quickly. The bottom half then saves the hardware device state, resets it for a new event, and then re-enabling interrupts. This will return to the interrupted task or, if the event is high priority, return to the top half.

Hardware Support for Context Switches

It's always necessary to save the CPU state (registers) before an interrupt handler runs. Hardware (and assembly ISA) will typically provide some support to perform this operation very quickly.

2.5 x86 Mode Transfer

1. Mask interrupts
2. Save the stack pointer (`esp` and `ss` registers), execution flags (`eflags` register), and instruction pointer (`eip` and `cs` registers) to internal, temporary registers in hardware
3. Switch stack pointer to kernel interrupt stack
4. Push the previously saved values onto the stack
5. Push an error code (or a dummy value if not applicable)
6. Jump to the interrupt handler

2.6 Secure System Calls

The process is

1. User program calls the user stub syscall
2. The user stub executes the trap instruction

3. The hardware transfers control to the kernel, uses the syscall vector table to jump to the syscall handler, which is a stub on the kernel side copying/checking args and calling the kernel's implementation of the syscall
4. The syscall's completion returns control to the handler
5. The handler returns to the user level stub
6. The stub returns to the caller

In x86, the system call code is stored in register `eax`.

The kernel stub is a bit complex; it must

- Locate syscall arguments. This is on the stack for x86, but is passed via registers in modern architectures (x86-64).
- Validate parameters.
- Copy before checking to avoid **TOCTOU**
- Copy back any results from kernel to user memory.

2.7 Starting a New Process

In order to initialize a new process, the kernel must:

- Allocate the process control block (this should happen at kernel initialization, actually)
- Allocate process memory
- Load the program from disk into allocated memory
- Allocate a user-level stack
- Allocate a kernel-level stack
- Copy arguments into user memory (`argv`)
- Switch to user mode and transfer control to the process

2.8 Implementing Upcalls

An upcall is the *reverse of a syscall*, allowing the kernel to "call up" into a user process. This allows the kernel to notify applications of events, which helps some applications behave more like operating systems. In UNIX, there are called **signals**. In Windows, asynchronous events.

There are several uses of upcalls:

- **Preemptive user-level threads** via a periodic timer upcall to allow applications to switch between tasks (e.g. web browser terminates third party script)

- **Asynchronous I/O notification** so that the application doesn't have to poll the kernel while waiting for I/O.
- **Interprocess communication** via a syscall from the sender and an upcall to the receiver.
- **User-level exception handling**, i.e. when an application implements its own, separate exception handling.
- **User-level resource allocation**, i.e. some applications optimize resource usage according to their resource allocation. So, an OS may upcall to inform the process of a change in allocated resources.

Upcalls are not always needed, as applications can simply poll the kernel. This is not necessarily as desired though.

We describe some features of the UNIX signals that have proved useful. Note the similarity with interrupts.

- Different signal types
- Signal handlers
- Signal stack
- Signal masking
- Saving processor state

Chapter 3: The Programming Interface

Everything in the operating system must consider a tradeoff between

- **Flexibility.** It's much easier to modify OS code that lives outside the kernel. For example, the syscall interface must be kept as backwards compatible as possible, making it difficult to change significantly.
- **Safety.** Obviously.
- **Reliability.** Kernel modules are not protected from each other, so a bug in kernel code can result in system crashes. Some OS's try to keep the majority of the code outside of the kernel--this is known as a *microkernel*.
- **Performance.** Mode switches significantly impact performance. Modern kernels are primarily *monolithic kernels* (essentially the opposite of a microkernel) for this reason.

3.1 Process Management

Early operating systems had the kernel in control of everything. Modern operating systems allow user programs to create and manage their own processes. A *shell*, for instance, is a job control system, allowing you to run several instructions in sequence.

UNIX manages processes via `fork()` and `exec()`. `fork()` duplicates the parent process, while `exec()` replaces the current process with an entirely new one. The parent may `wait()` until the child terminates.

3.2 Input/Output

The basic ideas of UNIX I/O are

- **Uniformity.** All device I/O, file operations, and interprocess communication use the same set of system calls: `open`, `close`, `read`, `write`.
- **Open before use.** Before an application does I/O, it must first call `open`. This allows for access permission checks and internal bookkeeping for the OS.
- **Byte-oriented.** All devices transfer data through byte arrays.
- **Kernel-buffered reads.** Stream data is stored in a kernel buffer and returned to the application on request. This enables uniformity between devices with streaming reads and those with block reads.
- **Kernel-buffered writes.** Same thing here.
- **Explicit close.** When an application finishes with a device/file, it must call `close`.

There's a bit more to discuss with IPC.

- **Pipes.** A UNIX pipe is a kernel buffer with a file descriptor for writing and another for reading. Since the data is buffered, producer and consumer execution can be decoupled. The pipe terminates when either endpoint closes the pipe or exits.
- **Replace file descriptor.** A shell can manipulate the file descriptors of a child process to change stdin/stdout/stderr to files, pipes, etc.
- **Wait for multiple reads.** The UNIX call `select(fd[], n)` allows waiting for input from a set of file descriptors, rather than polling each in turn.

3.4 Interprocess Communication

Three common ways:

- **Producer-consumer.** Communication is a one-way street. Can be multiple producer or multiple consumer, too. Is typically done through a simple pipe.
- **Client-server.** Bidirectional communication. Client asks server to perform tasks, and the server responds with results. Is typically done through two pipes, one for each direction.
- **File system.** Here, communication can be separated temporally. Is typically done through files (consume permanent disk space).

3.5 Operating System Structure

Two types of kernels, as mentioned previously.

- **Monolithic Kernel.** Most of the operating system runs inside the kernel. This is the most performant, and is what most modern operating systems used. All modern systems have a hardware abstraction layer and dynamically loaded device drivers. This separates OS development from the hardware.

Notably, though, errors in device drivers can corrupt the kernel. Thus, kernel developers must review drivers extensively before allowing incorporation into the kernel. More modern solutions include user-level device drivers or device drivers isolated by VMs and sandboxing.

- **Microkernel.** Minimal kernel space code, with most operating system functionality executing in userspace. This is much easier to develop, as it enables easy modularization and debugging. However, the frequency of context switches reduces performance.

Chapter 4: Concurrency and Threads

4.1 Why Threads?

- **Program Structure: logically concurrent tasks**

Programs that interact with the real world frequently have concurrent activities to perform.

- **Responsiveness: shifting work to run in the background**

For interactive computers, threads help improve responsiveness and perceived performance for the user by creating threads to perform work in the background while allowing the user to still do other things with the program/computer. For the OS, this is critical to ensure that the computer remains responsive while expensive operations (like disk writes) are happening.

- **Performance: exploiting multiple processors**

As hardware continuously improves and more and more cores are embedded into a single computer, utilizing multiple CPUs of a computer has become critical to improving performance. Threads are a way to split work between different processors.

- **Performance: managing I/O devices**

While expensive I/O device operations are happening, it's more useful for the operating system to perform other tasks in the meantime. By creating a separate thread for I/O operations, the kernel can effectively multitask, using processor power for other processes while waiting on I/O operations. It also allows the kernel to respond to sudden, external events like key presses while still working on other tasks.

4.2 Thread Abstraction

A thread is a *single execution sequence* that represents a *separately schedulable* task.

- **Single Execution Sequence:** Simply put, each thread executes a sequence of instructions, as in a normal process.
- **Separately Schedulable Task:** An OS can run, suspend, or resume a thread at any time.

In the thread abstraction, the applications are provided with the *illusion* of an infinite number of processes. This is, of course, untrue. But the OS is able to implement this via a **thread scheduler**, allocating time slices for each thread to be able to use a processor. A consequence of this design, though, is that the thread abstraction paints threads as running on a dedicated virtual processor with unpredictable, variable speed.

Cooperative vs. Preemptive Multithreading

Cooperative multithreading is a system in which a thread runs until it *voluntarily* relinquishes control of the processor. These systems were used as recently as the early 2000s, but are no longer a part of modern operating systems. (A malicious application could easily abuse this to hog the processor!). The idea remains in use on the application level, though.

Preemptive multithreading is a system in which the processor has full control over which thread is running, able to arbitrarily switch from one thread to another at any given time. This is used in the vast majority of modern operating systems.

4.3 POSIX Thread API

The important functions in the POSIX Thread (`pthread`) API can be found in the `pthread.h` header and are as follows:

```
/*
The pthread_create() function starts a new thread in the calling
process. The new thread starts execution by invoking
start_routine(); arg is passed as the sole argument of
start_routine().
*/
int pthread_create(pthread_t *restrict thread, const pthread_attr_t *restrict
attr, typeof(void *(void *)) *start_routine, void *restrict arg);

/*
pthread_yield() causes the calling thread to relinquish the CPU.
The thread is placed at the end of the run queue for its static
priority and another thread is scheduled to run.
*/
int pthread_yield(void); // deprecated

/*
The pthread_join() function waits for the thread specified by
thread to terminate. If that thread has already terminated, then
pthread_join() returns immediately. The thread specified by
thread must be joinable.
```

If `retval` is not `NULL`, then `pthread_join()` copies the exit status of the target thread (i.e., the value that the target thread supplied to `pthread_exit(3)`) into the location pointed to by `retval`. If the target thread was canceled, then `PTHREAD_CANCELED` is placed in the location pointed to by `retval`.

If multiple threads simultaneously try to join with the same thread, the results are undefined. If the thread calling `pthread_join()` is canceled, then the target thread will remain joinable (i.e., it will not be detached)

*/

```
int pthread_join(pthread_t thread, void **retval)
```

/*

The `pthread_exit()` function terminates the calling thread and returns a value via `retval` that (if the thread is joinable) is available to another thread in the same process that calls `pthread_join(3)`.

Any clean-up handlers established by `pthread_cleanup_push(3)` that have not yet been popped, are popped (in the reverse of the order in which they were pushed) and executed. If the thread has any thread-specific data, then, after the clean-up handlers have been executed, the corresponding destructor functions are called, in an unspecified order.

When a thread terminates, process-shared resources (e.g., mutexes, condition variables, semaphores, and file descriptors) are not released, and functions registered using `atexit(3)` are not called.

After the last thread in a process terminates, the process terminates as by calling `exit(3)` with an exit status of zero; thus, process-shared resources are released and functions registered using `atexit(3)` are called

*/

```
void pthread_exit(void *retval) // noreturn
```

Source.

Fork-Join Parallelism is a very common method of multithreading, in which a parent thread forks several child threads to perform work, before joining (waiting for) all their results. Each thread executes independently and deterministically. With the pthread API, this is done via `pthread_create` and `pthread_join`.

4.4 Thread Data Structures and Life Cycle

Every thread has some per-thread state and metadata associated with it that the processor must save and restore when switching between threads. These are stored in what is commonly known as the **Thread Control Block (TCB)**. For ELF files (Linux), this is typically called **thread local storage**.

The TCB holds two types of per-thread information:

- **Computation State:** This is primarily a pointer to the thread's stack and a copy of the processor registers. These are saved and restored for the processor's use whenever switching threads.
- **Metadata:** Thread ID, scheduling priority, status, etc.

Aside: Stack Size

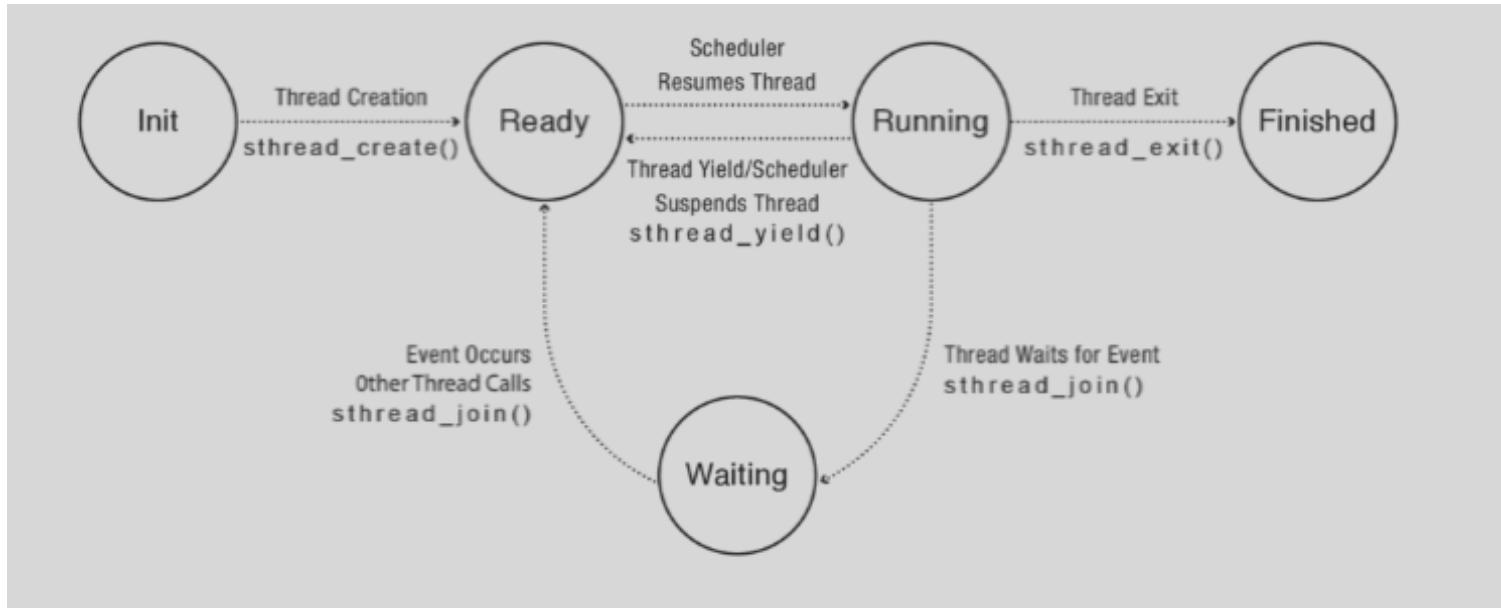
One interesting problem is how much stack space should be allocated for each thread. For kernel threads, since they operate directly on physical memory, they are typically quite minimal. For Linux running on Intel x86, the default is 8 KB, and the kernel takes extra precautions to ensure a thread does not overflow its stack.

Meanwhile, for userspace threads, the kernel can provide the illusion of infinite memory via virtualization. Thus, these stacks can grow without bound. However, large or extremely rapid increases in size are generally caught by the operating system, since this typically indicates unbounded recursion or other unintended behavior.

Threads do still share some state between threads running in the same process or within the operating system kernel. Most prominently, program code, statically allocated global variables, and heap variables are typically shared between threads. Note that some of these can become thread-local, e.g. heap arenas are frequently thread-specific for performance reasons (better cache locality).

4.5 Thread Life Cycle

The thread life cycle can be summarized in the below diagram:



- **INIT:** Thread creation moves a thread into the INIT state and initializes any per-thread data structures. Once finished, `pthread_create` will add the thread to the *ready list*, the set of runnable threads waiting for a processor.

Sidenote: the list of runnable threads is not actually a list. In Linux, for example, it is a red-black tree balanced by priority.

- **READY:** A thread in the READY state is available to run and waiting for a processor. Its TCB is on the ready list, with the saved computational state. At The scheduler can bring a READY thread to the RUNNING state at any time.
- **RUNNING:** A thread in the RUNNING state is currently running. To move to the READY state, one of two things can happen:
 - The scheduler preempts the RUNNING thread and moves it to the READY state after saving its current state and switching the processor to another READY thread.
 - The RUNNING voluntarily relinquishes the processor (`pthread_yield`)
- **WAITING:** A thread in the WAITING state is waiting for an event, possibly I/O or something else. It is not running on the processor during this time, to allow other applications to run. The TCB is stored on the *waiting list* of some *synchronization variable* associated with the event. This could be caused by a function like `pthread_join`.
- **FINISHED:** A thread in the FINISHED state is done, and will never run again. The state is not immediately freed yet, in order to let functions like `thread_exit` pass its exit value to

its parent thread if necessary. Its TCB is on the *finished list*, and the space will eventually be reclaimed once its data is no longer needed.

4.6 Kernel Threads

We first consider the simplest case: the implementation of threads that run inside the operating system with full privileges.

Thread Creation

1. **Allocate per-thread state.** This generally involves allocating space for the TCB and the kernel stack (recall that this is fixed size in the kernel).
2. **Initialize per-thread state.** First, the constructor initializes the thread's registers. It then starts the thread in a dummy "stub" function that will call `start_routine(arg)`. This extra step is necessary in case the `start_routine` function never calls `thread_exit`. In this case, it would return to whatever is at the top of the stack. In the stub function, the kernel ensures that `thread_exit` is called.
3. **Put TCB on ready list.** INIT → READY.

Thread Deletion

1. **Put TCB on finished list.** RUNNING/READY → FINISHED.
2. **Free per-thread state.** Importantly, a thread should not be deallocating its own state. This could lead to issues (e.g. interrupt during freeing leads to memory leak, context switch code tries to write to deallocated memory, etc.). It is always necessary for *another* thread to free this thread's state.

Thread Context Switch

There are two types of thread context switches: voluntary and involuntary. Voluntary switches are triggered by functions like `pthread_yield`, and proceed as follows:

1. **Turn off interrupts.** This prevents the kernel from attempting to perform two context switches at the same time.
2. **Save per-thread state.**
3. **Switch to next thread on ready list.** The next thread to run is chosen by the scheduler.

Meanwhile, an involuntary context switch might occur due to an interrupt, processor exception, or trap.

1. **Save per-thread state.** Hardware saves some state upon interrupt/exception, and software saves the rest of the state when the handler runs.
2. **Run handler.** No need to switch from user mode to kernel mode, since a kernel thread was interrupted.
3. **Switch to next thread on ready list.** In contrast to what you might expect, the kernel will then switch to the next thread on the ready list, rather than always resuming the thread that was just suspended.

4.7 Kernel Threads and Single-Threaded User Processes

Since each process can have multiple threads (note: on Linux, this is not true; threads are simply lightweight processes), each user process needs a **Processor Control Block (PCB)** that contains process metadata. Each thread, meanwhile, must have its own interrupt stack too.

Note one important difference between context switching from a kernel thread to a handler and doing the same for a user thread. If the processor is already in kernel mode, it simply pushes the instruction pointer and eflags onto the existing stack. This is because kernel threads don't have interrupt stacks, as their stack functions fine for this. Meanwhile, context switching from a user thread requires the hardware to change the stack pointer to the interrupt stack, and then push the instruction pointer, eflags, *and original stack pointer* onto the stack. This means that the `iret` instruction will also function differently, depending on the contents of the saved eflags. That is, after popping the saved eflags, it'll check the mode to see if it needs to pop the stack pointer too.

4.8 Multithreaded Processes

Each thread must have

- A userspace stack
- A kernel interrupt stack for traps, processor exceptions, and interrupts
- A kernel TCB

Though, there do exist non-operating-system threading applications. Some userspace threading libraries exist. For instance, the earliest implementations of Sun's Java Virtual Machine used **green threads**. The idea is essentially the same, except all the normal process and thread management data structures are allocated in the process's address space rather than the operating system. The only issue with green threads, though, is that the OS remains

unaware of the state of the userspace ready list. So, if an application blocks to wait for I/O, the kernel doesn't know to switch to another userspace thread.

Most operating systems do allow preemption among userspace threads executing within a process. This functionality is delivered through upcalls, or signals on UNIX systems. Frequently, userspace thread implementations will register timer signal handlers to context switch between threads periodically.

Modern operating systems, though, have built-in **kernel support** for userspace threads. For instance, `thread_join` is frequently made hybrid, in which `thread_join` doesn't always have to make a syscall, and should recognize when a thread has exited from data structures in the process. On a multiprocessor, it may even be more efficient to let `thread_join` block for a few microseconds in hopes the other thread will finish during this time.

Multiprocessors may also implemented per-processor kernel threads. This allows a userspace library to multiplex userspace threads on top of kernel threads while kernel threads are multiplexed on top of physical processors. This allows running the user-level scheduler in parallel across all kernel threads.

Recent developments have also led to **scheduler activations**. Essentially, the userspace thread scheduler is notified (*activated*) of every kernel event that potentially affects the userspace thread system. These events include

- Changing the number of virtual processors
- Transitions between thread states (e.g. waiting for I/O)

Alternative Abstractions

Asynchronous I/O and Event-Driven Programming

In async I/O, a single-thread process will make multiple concurrent I/O requests simultaneously. The operating system will provide the result later, *asynchronously*, by calling a signal handler, placing the result in a queue in process memory, or storing the result in kernel memory until the process makes a syscall (polls).

Asynchronous I/O has given rise to **event-driven programming** where a thread spins in a loop, with each iteration processing a different I/O event.

Data Parallel Programming

Data parallel programming is simply executing a single operation on multiple pieces of data simultaneously. Most commonly, **SIMD** (single instruction multiple data), or more recently **MIMD** (multiple instruction multiple data). GPUs are especially powerful for these types of tasks.

Chapter 5: Synchronizing Access to Shared Objects

Sequential reasoning does *not* work in programs with cooperating threads:

1. Threads' access to shared state may be interleaved
2. Nondeterministic program execution due to varying scheduling decisions, processor frequencies, etc.
3. Compilers and processor hardware may reorder instructions

5.1 Challenges

- **Race condition.** When behavior of a program changes depending on thread interleaving.
- **Atomic operations.** These operations cannot be interleaved with/split by other operations.

5.2 Structuring Shared Objects

Shared objects can be accessed safely by multiple threads. All shared state in a program should be encapsulated in a shared object.

There are several layers in shared object implementation:

- **Shared object layer.** The underlying details of these objects are abstracted away—they seem to work perfectly in multithreaded situations.
- **Synchronization variable layer.** Synchronization variables include semaphores, locks, and conditions variables. These coordinate access to *state variables*.
- **Atomic instruction layer.** Some instructions must execute atomically in order to support synchronization variables.
- **Hardware layer.** Of course, there must be hardware support for atomic instructions.

5.3 Locks: Mutual Exclusion

A lock is a synchronization variable that provides a thread *exclusive* access to some subset of shared thread. Any operations done while a thread holds a lock appear to be *atomic* to other threads.

A lock has two states—BUSY (held by a thread) or FREE (can be acquired)—and two associated methods—`Lock::acquire()` and `Lock::release()`. `Lock::acquire()` blocks until the lock is FREE, and then atomically transitions the lock state to BUSY. `Lock::release()` atomically transitions the lock state to FREE. If there are pending `acquire` operations, it will trigger one to proceed.

Formal Properties. A lock should ensure the following 3 properties:

- **Mutual Exclusion:** At most one thread holds the lock.
- **Progress:** If no thread holds the lock and any thread attempts to acquire the lock, eventually some thread will succeed.
- **Bounded waiting.** If thread T tries to acquire a lock, there is a bound on the number of times other threads can successfully acquire the lock before T does.

Critical Section. A critical section is a section that atomically accesses shared state. So, a section of code in which a lock is acquired by a thread is a critical section.

5.4 Condition Variables: Waiting for a Change

A condition variable is a synchronization object that lets a thread efficiently wait for a change to shared state protected by a lock. That is, it avoids busy waiting. It has three methods:

- `CV::wait(Lock *lock)`. *Atomically releases* the lock and *suspends execution* of the calling thread. This avoids busy waiting by immediately yielding the processor. When the calling thread continues, it will *reacquire* the lock before returning.
- `CV::signal()`. This call takes a thread off the condition variable's waiting list and marks it as ready to run (READY state), if the waiting list is nonempty.
- `CV::broadcast()`. Same as `CV::signal()`, but for all threads.

A condition variable also has three noticeable properties:

- **Memoryless.** A condition variable has no internal state other than a queue of waiting threads.
- **CV::wait atomically releases the lock.** This is necessary to avoid a race condition, in which a signal might slip in between the lock releasing and the thread being put onto the waiting list.
- `CV::signal` or `CV::broadcast` does **not** guarantee a waiting thread will run **immediately**. It will simply move the thread to the ready queue with no special priority, and then the scheduler can run it at a later time at its own discretion.

Notably, the above properties mean that `wait` must *always* be called within a loop, as, after a `signal` or `broadcast` call, the lock may not be in the FREE state anymore and the shared state variables may not satisfy the desired conditions. e.g., the following code...

```
if (testStateVariables(...)) {  
    wait(&lock);  
}
```

...could run into issues, as other threads may have run between the time of the `signal` and this thread continuing, resulting in changes to the shared state such that `testStateVariables` is no longer true, despite it being true when `signal` occurred.

Mesa vs. Hoare Semantics

These are different semantics for condition variables. Mesa has `signal` and `broadcast` calls pop waiting threads from a condition variable's waiting list and puts them on a ready list. In contrast, Hoare has `signal` immediately suspend execution of the signaling thread and transfer lock ownership to a waiting thread, whose execution is immediately resumed. After this thread finishes, the lock ownership returns to the signaling thread.

Hoare semantics allows programmers to use `if` instead of `while` for condition variables. However, practically all modern systems use Mesa semantics due to the modularity advantage and, primarily, the reduction in context switches it provides.

Revisiting the thread life cycle, it's important to note that a RUNNING thread that calls `wait` is put into the WAITING state, and its TCB is moved into the WAITING thread queue of the *condition variable*. That is, there are multiple queues for different CVs. Locks function similarly, with a queue of waiting TCBs per lock.

5.5 Designing and Implementing Shared Objects

High Level Approach

1. **Add a lock.**
2. **Add code to acquire and release the lock.** In particular, the lock should be acquired/released at the beginning/end of each function that accesses shared object. This is the easiest, simplest method, and should generally always be used since acquiring a lock is relatively inexpensive (i.e. acquiring/releasing later/earlier will not significantly optimize the code).
3. **Identify and add condition variables.** Consider each method and ask when it can wait. Occasionally, it may also be more convenient to define fewer condition variables for a

large number of possible conditions, and simply force methods to broadcast for the condition variable after releasing a lock.

4. **Add loops to wait for using the condition variables.** As aforementioned, condition variables should always be used with a loop. Modern operating systems often even allow spurious wakeups, sometimes (thread returns without any `signal` / `broadcast`). This also increases code *modularity*, as this provides guarantees about whatever condition was being waited on for the code following this loop.
5. **Add `signal` and `broadcast` calls.** `signal` is appropriate when (1) at most one waiting thread can make progress at a time and (2) any arbitrarily chosen thread can make progress. Meanwhile, `broadcast` is appropriate when (1) multiple waiting threads can make progress simultaneously or (2) the condition variable is used for different conditions/predicates for multiple threads.

Implementation Best Practices

1. **Consistent structure.** Having clean, consistent code allows for easier reasoning about the concurrency and more focus on the core problem.
2. **Always synchronize with locks and condition variables.** As in, use locks and CVs instead of semaphores, because they are more "self-documenting."
3. **Always acquire the lock at the beginning of a method and release it right before the return.** As previously discussed, this is the easiest, simplest method. If there is a legitimate reason for not doing this, perhaps this logic should be made into its own function.
4. **Always hold the lock when operating on a condition variable.** Otherwise, it's undefined behavior.
5. **Always `wait` in a `while()` loop.** Discussed previously, ensures modularity and portability with implementations that allow spurious wakeups.
6. **(Almost) never use `thread_sleep`.** `wait` is generally better and more useful.

Three Pitfalls

1. **Double-checked locking.** An "optimization" that makes a check before acquiring a lock to see if acquiring a lock is necessary, then acquires the lock, then checks again. The reason the check inside the lock is necessary is because of a race condition between the first check and the lock acquisition. However, this practice can result in code that is complex or even wrong.
2. **Avoid defining a synchronized block in the middle of a method.** Again, discussed previously.

3. **Keep shared state classes separate from thread classes.** Making thread classes have shared state as well blurs the lines between threads and shared objects, which can make things confusing.

5.7 Implementing Synchronization Objects

Some hardware primitives are required for synchronization objects:

- Disabling interrupts.
- Atomic read-modify-write instructions.

On a uniprocessor, it's possible to implement locks by simply disabling interrupts on acquisition, and re-enabling on release. However, untrusted userspace program could hold a lock forever, monopolizing the processor. Instead, it's better to use uniprocessor **queuing locks**. This only disables interrupts while acquiring/releasing the lock, but re-enables interrupts once the lock is acquired/released. This is because, if other threads do try and acquire this lock while it's BUSY, the thread will simply go to the lock's waiting list. In this way, the lock works as expected while avoiding processor monopolization by a single thread.

Critically, if a thread tries to acquire a lock and ends up sleeping, it is the responsibility of the *next thread* that wakes up from a lock to re-enable interrupts.

On a multiprocessor, though, disabling interrupts is insufficient, as other cores may be executing threads that modify the synchronization objects. Thus, processor architectures must provide **atomic read-modify-write instructions** to support synchronization. One possible implementation of locks is spinlocks, in which the thread waiting for a BUSY lock spins (busy-waits) while waiting for another thread to release the lock. This can be inefficient, though, if locks are held for a while.*

**Spinlocks can be useful, primarily when the expected waiting time is short, i.e. less than the overhead of context switching to a different thread.*

Interrupt handlers and spinlocks

Notably, interrupt handlers that access shared data must use spinlocks. This is because a queuing lock might result in an interrupt handler being blocked, which cannot occur. Also, when a spinlock is acquired for an interrupt handler, the thread *must* disable interrupts. Otherwise, a deadlock could result if another interrupt arrives while the spinlock is still held.

For multiprocessor queuing locks, it's necessary to have a spinlock for the scheduler READY list. To reduce contention, each lock's internal state is guarded a spinlock. Importantly, it is

also necessary to disable interrupts before locking the ready list, as a preemption could result in a deadlock.

Linux's lock implementation exploits the fact that most locks are generally FREE when acquisition is attempted. Therefore, it provides a fast path for acquiring a lock using several hardware optimizations and a design that allows lock acquisition and release *without* first acquiring the spinlock or disabling interrupts. It does this by keeping an **atomic** count variable that can denote the lock as FREE (1), BUSY (0), or BUSY and having possible waiting threads (any negative number). The fast path atomically decrements this count variable and then checks the value to see if it's FREE. If not, it falls back to the slow path, which requires acquiring the spinlock and disabling interrupts.

Condition variables are implemented very similarly. However, it's notably necessary to acquire the scheduler spinlock *before* releasing the lock associated with the CV. This ensures there are no race conditions with signaling/moving threads off the waiting list.

Meanwhile, application-level synchronization can be implemented via kernel-managed threads or user-managed threads. The former generally works very similarly. The latter has all synchronization data structures in process memory, and also disables signals/upcalls rather than interrupts (of course, the ability to disable interrupts from userspace would be devastating for security).

5.8 Semaphores Considered Harmful

Semaphores:

- A semaphore has a **non-negative** value x .
- x is initialized upon semaphore creation with the desired "capacity" of the semaphore, i.e. how many concurrent accesses are allowed.
- `semaphore::P()` waits until $x > 0$, then atomically decrements x by 1.
- `semaphore::V()` atomically increments x by 1, and then enables a thread waiting in `semaphore::P()` to run.
- No other operations allowed. In particular, no thread can read the semaphore's internal state, specifically x .

Semaphores can be used for locks and condition variables. A mutual exclusion lock is implemented with $x = 1$. For condition variable-like waiting, `semaphore::P()` functions very similarly to `CV::wait()` and `semaphore::V()` functions similarly to `CV::signal()`. However, there are some distinct differences. Notably, `semaphore::P()` does not release an associated lock and a condition variable is stateless, while a semaphore is not. The latter, in fact, results

in an observable difference: if a thread calls `CV::signal` with nothing on the waiting list, nothing happens; but, if a thread calls `semaphore::V`, the next call to `semaphore::P` will not block.

In general, it's more difficult to write concurrent programs with semaphores, for little benefit. Though, they are critically more performant for synchronizing communication between an I/O device and threads blocking for I/O. This is because the *statefulness* of semaphores allows for avoiding a race condition between an I/O device calling `signal` and the destination thread, when it's in between checking for I/O and waiting.

Aside: Atomic Read-Write-Modify Instructions

As aforementioned, multiprocessors cannot use the interrupt method to provide synchronization primitives. Moreover, another negative of the interrupt method is that *users cannot use this lock*, since users cannot enable/disable interrupts.

An effective alternative is **atomic read-write-modify instructions**. Hardware is responsible for implementing this properly. Notably, on multiprocessors, this introduces some complexity in the cache coherence protocol. This class considers the following three atomic RWM instructions:

```
int test_and_set(int* address) {
    int result = *address;
    *address = 1;
    return result;
}
```

```
void swap(int* address, int* reg) {
    int tmp = *address;
    *address = *reg;
    *reg = tmp;
}
```

```
bool compare_and_swap(int* address, int* reg1, int* reg2) {
    if (*reg1 == *address) {
        *address = *reg2;
    }
}
```

```

        return true; // success
    } else {
        return false; // failure
    }
}

```

162 actually defines them as follows, exactly:

Test and Set (TSET)	Atomic Swap (SWAP)	Compare and Swap (CAS)
<pre> int TSET(&addr) { int result = M[addr]; M[addr] = 1; return(result); } </pre>	<pre> int SWAP(&addr,expr) { int result = M[addr]; M[addr] = expr; return (result); } </pre>	<pre> bool CAS(&addr,expr1,expr2) { if (M[addr] == expr1) { M[addr] = expr2; return true; } else { return false; } } </pre>

Consider `test_and_set`. We can implement a simple spinlock:

```

int lock = 0;

void acquire(int *lock) {
    // wait until old value is 0 (FREE), set to 1 to claim lock (BUSY)
    while(test_and_set(lock));
}

void release (int *lock) {
    // set to FREE
    *lock = 0;
}

```

Of course, spinlocks involve busy waiting, which is not ideal. We can do better!

```

int guard = 0; // global variable, spinlock for checking actual lock
int lock = FREE;

void acquire (int *lock) {
    while(test_and_set(guard)); // short busy-waiting/spinlock
}

```

```

if (*lock == BUSY) {
    thread_state -> WAITING;
    guard = 0;
    sleep();
} else {
    *lock = BUSY; // claim lock
    guard = 0;
}
}

void release(int *lock) {
    while(test_and_set(guard)); // short busy-waiting
    if (wait_queue.size() > 0) {
        wait_queue.pop() -> READY; // first thread gets lock next
    } else {
        *lock = FREE;
    }
    guard = 0;
}

```

Aside: Fast Userspace Mutex (Futex)

`futex` is a syscall with the following signature (according to lecture):

```

int futex(int *uaddr, int futex_op, int val,
          const struct timespec *timeout);

```

`futex_op` can be...

- `FUTEX_WAIT` $\implies$ if `*uaddr == val`, sleep until `FUTEX_WAKE`, i.e. put on waiting queue associated with `uaddr`. In essence, this is an **atomic check** that the condition still holds after disabling interrupts in the kernel. (That is, the user should check this first before calling the `futex` to avoid context switching overhead). Otherwise, return immediately.
- `FUTEX_WAKE` $\implies$ wake up at most `val` waiting threads associated with `uaddr`'s waiting queue.

Essentially, a *conditional sleep* for threads, and can be used to implement synchronization primitives like locks, semaphores, etc. We will consider implementing a lock.

The naive attempt might look something like this:

```
int lock = 0;

void acquire(int *lock) {
    while (test_and_set(lock)) {
        // assert that lock is BUSY and let current thread sleep
        futex(lock, FUTEX_WAIT, 1);
    }
}

void release(int *lock) {
    *lock = 0;
    futex(lock, FUTEX_WAKE, 1);
}
```

This avoids spinning by inducing each thread to sleep if the lock is currently BUSY (via an atomic check). However, every `release` requires a syscall to wake up the next thread, even if no threads are waiting. Thus, we can extend this implementation slightly to avoid this overhead.

```
bool maybe = false; // if there *may* be threads waiting
int lock = 0;

void acquire(int *lock, bool *maybe) {
    while (test_and_set(lock)) {
        // sleep, since lock is busy; and note that there are threads waiting
        *maybe = true;
        futex(lock, FUTEX_WAIT, 1);

        // if woken up by release(), ensure next call to release knows there are
        // threads possibly waiting
        *maybe = true;
    }
}
```

```

}

void release(int *lock, bool* maybe) {
    *lock = 0;
    if (*maybe) { // if there may be threads waiting
        // no threads are considered to be waiting since the next thread dequeued
        // will be run
        *maybe = false;

        // try to wake up a thread
        futex(lock, FUTEX_WAKE, 1);
    }
}

```

This implementation is much better because it avoids any syscalls if the lock is uncontended.

Aside: Circular Buffer

We can implement a simple, spin-locked producer-consumer queue with these synchronization variables, notably.

```

mutex lock = FREE;

Producer(item) {
    acquire(&lock);
    while (buffer_is_full()) {
        release(&lock);
        acquire(&lock);
    }
    enqueue(item);
    release(&lock);
}

Consumer() {
    acquire(&lock);
    while (buffer_is_empty()) {
        release(&lock);
    }
}

```

```

        acquire(&lock);
    }
    item = dequeue();
    release(&lock);
    return item;
}

```

A better method, though, is to use semaphores!

```

Semaphore mutex = 1; // equivalent to regular lock for synchronizing access to
buffer
Semaphore fullSlots = 0;
Semaphore emptySlots = bufSize;

Producer(item) {
    semaP(&emptySlots);
    semaP(&mutex);
    enqueue(item);
    semaV(&mutex);
    semaV(&fullSlots);
}

Consumer() {
    semaP(&fullSlots);
    semaP(&mutex);
    item = dequeue();
    semaV(&mutex);
    semaV(&emptySlots);
    return item;
}

```

Aside: Monitor

Monitors are a *better abstraction* to semaphores.

A **monitor** is *1 lock* and *0 or more* condition variables for managing concurrent access to shared data.

Essentially, it's a generalization of locks/condition variables. Here's an example of an *infinite synchronized buffer* implemented with a monitor.

```
lock buf_lock;
condition buf_cv;
queue queue;

Producer(item) {
    acquire(&buf_lock);
    enqueue(&item);
    cond_signal(&buf_cv); // signal waiters that item was enqueued
    release(&buf_lock);
}

Consumer() {
    acquire(&buf_lock);
    while (isEmpty(&queue)) {
        cond_wait(&buf_cv, &buf_lock); // if no items, sleep
    }
    item = dequeue(&queue);
    release(&buf_lock);
    return item;
}
```

There are different types of monitors. This is just Mesa vs Hoare semantics, again.

Hoare monitors.

Signaling thread gives up lock *and* CPU to waiting thread, and waiting thread runs immediately. When the waiting thread finishes (releases lock), CPU and lock are returned to signaling thread. This allows for this convention:

```
if (someCondition(...)) {
    cond_wait(&cv, &lock);
}
```

This makes it easier to prove the program's correctness (*liveness*) and guarantees the condition holds on resumption. However, it is more complex to implement and, critically,

frequently context switches, reducing performance.

Mesa monitors.

Signaling thread keeps lock, waiting thread is placed on CPU's READY list. Waiting thread runs at some arbitrarily later point in time; this means the original condition, which was true at signal time, may not be true anymore! Thus, you must enclose `wait` calls in a while loop:

```
while (someCondition(...)) {  
    cond_wait(&cv, &lock);  
}
```

This is more performant due to less context switches and separates the signal from the condition check, which allows the decoupling of processes.

Aside: RW Locks

The problem is phrased as follows. We have a database that we'd like to allow concurrent access to. There are some readers and writers, but they cannot arbitrarily read/write to the database; otherwise, uncontrolled concurrency can introduce errors. In particular,

- Reader
 - Can read while other readers exist
 - Cannot read while there is a write in progress
 - Should wake up a waiting writer if there are no readers when it concludes
- Writer
 - Cannot write while there are reads/write in progress
 - Should wake up waiting writer upon conclusion, or waiting readers if no waiting writers

Here is an example solution.

```
int AR = 0; // active readers  
int WR = 0; // waiting readers  
int AW = 0; // active writers  
int WW = 0; // waiting writers  
condition can_read = NIL;  
condition can_write = NIL;  
lock lock;
```

```

char *Reader::read() {
    char *buf;

    acquire(&lock);

    // while >0 writers waiting/active, wait to read
    while ((AW + WW) > 0) {
        WR++;
        cond_wait(&can_read, &lock); // sleep until can maybe read
        WR--;
    }

    // now active!
    AR++;
    release(&lock);

    // read-only access
    read(buf);

    // checkout of system
    acquire(&lock);
    AR--;

    // if all readers are done and >0 waiting writers
    if (AR == 0 && WW > 0) {
        cond_signal(&can_write); // wake up a writer
    }
    release(&lock);

    return buf;
}

char *Writer::write(char* buf) {
    acquire(&lock);

    // while >0 readers/writers active, wait to write

```

```

while ((AR + AW) > 0) {
    WW++;
    cond_wait(&can_write, &lock);
    WW--;
}

// now active!
AW++;
release(&lock);

// exclusive access
write(buf);

// checkout of system
acquire(&lock);
AW--;

// if >0 waiting writers
if (WW > 0) {
    cond_signal(&can_write); // wake up a writer
} else if (WR > 0) { // else if >0 waiting readers
    cond_broadcast(&can_read);
}
release(&lock);
}

```

Aside: Language-Specific Synchronization

C

Well, they provide functions and structs for synchronization! Beyond that, though...

C++

Lock guards! In the destructor of a lock guard, it releases the associated lock. This allows for, e.g.

```

#include <mutex>

std::mutex lock;

void some_func() {
    std::lock_guard<std::mutex> lg(lock);

    ...

    return;

    // mutex released here, after the function returns
}

```

Python

The `with` keyword. e.g.,

```

import threading

lock = threading.Lock();
with lock: # automatically calls acquire()
    ...
# automatically releases once `with` statement is done

```

Java

The `synchronized` keyword. Every Java object has an associated lock. Lock is acquired on entry into and released on exit from each `synchronized` method. If an exception occurs in the method, it is automatically released.

Java also has support for monitors—every `synchronized` object has a condition variable associated with it. For waiting, there is `void wait()` and `void wait(long timeout)`. For signaling, there is `void notify()` and `void notifyAll()`.

Chapter 6: Multi-Object Synchronization

6.1 Multiprocessor Lock Performance

On a multiprocessor system, throughput may only be slightly faster than on a uniprocessor. This is typically a result of one of

- **Locking.** Exclusive access to data.
- **Communication of shared data.** Cache thrashing between processor caches, requiring communication of shared data between caches.
- **False sharing.** Due to the size of cache lines, there may two separate pieces of shared data in one cache line, inducing cache line bouncing between different processors when a piece of shared data is changed, despite not needing any communication.

Linux kernel locking evolution. Symmetric multiprocessing (SMP) was first added in version 2.0. To avoid concurrency issues, Linux added the *Big Kernel Lock (BKL)*, which protected all shared data structures in the kernel. This allowed Linux to run on a multiprocessor machine, while more fine-grained SMP support was still being developed. As of version 2.6, the BKL is deprecated, and removed from most kernel subsystems.

6.2 Lock Design Patterns

Four design patterns

- **Fine-Grained Locking.** Partition object state into more fine-grained subsets, each with its own lock.
- **Per-Processor Data Structures.** Partition object state so most accesses are performed by threads on the same processor. This provides better hardware cache behavior, as it reduces the amount of communication between processors. However, the partitioning may result in decreased cache effectiveness, (e.g. distributed database cache, when machine X has a result cached and machine Y doesn't, and a request comes to machine Y).
- **Ownership Design Pattern.** Remove shared object from shared container on use to ensure mutually exclusive access. This works best when there is, e.g., queue of objects that must have work performed on each in stages.
- **Staged Architecture.** Divide system into multiple stages, so only threads within each stage can access that stage's shared data. This increases modularity, making it easier to reason about code, and improves cache locality for each thread. Note, however, that the processing in each stage must be expensive enough such that the cost of communication between stages is amortized. Also, staged architectures' throughput is limited by the throughput of the slowest stage, which can result in problems when a system is overloaded. (One solution is to dynamically allocate resources/threads to stages).

Commutative interface design. The UNIX open syscall API specifies that the syscall returns the next open file descriptor (ascending order). This forces open to use a lock, as two calls to open one after the other must return $x, x + 1$ (or something similar in ascending order). A *commutative* API ensures that the result of two calls is the same regardless of which call was made first. For open, this would require returning any unique integer not currently associated with a file descriptor, and thus would not need a lock (outside of handling stdin, stdout).

6.3 Lock Contention

6.3.1 MCS Locks

The Mellor-Crummey-Scott (MCS) lock is an implementation of a spinlock optimized for the case when *there are many waiting threads*. On a multiprocessor system, acquiring and releasing a lock is associated with a much higher overhead, since the lock and shared data must be transferred between all waiting processors. Moreover, each thread attempting to acquire a lock queues an atomic RMW instruction, and the hardware has no way to know which one to prioritize (the release instruction!).

A naive solution is *test and test-and-set*. In essence, the spin lock is implemented as

```
while(lock == BUSY || test_and_set(&lock));
```

However, this doesn't help. The lock's release forces the previously holding processor to communicate the updated lock state to the waiting processors. This is an issue if a lock's critical section is typically short, as most time is spent on inter-processor communication.

A more scalable solution is to assign each thread *its own memory location* to spinlock on, and releasing the lock sets a bit for *one thread's* memory location, instead of letting all waiting threads know. Almost like a CV's signal instead of broadcast for spinlocks.

MCS is a popular implementation of this, that essentially maintains a queue of waiting threads. Waiting threads add themselves to the tail of the queue, and spin on the flag in its queue entry. The thread releasing the lock will set the flag for the first entry in the queue. This makes it much more efficient when there are many waiting threads, as the lock is passed efficiently along the threads in the queue. The overhead of the queue makes MCS less efficient with few waiting threads.

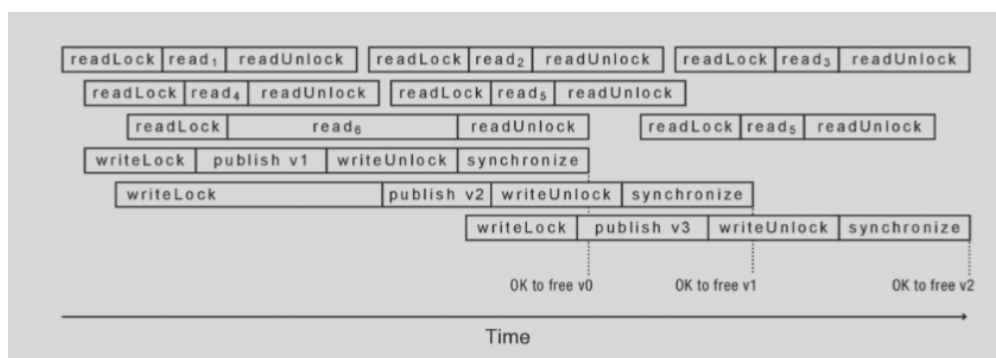
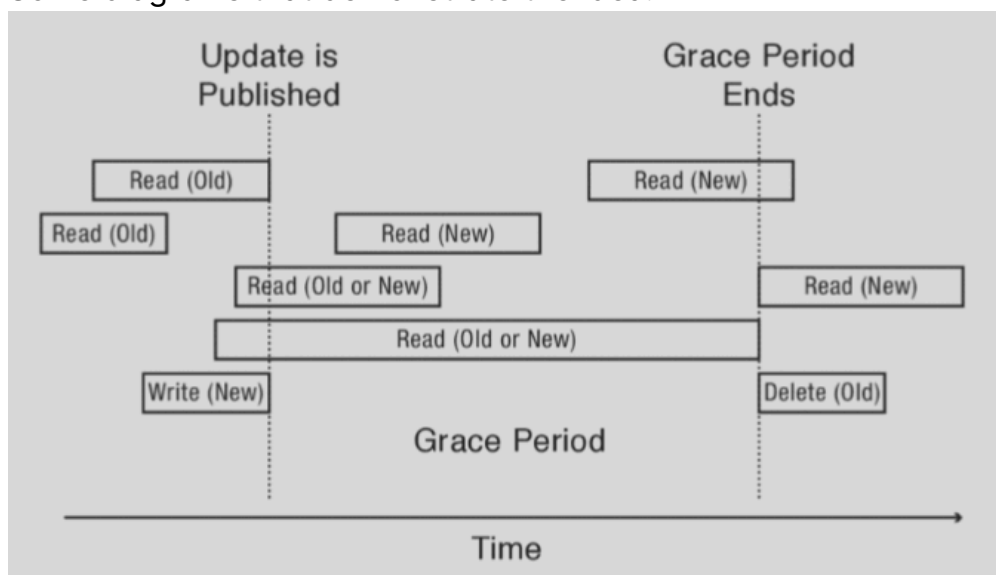
6.3.2 Read-Copy-Update (RCU) Locks

RCU locks are a read/write-lock that is highly optimized for read operations, while sacrificing write operations.

RCU makes three important changes to the standard RW lock:

- **Restricted update.** A writer thread *publishes* its changes to the shared data structure with a single atomic memory write. Typically, this is done by the writer thread (1) copying the data structure, (2) making changes, and (3) publish the corresponding pointer (by updating a shared location) to point to the new version.
- **Multiple concurrent versions.** There can be multiple versions of the shared data structure being read at one time. In essence, read-only critical sections can be reading different versions of the shared data structure, depending on when they started executing.
- **Integration with thread scheduler.** The RCU lock only frees an old version once it is confident no reader is still accessing it. Essentially, it waits until all readers accessing the version have released the read lock. The time between a new update being published and the last reader finishing is the *grace period*.

Some diagrams that demonstrate the idea:



As you may notice from the second figure, writes must still be serialized. In particular, their `publish` phases must execute in order (otherwise, versioning would not be linear). However, any number of concurrent reads may occur during a write.

The writers must also *synchronize*, i.e. wait until all readers are finished before freeing the previous version. To do this performantly, RCU locks often interface with the scheduler, where the scheduler has some function `quiescentState()` that informs the waiting writer that the old version is *quiescent*, i.e. no reader has access to it.

6.4 Multi-Object Atomicity

Sometimes, it is necessary for there to be concurrency-safe (essentially atomic) interactions *between* shared objects. There are some solutions to consider for this problem:

- **Careful Class Design.** Simply expose only APIs that allow concurrency-safe object interaction. This can make design more complex, however.
- **Acquire-All/Release-All.** A thread acquires/releases all locks associated with a set of operations/shared objects it must access for a critical section. This also adds *serializability* to programs, in which requests to different shared objects are guaranteed to be performed sequentially due to the locking mechanisms. However, this solution presents a problem—it's not always immediately obvious what locks are needed for each request.
- **Two-Phase Locking.** Two-phase locking involves acquiring only the locks necessary at each point. For instance, `f1` needs `lock1` and `f2` needs `lock2`. In some cases, `f1` calls `f2`, in which case `lock2` is acquired later.

6.5 Deadlock

A deadlock is simply a *waiting cycle* among threads. The simplest example occurs in *mutually recursive locking*, i.e. thread 1 acquires A then B, while thread 2 acquires B then A. Another example is *nested waiting*, i.e. thread 1 acquires two locks, releases one with a call to a CV's `wait`, but other thread that will `signal` CV needs the other lock.

Deadlock vs. Starvation

Starvation: a thread fails to make progress for an indefinite period of time.

Deadlock: starvation, but no threads within a group can make progress ever.

There are four necessary conditions for deadlock.

- **Bounded resources.** There are a finite number of threads that can simultaneously use a shared resource.

- **No preemption.** Once a thread acquires a resource, its ownership cannot be revoked until it releases it.
- **Wait while holding.** A thread holds one resource (lock) while waiting for another. Also known as *multiple independent requests*.
- **Circular waiting.** Exactly what it sounds like.

Note, though, that these conditions are necessary *but not sufficient* for deadlock.

There are a number of ideas to help prevent deadlock.

- **Exploit/limit program behavior.** In other words, change program behavior to never satisfy one of the above conditions, e.g.
 - Provide sufficient resources. For instance, threads often reserve space in the TCB for insertion into ready/waiting lists.
 - Preempt resources. Allow runtime to forcibly reclaim thread resources.
 - Release lock when calling out of module (i.e., a section protected by a lock). In other words, no nested locking.
 - Lock ordering. Ensure locks are only ever acquired/released in the same order, whenever they are used. This prevents circular waiting/mutually recursive locking from ever occurring.
- **Predict the future.** If we know what threads may/will do, avoid deadlocking by waiting now.
- **Detect and recover.** Detect deadlocking, and restore a previous state.

6.5.4 Banker's Algorithm for Avoiding Deadlock

Banker's Algorithm is a procedure developed by Dijkstra that improves on the performance limitations of acquire-all. In the algorithm, a thread states its maximum resource requirements upon beginning a task, and acquires/releases resources incrementally as the task runs. Banker's algorithm uses this knowledge of each thread's maximum resource requirements to ensure that, if a request is added, all threads will eventually complete, via simulating the resource allocations. If a request comes in that would result in an *unsafe* state, i.e. could potentially lead to a deadlock, it will delay that request.

6.5.5 Detecting and Recovering from Deadlocks

There are two primary approaches:

- **Proceed without the resource.** This is typically seen in systems where maintaining low latency is paramount. For instance, say a customer orders on Amazon, but some deadlock

prevents the inventory from notifying the client of if the item is in stock. The client may simply give up, queue a check to be run later, and continue with normal web operations. If the item was in fact out of stock, the customer is notified later with an apology, rather than having the website hang.

- **Transactions.** Intel Transactional Synchronization Extensions (TSX) are a good example. When deadlocks are detected, the thread (1) rollbacks to a clean state, (2) stops/rollbacks one or more victim threads, (3) lets other threads process, and (4) lets the victim threads finally complete. (Intel TSX actually does this slightly differently, allowing execution without any locking that rollbacks if any contention is detected). During a transaction, the results are not visible to any other thread; upon success, the transaction will *commit* its result, and the changes to the shared state will become visible.

One decision to make with transactions is *which* thread to abort. A suggested solution is *wound wait*, where the youngest transaction is aborted. This ensures liveness/avoids starvation, as eventually the oldest transaction will complete, and so on.

6.6 Non-Blocking Synchronization

In essence, non-blocking synchronization is the idea of writing concurrent programs entirely without locks. This eliminates lock contention and deadlocks, but can make programs much more complex and dangerous to design. In particular, *wait-free* data structures guarantees progress for every thread, regardless of other threads' states, while *lock-free* data structures guarantee progress for at least one thread. In essence, wait-free implies lock-free, and lock-free can cause starvation.

Aside: Handling Deadlocks

162 defines 4 ways of handling deadlocks:

- **Denial.** Pretend it doesn't exist. Applicable in situations where deadlock is rare and, in the case of a deadlock, the system can simply be restarted.
- **Prevention.** Write systems that don't satisfy all four deadlock conditions.
- **Recovery.** Let deadlocks happen, and recover from them.
- **Avoidance.** Dynamically delay resource requests so deadlock doesn't happen, e.g. Banker's algorithm.

Chapter 7: Scheduling

7.1 Uniprocessor Scheduling

Notation

- Workload: a set of tasks for some system to perform, where each task has an arrival date and completion duration
- Compute-bound: describes a task whose completion duration is limited by the CPU processing power
- I/O-bound: describes a task whose completion duration is limited by waiting for I/O
- Work-conserving: describes a scheduling policy in which the processor is never left idle if there is work pending

Metrics

- Average response time (minimize)
- Variance of average response time (minimize)
- Fairness

FIFO

First-In-First-Out scheduling. Exactly how it sounds, also known as First-Come-First-Served (**FCFS**). *Average response time* can be *extremely high* due to head-of-line blocking (a.k.a. **convoy effect**) from a very long task. Note that the convoy effect also implies potential **starvation** of later tasks!

SJF

Shortest Job First scheduling. The preemptive version is known as Shortest-Remaining-Time-First (**SRTF**). Always runs the job that will end the soonest. For SRTF, a short task arrives while a long task is running, and the short task will finish before the long task finishes, the short task will preempt this task. It turns out that SRTF is precisely the *optimal* scheduling policy for minimizing *average response time*. However, it is also the *pessimal* policy for minimizing *variance*. Also, note that SJF is not feasible in the real-world, since it requires knowledge of when each job will end, which is not known a priori. SJF can also suffer from *starvation* and *frequent context switches*. It's also an *unfair* scheduler, and programs can game the scheduler by submitting itself in small segments. (Note: SJF can still suffer from head-of-line blocking without preemption).

RR

Round Robin rotates through all the tasks in order. Each task is allowed to run up to some threshold of time, known as a **time quantum**. When the time quantum elapses or the task finishes/yields the processor, the next task is scheduled on the processor. RR is most effective at *preventing starvation*. However, the time quantum must be chosen carefully—in particular, the time quantum should be longer than the context switching overhead, but reasonably short so that tasks will not wait too long until their turn. Also, short time slices can negatively affect the cache, as tasks evict each previous tasks' cached data.

Simultaneous Multi-Threading (Hyperthreading)

SMT is a mechanism of simulating two virtual/logical processors on a single physical core, alternating between them on a cycle-by-cycle basis. This helps maximize the use of each physical core, especially since threads frequently need to wait for memory at times. So, at the hardware level, we can technically have zero-overhead context switches!

RR does have weaknesses beyond just the static time quantum tunable. For a workload with several tasks that start at the same time and are the same length, RR will finish all tasks at roughly the same time. This is pessimal in average response time! Additionally, the context switching adds overhead without any benefit here.

RR is also poor when running a mixture of I/O-bound and compute-bound tasks. In these instances, the I/O-bound tasks will yield the processor frequently, whenever I/O blocks. This results in them spending much less time on the processor than their given time quantum, each time they are scheduled. Moreover, they are scheduled much later in the future, as the processor must finish letting the rest of the tasks run before getting back to the I/O-bound task. This results in the I/O-bound task being very slow to complete.

Max-Min Fairness

Fair allocation of resources is also important for scheduler design, particularly for multi-user machines or datacenters. Essentially, max-min fairness is a scheduling algorithm to fairly allocate a single resource. Max-min fairness iteratively increases the minimum allocation of any process until the resource is fully assigned (attempts to maximize minimum allocation). For instance, if all processes are compute-bound, max-min fairness will give each process an equal share of the processor, i.e. RR. If some processes cannot use their entire share, then the unused portion is redistributed equally to other processes.

Also, note that the scheduler cannot switch to the process with the least time so far at every tick—this would be incredibly inefficient, due to context switches. Instead, we take inspiration from RR, and allow each process to get ahead of its fair allocation by *one time quantum*.

Note that, for multi-resource fairness, schedulers like [Dominant Resource Fairness](#) can perform better in terms of fairness.

Weighted Max-Min Fairness

Weighted Max-Min Fairness is a variant of Max-Min Fairness in which each user is assigned a weight, with higher priority users receiving higher weights. Then, user k gets an allocation of $\frac{w_k}{\sum_i w_i}$ of the resource (scheduler).

MFLQ

Multi-Level Feedback Queue is designed with several goals in mind.

- **Responsiveness.** Run short tasks quickly.
- **Low Overhead.** Minimize preemption, and minimize scheduler algorithm overhead.
- **Starvation-Freedom.** All tasks should make progress.
- **Background Tasks.** Defer system maintenance tasks so they don't interfere with user work.
- **Fairness.** Assign non-background processes approximately their max-min fair share.

MLFQ has multiple queues, typically each running RR (though there can be heterogeneous scheduling policies), of tasks, each with a different priority level and time quantum. Higher priority tasks run before lower priority tasks, and have shorter time quanta. If a task uses up its time quantum, it drops a level. If a task yields the processor to wait on I/O, it stays at the same level or is upgraded a level.

This does not quite achieve starvation-freedom or max-min fairness. To address this, the operating system can maintain two queues—one of tasks who have received their fair share, and another of those who have not—and/or periodically promote processes that have not received their fair share yet to higher priorities.

7.2 Multiprocessor Scheduling

7.2.1 Scheduling Sequential Applications on Multiprocessors

A potential solution is a centralized MLFQ, with a lock to prevent contention issues. However, this has several performance problems:

- **Lock Contention.** If threads yield the processor frequently (i.e. lots of I/O), the lock could be highly contended, especially as processors increase.
- **Cache Coherence Overhead.** Due to the lock, the MLFQ data must be transferred between processor caches, causing additional overhead. In particular, this is done while holding the MLFQ lock, resulting in even more lock contention.
- **Limited Cache Reuse.** Since the MFLQ schedules threads to be run on the first available processor, a thread that previously ran on processor 1 might end up being scheduled to run on processor 2 next. This reduces the benefits of each processor's cache, and, moreover, data will need to be fetched from processor 1's cache into processor 2's cache.

Therefore, modern operating systems use a *per-processor* MLFQ. These processors use **affinity scheduling**—once a thread is scheduled on a processor, it is always returned to the same processor when rescheduled, to increase cache reuse. Rebalancing the processors' MLFQs occurs periodically, only if the imbalance is large enough to compensate for the time to reload processor caches for each thread.

7.2.2 Scheduling Parallel Applications

Oblivious Scheduling

Oblivious scheduling describes an operating system that schedules threads while *oblivious* to the parallelization of the actual applications. Unfortunately, oblivious scheduling can be problematic for several reasons.

- **Bulk synchronous delay.** Commonly, parallel programs split works into equal sized chunks, and then synchronize at some barrier in the program. However, this results in the program relying on the latest finishing task, i.e. tail latency. Oblivious scheduling does not take this into consideration, and thus tail latency may be exorbitantly high.
- **Producer-consumer delay.** Some programs have a producer-consumer design pattern, in which the results of a thread are fed into the next thread, and so on, in sequence. Preempting a thread in this chain can stall all processors in the chain.
- **Critical path delay.** Most parallel programs have a *critical path*—minimum sequence of steps for the application to compute its result. Preempting a non-critical thread is okay, but preempting a thread on the critical path slows down the end result.
- **Preemption of lock holder.** Threads that are waiting for lock holder cannot acquire the lock. Frequently, parallel program will spin briefly to see if a lock will be released, before yielding back to the processor. This results in non-negligible overhead.
- **I/O.** Parallel applications frequently do I/O, which results in threads yielding to the processor. To keep the processor busy, it's desired to allocate more threads than processors to keep the processors busy. If the thread does not block on I/O, though (e.g.

file read, contents are cached), then the processor has to multiplex threads onto processors—resulting in the above issues.

Gang Scheduling

One potential solution is gang scheduling, in which the threads of a single task/program are either scheduled together or not at all. This is especially helpful for programs with barriers. Applications can also *pin* threads to a specific processor and mark it as high priority. This allows reserving a majority of processors for a parallelized application.

However, gang scheduling can be inefficient when there exist multiple parallel programs. Instead, it's more efficient to divide the processors into two groups, and pin each program to one group *instead* of time slicing. This minimizes processor context switches, and is known as *space sharing*.

Scheduler Activations

Applications are essentially given *execution context* regarding each processor assigned to the application via an *upcall*. This allows the application to handle its own thread management, i.e. user-level thread management, and decide how it multiplexes its threads onto its assigned processors.

7.3 Energy-Aware Scheduling

There are several different optimizations to consider

- **Processor design.** Processors designed for lower clock speeds can tolerate lower voltage swings at the circuit level. Some systems have chosen to have both a high power, high performance multiprocessor and a low power, low performance uniprocessor on the chip.
- **Processor usage.** Idling processors will, of course, draw much less power. Also, parallel programs have diminishing returns from extra processors; thus, a tradeoff between faster execution and lower energy use.
- **I/O device power.** Devices in use cannot be turned off. Devices such as WiFi and networking interfaces consume a lot of power; it's optimal to only turn them on when necessary.

Heat dissipation

Heat dissipation is a closely related topic. In particular, the operating system must ensure it never surpasses a certain temperature threshold.

Something important to note is a user's perception of an operating system's responsiveness. There naturally exists a limit to human perception. But, beyond this limit, the perceived delay

for user increases at a logarithmic rate. We can take advantage of this to form a three prong strategy:

- **Below the threshold of human perception.** Optimize for energy use to the extent that the user does not notice.
- **Above the threshold of human perception.** Optimize for response time if the user already will notice slowdown regardless.
- **Long-running or background tasks.** Balance energy and responsiveness depending on available battery.

7.4 Real-Time Scheduling

Occasionally, the operating system scheduler must account for process deadlines. In these situations, a task is *only useful* if it is completed before a certain deadline. There are known as **real-time constraints**.

A scheduler should first consider any real-time task as a higher priority than any less time critical tasks. Beyond that, there are three techniques to increase the likelihood of tasks meeting deadlines:

- **Over-provisioning.** Quite simply, ensure real-time tasks use only a fraction of total resources. Thus, there's always space to quickly schedule real-time tasks.
- **Earliest Deadline First. (EDF)** This scheduling policy would simply schedule the task with the earliest deadline first. This can result in issues, though, especially if one task has I/O that it must wait for, but is scheduled later because its deadline is later. This limitation can be somewhat mitigated by breaking tasks into shorter units, with their own deadlines. Also, EDF won't work if you have too many tasks, i.e. it's infeasible to complete all the tasks!
- **Priority donation.** There can be issues with shared data structures, though. For instance, a low priority task may be holding a lock, but then is preempted by a medium priority task. Then, a real-time task preempts the medium priority task. If the real-time task then tries to access the shared data structure, only to realize the lock is being held, it will yield to the other tasks. However, the low priority task will not start running yet, since the medium priority task is still running; hence, the lock will not get released until much later. This is called **priority inversion**, i.e. when a high priority task must wait on a lower priority task. This can be solved via *priority donation*, i.e. a higher priority task donates priority to a lower priority task.

Note that there also exists a distinction between **hard** and **soft** real-time constraints. A hard real-time constraint might exist in a car's braking system, where the deadline must be hit (and otherwise completing the task has no value), while a soft real-time constraint might exist for a

streaming service, where it's best to hit some deadline, but not absolutely necessary. Note also the distinction between a *static* and *dynamic* scheduling policy—a dynamic scheduling policy can change task priorities (priorities set during runtime), while a static scheduling policy cannot change task priorities once set (priorities set before runtime).

For *hard* real-time constraints, the following scheduling algorithms have seen use:

- Earliest Deadline First (**EDF**) — dynamic scheduling policy (described as above, TL;DR chooses earliest absolute deadline), with priorities that can change as deadlines approach; can deal with periodic and aperiodic tasks. (Note how it is dynamic because it may adjust priorities and preempt a task if a new task with an earlier deadline arrives).
- Deadline-Monotonic Scheduling (**DMS**) — static scheduling policy, tasks with shorter relative deadlines are given higher priorities
- Rate Monotonic Scheduling (**RMS**) — static scheduling policy, tasks with shorter periods are given higher priorities
- Least Slack/Laxity Time First (**LSTF**) — dynamic scheduling policy, tasks with less "slack time" ($d - t - s$, where d = deadline, t = current time, s = remaining service time of task) are given higher priorities; in other words, how much time of *not running the task* until the task would miss its deadline. (Note how it is dynamic because slack time of tasks decreases over time).

For *soft* real-time constraints, the following scheduling algorithms have seen use:

- Constant-Bandwidth Server (**CBS**) — for every period/time quantum, the system *reserves* a certain amount of time for the task that is guaranteed to it (ideally, the task's requested time). Moreover, if there is leftover idle time, CBS allows tasks to execute earlier and consume *future* reserved time. It is frequently used in conjunction in EDF (at least, it is in the Linux kernel). How exactly? idk—haven't read through kernel source code yet :P

7.5 Queueing Theory

7.5.1 Definitions

- **Server**. A server is anything that performs tasks.
- **Queueing delay (W) and number of tasks queued (Q)**. Queueing delay of a task is total delay waiting to be scheduled.
- **Service time (S)**. Time to complete a task on a processor.
- **Response time (R)**. $R = S + W$.

- **Arrival rate (λ) and arrival process.** The average rate at which new tasks arrive, and how bursty the arrivals are.
- **Service rate (μ).** The number of tasks the server can complete per unit of time when there is work to do. $\mu = \frac{1}{S}$.
- **Utilization (U).** The fraction of time the server is busy, $U = \min\left(\frac{\lambda}{\mu}, 1\right)$.
- **Throughput (X).** $X = U\mu$.
- **Number of tasks in the system (N).** $N = Q + U$.

7.5.2 Little's Law

Little's Law applies to *any stable system* where the arrival rate matches the departure rate ($\lambda \leq \mu$). In essence, it states that $N = XR$.

7.5.3 Response Time vs Utilization

Best Case: Evenly spaced arrivals

- $\lambda < \mu \rightarrow$ no queueing, $R = \mu$.
- $\lambda = \mu \rightarrow$ if queues are empty, they remain empty. if queues are full, they remain full.
- $\lambda > \mu \rightarrow$ the queues will grow without bound / system is not equilibrium $\implies$ steady state response time is undefined.

Worst case: Bursty arrivals.

In general, increases response time in every case due to forced queueing, even if $\lambda \ll \mu$.

Typically, since most systems lie between these two cases, it's useful to model queueing systems with an *exponential distribution*. Essentially, a queueing system behaves as a finite state machine; at each change, the queue length increases or decreases, each with some associated probability (in particular, λ and μ , respectively). We can thus model the average response time as $R = \frac{S}{1-U}$.

7.6 Overload Management

The key idea is to *design your system to do less work when overloaded*. This is because, when overloaded, your system cannot feasibly serve all requests.

One step is to simply reject some requests. This preserves reasonable response time for the remaining requests, and is often applied for, e.g., streaming applications. Another step is to reduce service time per request. This might involve switching to a slightly lower quality service, e.g. reducing bit rate in streaming, to be able to serve the overload. Yet another step

is to return requests early, and update later if the request processing actually returned a different result, if the distribution significantly favors a specific result.

Many systems do the exact opposite, though. Highway traffic throughput is limited significantly when there are many cars, due to stop-and-go traffic. Time-slicing also has this issue, due to fewer cache hits from rotating between many different tasks. Naive network congestion also runs into this issue, as the sender will retransmit packets dropped due to overload, further adding to the load. Though, TCP congestion control is designed precisely to deal with this.

Aside: Lottery and Stride

Lottery: Randomized

Each task/thread is given some number of lottery tickets, with higher priority threads being given more tickets. Each thread receives at least one ticket, to prevent starvation. Then, at each time slice, a lottery ticket is drawn from the remaining pool, and the corresponding thread is scheduled on the processor.

Stride: Deterministic

Stride is the deterministic version of a lottery scheduler. Like the lottery scheduler, each task is assigned some number of tickets. Then, each task is given a **stride**, which is inversely proportional to the number of tickets. This is typically calculated as $s_i = \frac{W}{n_i}$, where W is a really large number and n_i is task i 's number of tickets. On every time slice, the task with the lowest **pass** is chosen. Pass starts at 0 for all tasks, and is incremented by the task's stride every time that task is chosen. Essentially, for threading, it tracks how long each thread has been scheduled so far. Notably, a smaller stride (more lottery tickets) will mean that the thread will run more frequently.

Aside: EEVDF

Fluid Flow Systems

Fluid flow systems can be likened to the wires in a network context. Each wire has a bandwidth; this is known as c , the capacity, in a fluid flow system. Similarly, each connection traversing this wire has a packet rate; this is known as r_i , the flow arrival rate. For each connection, it's also assigned a link fair rate, f , which can be computed as $\frac{c}{n}$. Then, for fairness, each separate flow is assigned an allocation of the link, which is equivalent to

$\min(r_i, f)$. We can also have weighted fairness, though, in which $f = \frac{c}{\sum_i w_i}$ and each packet is assigned an allocation of $\min(r_i, f \cdot w_i)$.

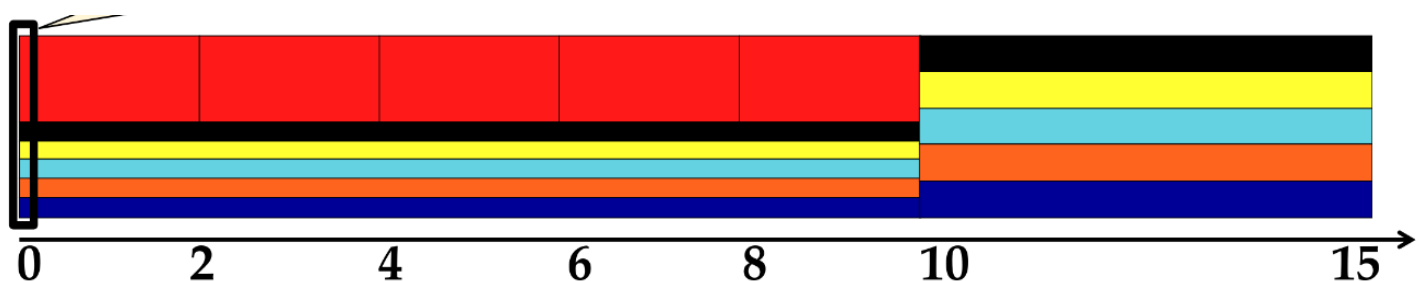
GPS

Generalized Processor Sharing is the theoretical analogue of fluid flow systems in processor scheduling. Essentially, a processor acts as a *link* and the threads sharing the processors act as *flows*. The ideal situation involves having every thread running at the same time, given exactly their flow allocation of the processor. Of course, this is not possible, as (ignoring hyperthreading) only one thread can run on the processor at a time. Instead, for GPS, we require that each thread should receive their proportional share of *time* on the processor. In essence, the **service time** of a task i between times t_0 and t_1 is $S(t_0, t_1) = f_i(t_0) \cdot (t_1 - t_0)$, where $f_i(t) = \frac{w_i}{\sum_j w_j}$.

WFQ

We construct an algorithm known as **Weighted Fair Queuing** on top of the GPS model. In essence, we utilize **weighted round robin (WRR)**, in which the quantum of each task corresponds to its weight! This successfully approximates GPS—every period of $\sum_i w_i$ units of time, the processor will have done the same work as GPS. What WFQ is doing, essentially, is just choosing the task that finishes first in GPS within each time period, *provided no other tasks are added to the system*.

This following visualization of GPS may help show this dynamic of a task finishing first. Note that, here, the red task has a weight of **5**, while the others have weights of **1**.



Virtual Times

If other tasks are added into the system, the finishing times of each task may change! But, critically, the order in which tasks finish never changes (excluding the newly added task, of course, since it can be arbitrarily short/long). Thus, rather than scheduling tasks according to the physical finishing time, we track the *number of WRR rounds needed to finish the task*. This is known as the **virtual finishing time**. In essence, virtual time measures the service needed to

complete a task. Note also that the system virtual time is thus the *index* of the round in the WRR scheduler.

More formally, let $V(t)$ denote the system virtual time. Then, the slope of $V(t)$, when graphed against t , represents the "normalized rate at which every task receives service in the fluid flow system." Naturally, the rate of $V(t)$ scales *inversely proportional* to the sum of the weights of all these tasks. In particular, $\frac{dV}{dt} = \frac{1}{\sum_i w_i}$. Note that $\sum_i w_i$ is equivalent to **1 virtual time unit**. Then, in the ideal situation, for every time unit, each task should run exactly once (as it would in WRR).

In other words, as more tasks enter the system, the *virtual time slows down!!* Therefore, while every task's virtual finishing time stays the same, the virtual time progresses more slowly for each time quantum. In other words, $\frac{dV}{dt}$ decreases.

We can also calculate the virtual time from 0 to t_1 as tasks are added/removed from the system. Just solving the differential equation above for V provides

$$V(t_1) = \int_0^{t_1} \frac{1}{\sum_i w_i} dt$$

Since tasks enter/exit the system at discrete times, we can split this integral up into multiple integrals, with a new one each time the set of tasks changes (i.e. one per interval where the set of tasks is constant).

EEVDF

Earliest Eligible Virtual Deadline First is an algorithm that builds on top of these stated ideas. Like EDF, it schedules the task with the earliest deadline. However, unlike EDF, it uses *virtual* deadlines instead of physical deadlines, and only considers tasks that are *currently eligible*, where eligibility is calculated from a task's previously served requests.

Virtual time is essentially a representation of time *weighted* by the threads' weights. An increment of t in real time corresponds to an increase of $\frac{t}{\sum w_i}$ in virtual time. In essence, you can think of it as a *weighted average* of all threads' progress. Then, if request r from thread i takes t real time to complete, it should expect to complete it by within a duration of $\frac{t}{w_i}$, if provided its fair share.

① The below was written using the Linux explanation of EEVDF, and represents the real-world implementation. Might help to skip this and come back to it after reading through the rest. (i.e. this might confuse you...)

To understand eligibility, consider how EEVDF would handle two threads, T_1 and T_2 . WLOG, in the first time period, let T_1 run for more time than it should have, and T_2 run for less time than it should have. Then, EEVDF calculates the **lag** of each thread, i.e. the difference between the expected service time and the actual service time. A positive lag denotes a thread that did *not* receive its fair share, and a negative lag denotes a thread that received *more* than its fair share. Then, for the next time period, only threads that currently have negative lags are considered **eligible**. In other words, T_2 is the only eligible thread, and will be scheduled since there are no other threads.

We can also calculate the **virtual deadline** of each thread, since we seek to select the thread with the earliest virtual deadline among those eligible. The virtual deadline denotes the earliest virtual time at which this thread will be fully serviced. First, we note a new term—**eligible time**—which denotes the earliest time at which a thread becomes eligible. This is just equivalent to $\min(-\text{lag}, 0)$ (relative to the current time, i.e. virtual). Then, the virtual deadline of a thread is just the sum of its **virtual eligible time** and **virtual service time**. Thus, EEVDF effectively balances fairness (lag/eligible time) and the priority of each thread (virtual time, which is weighted).

More formally, we can write several recurrence relations that define the parameters of EEVDF.

- Recall that $S_i(t_0, t_1)$ denotes the virtual service time of a thread i from t_0 to t_1 .
- Let t_0^i denote the time of the 1st request made by thread i .
- Let $ve(k)$ denote the virtual eligible time of the k th request.
- Let $vd(k)$ denote the virtual deadline of the k th request.
- Let $r(k)$ denote the request length of the k th request. This represents the **ideal (with regards to fairness), real** service time the thread should receive for this request between the virtual eligible time and virtual deadline, i.e. $r(k) = S_i(ve(k), vd(k))$.
- Let w_i denote the weight assigned to thread i .

Then, we can write the following recurrence relations/equations.

$$ve(1) = V(t_0^i) \quad (1) \text{ Base Case}$$

$$vd(k) = ve(k) + \frac{r(k)}{w_i} \quad (2) \text{ Virtual Deadline}$$

$$ve(k+1) = vd(k) \quad (3) \text{ Virtual Eligible Time}$$

Let's proceed through this step-by-step.

(1) First, the virtual eligible time of the first request is defined as the $V(t_0^i)$, which is essentially the virtual time at which this thread makes its first request.

(2) The virtual deadline of the k th request is defined as the sum of the virtual eligible deadline and the virtual service time. Threads with a heavier weight will be assigned an earlier virtual deadline, since $\frac{r(k)}{w_i}$ will be smaller. In other words, threads with a higher priority will expect to finish sooner. Note that dividing $r(k)$ by w_i essentially **converts it to virtual service time!!**

⚠ The division by w_i to convert to virtual service time is critical to understand for EEVDF, and how these formulas really fairly weight threads. Make sure this makes sense before moving on!

(3) The virtual eligible time of the $k + 1$ th request is equivalent to the virtual deadline of the k th request. The updates for the $k + 1$ th request only occur when the k th request finishes! This may take longer than one quantum.

The above is also **very important** to understand for the fairness of the *theoretical/ideal* EEVDF. Consider a request that finishes—its next virtual eligible time is updated to its previous virtual deadline, which represents the virtual time it should finish at in the idealized model of a fluid flow system. However, even if it finishes earlier than this, it should not be scheduled until after its previous, fair virtual deadline has been reached! This may be difficult to understand, so let's consider an example situation. There are two threads, each with equal weights of 1 and requests of length 1. They enter the system at the same time. Then, both start with an eligible time of 0 and a deadline of 1. As aforementioned, it's not possible to have generalized processor sharing on a processor; thus, one of these threads must finish first. Let's say we schedule thread 1 first; it will finish its first request at virtual time 0.5. Then, its new virtual eligible time will be updated to 1. **Why?** Because it has already used up its fair time share in the time slice/quantum 0 – 1. Therefore, it should not be scheduled again until virtual time has reached at least 1, when it would be fair to schedule it again.

In a real-world CPU scheduling algorithm, step 3 changes slightly because **lag** can occur, since we do not know *a priori* how long a task will run for. Every time a quantum q elapses, we compute $ve(k + 1) = ve(k) + \frac{u(k)}{w_i}$, where $u(k)$ is the **actual** real service time that the request received. Note how, if $u(k) > r(k)$, the eligible time of this request gets pushed back more than it would have originally been. This is because the request received more time than it should have, i.e. negative lag, and thus its unfairly receiving more time. Similarly, if $u(k) < r(k)$, the eligible time of this request gets pushed back less than intended. This is compensating for positive lag, and ensuring this request gets scheduled sooner since it didn't receive as much time as expected.

Aside: Linux $O(1)$ Scheduler

This scheduler was used in the Linux kernel until version 2.6.

Note that the Linux $O(1)$ scheduler is a priority-based scheduler, with **nice** values of $[0, 100)$ reserved for the kernel and $[100, 140)$ reserved for the user. A lower nice value means a higher priority, and vice versa. The priority of each task determines its timeslice, like an MLFQ. Notably, there are also two separate task queues: an **active** and **expired** queue. Tasks that use up their timeslice are placed in the expired queue; once the active queue is empty, the queues swap.

The scheduler also leverages several $O(1)$ heuristics, including:

- User task nice value adjusted ± 5 based on `p->sleep_avg = sleep_time - run_time`. A higher sleep average corresponds to a more I/O bound task, and are provided higher priorities (lower nice values).
- Interactive credit: $+1$ when a task sleeps for a long time, -1 when a task runs for a long time. Tasks with high interactive credit may be placed back on the ready queue. Note that it can be negative! It's used to avoid impacting the responsiveness of interactive tasks by understanding **one-off behaviors** of tasks. This is important in the following scenarios:
 - Low interactivity task enters uninterruptible sleep; `p->sleep_avg` is high after exiting sleep. However, it's not actually interactive; it should not be prioritized by the scheduler. With interactive credit, it will have spent enough beforehand such that this one long sleep will not impact its interactive credit much $\Rightarrow$ scheduler knows this is a task that runs for a long time on processor (i.e. is not interactive).
 - High interactivity task uses up a lot of CPU time once. Again, its high interactive credit means it will not be penalized for high CPU usage once.
- Real-time tasks
 - **Always** preempt non-RT tasks
 - No dynamic priority adjustment
 - FIFO or RR (among same priority tasks)

Aside: CFS

The **Completely Fair Scheduler** is a scheduling algorithm designed to split CPU time between runnable tasks as fairly as possible. It was introduced in Linux kernel version 2.6, and phased

out in version 6.6. Essentially, it keeps track of the virtual time of each task, ordered in a red-black tree. Then, it simply chooses to run the task with the smallest virtual runtime, i.e. the task that has received the least execution time so far. Note how CFS essentially replaces the previous $O(1)$ heuristics (except for RT tasks), as tasks that sleep more will naturally be scheduled sooner.

However, we don't just want fairness in the Linux scheduler. We want to prioritize low latency and starvation freedom too.

- Constraint 1: Target Latency
 - i.e., period of time over which every process is *guaranteed* service at some point
 - $\text{Quanta} = \frac{\text{Target Latency}}{n}$
 - However, if n is too large, the time slice becomes too large, and context switching overhead becomes significant! This is reminiscent of the problem with RR. This impacts throughput negatively.
- Constraint 2: Minimum Granularity
 - i.e., minimum length of any time slice
 - Ensures context switching overhead never outweighs the time slice.

Nice values still exist in Linux, though—how does CFS account for this?

The answer is *changing the rate of CPU cycles given to threads*. Consider two threads A and B , where A has weight $4\times$ that of B . Then, while the *virtual CPU time* of both tasks in the red-black tree will be the same, the *physical CPU time* of A will be $4\times$ that of B 's. (i.e., the virtual CPU time of A advances $4\times$ slower than B 's). In fact, this is precisely the notion of weighted virtual time seen in EEVDF!

Chapter 8: Address Translation

What does address translation enables us to do? Here's a list of a few...

- Process isolation
- IPC
- Shared code segments
- Program initialization (run program before loading all code into RAM)
- Efficient dynamic memory allocation
- Cache locality (via *page coloring*)
- Program debugging (via fine-grained memory permissions)
- Efficient I/O
- Memory mapped devices (map files into address space)
- Virtual memory (abstraction of more memory than physically available)
- Saving and restoring state
- Persistent data structures (even when programs/systems crash)
- Process migration (between servers)
- Information flow control
- Distributed shared memory

8.1 Address Translation Concept

Goals

- **Memory protection.**
- **Memory sharing.**
- **Flexible memory placement.** Ability to place process anywhere in physical memory.
- **Sparse addresses.** Allow each dynamic section to grow as needed.
- **Runtime lookup efficiency.** Fast hardware address translation.
- **Compact translation tables.** Small space overhead.
- **Portability.**

Process sees *virtual* addresses, hardware sees *physical* addresses. OS translates from virtual → physical.

8.2 Flexible Address Translation

Base and Bound

The program memory must exist in the physical address range $[Base, Bound]$. In virtual address terms, this is $[0, Bound - Base]$. This method is simple and fast, but flawed. Dynamic memory regions are difficult since each program's memory must be continuous, and it only supports coarse-grained protections on the entire-process-level. Sharing is also extremely difficult.

Segmented Memory

This is a step up from Base and Bound, in which each segment of a process is assigned a base and bound. A virtual address is of the form $[Segment|Offset]$, where the segment number indexes into a *segment table*. This also supports more fine-grained protections on a segment-by-segment basis. Process sharing of specific memory regions is also enabled by this scheme. However, this can be difficult for shared libraries, as processes load different numbers of shared libraries, and thus may assign different segment numbers for each library. This can be fixed via *segment-local* addresses for the library code (addresses relative to each segment).

Copy-on-Write

UNIX uses a copy-on-write mechanism for `fork()`, in which memory is only copied over when a parent or child process attempts to write to said memory. With the segmenting memory management scheme, this is now possible. The OS can mark the page as read-only in both the parent's and child's segment table, and then copy that entire segment. This extends to pages (which we'll discuss soon) as well.

Segments also allow for efficient dynamic memory allocations. In particular, when we allocate more physical memory for a segment, we want to zero the contents of the memory; otherwise, this may leak information from another process. However, we also want this to be efficient, and only zero memory if it will be used (just like how we only copy-on-write for `fork()`). The OS can address this via *zero-on-reference*: allocating a memory segment for, e.g., the heap, but only zeroing the first few KB and setting the bound of this segment to just the zeroed part. If the program expands the heap, the bound register can be increased and the additional memory can be zeroed before resuming execution.

However, segments have some major drawbacks. One is extensive *external fragmentation*. Managing a large number of variable size, dynamically growing memory segments naturally results in a distribution of memory with allocated and free segments scattered across. The system can then reach a point at which there is enough free space for a new segment, but the free space is not contiguous!

Moreover, it's difficult to know *a priori* how much a memory segment must grow. For instance, how much memory should the OS set aside for the heap? Too large of a segment wastes space, but too small and it's not possible to expand the heap without copying it elsewhere—which is extremely expensive. This is arguably the most principal limitation of segmentation.

Paged Memory

With paging, memory is allocated in fixed-size chunks called *page frames*. Address translation also works similarly to segmentation: virtual addresses are [Page #|Offset], and each page has its own access permissions. However, address translation functions slightly differently. The page number is used to index into the page table, which provides the physical page frame, i.e. the physical address is [Frame|Offset].

The most clear-cut advantage of paging over segmentation is the versatility for dynamically allocated regions. Now, it's not necessary to know a priori what size should be allocated for the heap, as the pages that form the heap *do not need to be contiguous*. Thus, whenever more space is needed, it suffices to simply allocate a new page from anywhere in physical memory! Notably, though, the virtual address space for the heap still appears linear to the program.

Sharing memory is also convenient; like segment tables, the corresponding page entry for each process's page table just needs to point to the same physical page frame. This does require what's known as a *core map*, which records information about each physical page frame, in order to *reference count* them.

The other optimizations that can be used with segmentation also apply. COW (copy-on-write) simply requires setting the page table entries to read-only, and specially handling them. ZOR (zero-on-reference) requires setting the top of each dynamically allocated memory region to be invalid, i.e. an attempt to access it traps into the kernel and subsequently allocates and zeros a new page only when needed.

Paging also allows running a program before all of its code and data is loaded. Initially, the pages are all marked *invalid*. Then, when the program attempts to access a page, it causes a *page fault*, which the OS handles to bring in the page from disk into main memory.

Data breakpoints can also be implemented with paging. Data breakpoints force a program to pause when it references/modifies a certain memory location. These are typically implemented with hardware support, but with paging, the OS can mark the page containing the relevant address as read-only, and check every single access. (This does, of course, have a much higher overhead than using hardware support).

There is one significant issue with paging, though. The page table *scales proportionally to the virtual address space size*. This means, with a 64-bit virtual address space and 4 KB pages, we'd need a page table of at least petabyte size! In general, the more sparse the virtual address space, the greater the space overhead of the page table.

It's possible to reduce the space overhead of the page table by choosing larger page frames. For instance, by using 4 MB pages instead of 4 KB pages, we reduce the space overhead of a 64-bit virtual address space page table to terabyte size. However, this can lead to significant **internal fragmentation**, in which space within each page frame is wasted, as a process may not need all the memory provided in the frame. Thus, there's a difficult tradeoff to be made between internal fragmentation and page table space overhead.

Multi-Level Translation

Is there a better way to deal with the massive space overhead of paging? Well, consider our problem: we want to perform a lookup on a **sparse keyspace**. There exist, in fact, several potential solutions. A tree or hash table might come to mind.

Multi-level address translation is precisely the application of trees to our problem. This system is very popular among modern systems, as they support several desirable properties that we've previously discussed, including

- Both coarse and fine-grained memory protection and sharing
- Flexible memory placement
- Efficient (dynamic) memory allocation
- Efficient key lookup for sparse address spaces

Practically all modern systems also use paging as the lowest level of the tree, and differ only in their method of reaching the leaves of the tree: segments plus paging, multiple levels of paging, or segments plus multiple levels of paging. Why is paging desired at the leaves of the tree? Well, beyond the previously described advantages of paging,

- **Efficient memory allocation.** By allocating physical memory in fixed-size page frames, management of free space is provided via just a bitmap.
- **Efficient disk transfers.** Hardware disks are partitioned into fixed-size regions called *sectors*, which must each be read/written in its entirety. By setting the page size to a multiple of the disk sector, transfers from disk are simplified.
- **Efficient lookup.** Using a *translation lookaside buffer (TLB)*, we can make page translation lookups fast. Paging also allows lookup tables to be compact.

- **Efficient reverse lookup.** Fixed-size page frames make implementing the core map, which maps from a physical page frame to the virtual page, easy.
- **Page-granularity protection and sharing.** Coarse-grained and fine-grained sharing enabled by access permissions at each level of the tree.

Paged Segmentation

In paged segmentation, memory is segmented as usual. However, within the segment table, each entry actually points to a page table mapping the segment of memory. The entry's "bound" now describes the page table length, i.e. number of pages. The virtual address is thus [Segment|Page #|Offset].

To keep the memory allocator simple, the maximum segment size is also typically chosen to ensure the page table for each segment can be mapped in only a few pages. For instance, for a 32-bit virtual address space and 4 KB pages, by using 10 bits for the segment number and 10 bits for the page number (the 12 other bits are used for the offset), the page table for each segment would fit exactly into one physical page frame, if each page table entry is 4 bytes.

Multi-Level Paging

Multi-level paging essentially extends paging to a tree-level structure. In x86-64, for instance, there are 5 levels of paging: Page Global Directory (PGD), Page 4th Level Directory (P4D), Page Upper Directory (PUD), Page Middle Directory (PMD), and Page Table Entry (PTE). In essence, an entry in the PGD points to a P4D, an entry in P4D points to a PUD, etc. (You can actually have directories point directly to PTEs for pages of larger size too—refer to hugepages in Linux for more details). Thus, the virtual address looks like

[Index 1|Index 2|Index 3|Index 4|Offset]

This structure enables us to allocate lower levels of the tree on-demand; only the top-level page table must be allocated in the beginning (and stored in the **PDBR**, page directory base register). Moreover, access permissions can be specified at each level, and so sharing at the granularity of page directories is possible too!

Multi-Level Paged Segmentation

This is just a combination of the above two parts! Memory is segmented, and each segment has a multi-level page table. x86 actually uses this scheme, with a per-process **Global Descriptor Table** that is simply a segment table, and a multi-level paging scheme similar to the previously described scheme, but with only 2 indices. (Notably, x86-64 largely forgoes segmenting, besides the FS and GS registers, and just uses multi-level paging as previously discussed). Like x86-64, the OS has a register for this top-level table: **GDTR**, global descriptor

table register. x86 also keeps track of a segment register, which is used implicitly for relevant instructions.

Portability

The differences between translation mechanisms at the hardware level can make an operating system *more or less portable* to different processor architectures. Moreover, the operating system frequently needs to keep two records of address translation: one at the hardware level simply for address translation, and one at the software level to support COW, ZOR, and other features.

The software memory management data structures mirror the hardware structures, and consist of three parts.

- **List of memory objects.** Regardless of the underlying architecture's use of segmentation, the kernel memory manager needs to track which memory regions represent code, shared data, etc. Also, reference counts for COW.
- **Virtual to physical translation.** The kernel needs to track if an invalid page is really invalid, or just not loaded into the address space yet, or if a read-only page is truly read-only or just a COW page.
- **Physical to virtual translation.** That is, the *core map*. The kernel needs to keep track of all physical pages, and which processes map to each page, in order to update processes' page table entries when the physical page is switched out.

Regardless, though, operating systems must choose carefully how to structure their address translation mechanism to be portable to different architectures. Linux, for instance, has modeled its address translation methodology after the x86 architecture of segmented multi-level paging. This is great for porting Linux to x86, but makes it more difficult for ARM and other architectures. Meanwhile, Mach, and later Apple OS X, uses a *hash table* instead of a tree for software address translation. This is historically referred to as an **inverted page table**. This is not only more portable between different architectures, but using a hash table also speeds up translation, particularly as multi-level paging gains more and more levels to keep up with increasing memory.

Inverted Page Table

With an inverted page table, a table with size proportional to the number of physical page frames is kept. Virtual page numbers are the keys, and are hashed to find the corresponding physical page frame. Each hash table entry contains a tuple

(Virtual Page #, Physical Page Frame #, Permissions)

Note the necessity of the virtual page number—this helps with handling hash table collisions, via chaining or rehashing or probing, and validating which physical page frame actually corresponds to this virtual page number.

A useful consequence of using this more portable hash table for software paging is that the hardware multi-level translation table can be treated as a *hint*, i.e. the result of a computation whose results may no longer be valid, but where using an invalid hint will trigger an exception. In particular, if the translations and permissions of the hardware page table are a *subset* of those of the software page table, it is a hint.

What about hardware hash tables? Do we still need multi-level page tables for our hardware? It turns out that, yes, it's still needed. The difficulty with hardware hash tables is that hash collisions become much harder to handle. If two pages hash to the same hash table entry, only one can be stored in the physical page frame; the other must be stored in a secondary hash table entry or even on disk. Copying in the new page can thus be expensive, especially if hash collisions are frequent. Moreover, hash tables tend to have worse cache locality. Thus, modern systems only use inverted page tables in software to improve portability.

8.3 Efficient Address Translation

There's just one issue; these hardware address translations involve multiple extra memory references per access to memory. That's a lot of overhead! Can we do this faster?

Translation Lookaside Buffer

Well, recall that we already have caches for all of our memory, to cache recently or frequently used data. We can apply the same idea here to produce the **translation lookaside buffer (TLB)**, a cache for address translation.

The TLB simply maps previous address translations, with the following tuple for each entry

(Virtual Page #, Physical Page Frame #, Permissions)

Exactly like the inverted page table, in fact! Though, instead of hashing, the TLB hardware checks all entries simultaneously against the virtual page number to find a match. If there's a match, we successfully have a *TLB hit*, and avoid the overhead of translating the address.

❓ Access permissions are cached as well! Why?

Of course, like hardware caches, the TLB is located in fast, on-chip static memory nearby the processor. Modern systems even include multi-level TLBs now, just like the popular multi-level

cache design. This is expensive (cost-wise), but efficient and results in significant gains in processor performance.

A TLB also requires an address comparator for each entry to parallelize the search for a match. The TLB is, thus, frequently made *set-associative* to reduce the cost of the comparator; this results in slightly higher miss rates, but can reduce monetary costs significantly.

Software TLB?

You might wonder if we can have a software-loaded TLB as a fast path if the hardware TLB misses. This would also simplify kernel design, since it would no longer need to keep track of the hardware page tables. However, this requires the processor to trap into the kernel for the lookup; an expensive operation that can cost several hundred instructions. (While a TLB miss can be served by the hardware page table in an order of magnitude less instructions).

Superpages

There do exist some optimizations to improve TLB hit rate. For instance, kernels can implement *superpages* (a.k.a. hugepages). A superpage is essentially the result of having a PTE at a page directory higher in the page table hierarchy. For instance, with x86-64, with 4 KB pages, a PTE descending directly from a PUD might produce a 16 MB superpage. The benefit of this for the TLB is that a cached superpage will effectively cache address translations for more memory addresses!

Consistency

As with introducing a cache into any system, keeping cache consistency is always a challenge. There are typically three problems associated with TLB consistency.

- **Process context switch.** On a process context switch, the page table switches. Thus, TLB entries become largely invalid. One approach to this problem is **flushing** the TLB, discarding its old contents. Flushing is somewhat expensive, though. Thus, another approach is using a **tagged TLB**, where each TLB entry includes the PID, and thus the hardware only considers entries that match the current process PID.
- **Permission reduction.** When the kernel modifies a page table entry's permissions, hardware usually guarantees consistency with the processor's regular data cache of main memory; however, TLB consistency is typically *not* provided by hardware.

Why? There are a few reasons. (1) Page table entries can be shared between processes, so one modification can affect multiple TLB entries. (2) The TLB contains a virtual to

physical page mapping, but does not record *where this mapping information is stored* (page table), so it actually can't tell if a write to memory would affect an entry. (And, even if it did track this, it would be inordinately expensive to check every single memory store to see if it affects a TLB entry).

Thus, the kernel is responsible for keeping the TLB consistent when permissions change. When permissions are added, the kernel actually doesn't need to do anything—any instructions that require the new permissions will cause an exception anyways, and force the kernel to load the updated page table entry.

However, after a *permission reduction*, the kernel must ensure the TLB is updated; otherwise, a memory access may be valid, according to the TLB, when said permissions have been revoked! (e.g., think COW). In these cases, it typically suffices to simply *remove* the entry from the TLB.

- **TLB Shutdown.** On SMP (symmetric multiprocessor) systems, there are further complications. Each processor maintains its own TLB. Whenever a TLB entry is invalidated by any one processor, the TLBs of the other processors must be interrupted and updated as well. This is, of course, expensive, but certainly necessary, and is known as a TLB shutdown. In essence, the processor updating the TLB will send an interprocessor interrupt (IPI) to other processors, who will promptly update their own TLBs and acknowledge the original processor. Only then, *after receiving acknowledgements* from all processors, the original processor continues. Many kernels also **batch** TLB shutdown requests, since the overhead increases linearly with the number of processors.

Virtually Addressed Caches

We can further improve performance via virtually addressed caches. These are implemented in a small, on-chip cache, and cache contents of physical memory, but differ from normal hardware caches in that they are indexed by *virtual address*. Much like the TLB and normal hardware caches, the virtually addressed caches are frequently split into an instruction cache and data cache.

In essence, VACs are regular data caches, but with virtual address indexing. It also has the same consistency issues as TLBs do.

- **Process context switch.** Cache must be flushed or entries invalidated.
- **Permission reduction.** Cache must be flushed or the system must search and modify all affected entries, both of which are expensive operations.
- **TLB shutdown.** Again, same thing here.

Yet *another* issue, for VACs, is **memory address aliasing**. This is when processes sharing memory use *different* virtual addresses to refer to the same physical memory location. Each process would have its own TLB entry and VAC entries for this memory. If the memory is written to by one process, how does the system know to update the VAC to reflect this for the other copies (the entries for the other process)?

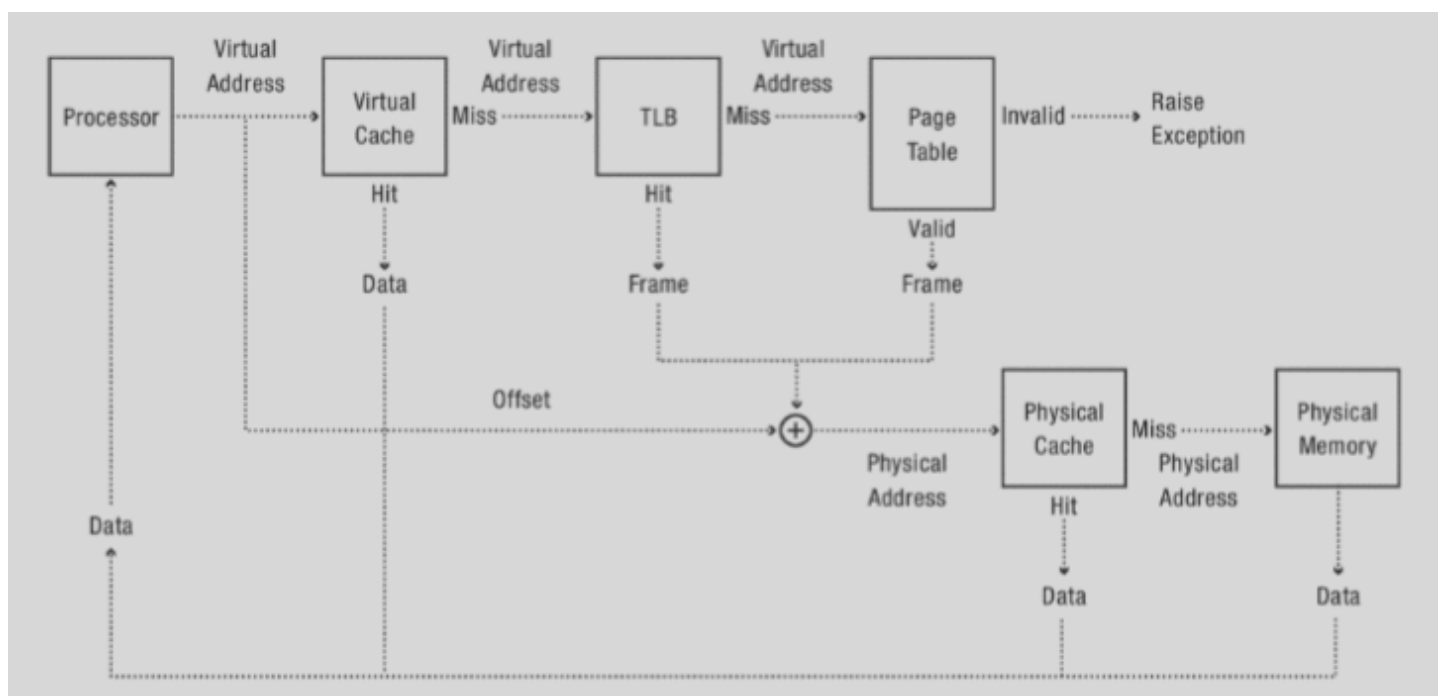
The most common solution to this is to store the *physical address* as well in the VAC. Then, in parallel with the virtual cache lookup, the TLB is consulted for physical address and page permissions. This is known as **overlapping** the VAC and TLB accesses, and results in more performant, consistent data retrievals. For store instructions, the system can also perform a *reverse lookup* in the VAC to find all entries that match the same physical address.

Physically Addressed Caches

And, finally, we've come full circle! On-chip (and typically multi-level) caches of main memory with physical addresses as indexes are included on all modern systems. These occur after a TLB/page table translation, and serve as a fast path before retrieving data from main memory. Critically, page tables are also frequently cached! This makes address translation, in the case of a TLB miss, even faster.

Diagram

Now, we can finally take a look at the complete diagram of, e.g., a load operation's process through hardware in efficiently retrieving data.



TLB Slice

This was briefly mentioned in the CS 162 slides. Essentially, since small TLBs are typically *fully-associative*, some TLBs have a very small, direct-mapped cache in front, known as a TLB slice, to speed up the most frequent TLB hits.

8.4 Software Protection

In this section, we consider software techniques to execute code in some restricted setting without relying on hardware address translation. But, why is this needed?

- Simplify hardware, increase portability.
- Application-level protection (e.g. web browser sandbox)
- Kernel-level protection (e.g. against third-party drivers)
- Portable security. Applications run across a variety of hardware devices and operating systems, and require a common runtime environment to isolate itself from these specifics. This includes downloading and running applications without concern of malicious application corruption.

Single Language Operating System

A very simple approach is to restrict all applications to be written in a single, carefully designed programming language. In particular, its language and environment should allow the kernel to enforce guardrails, and the compiler and runtime system should be trustworthy (and not require hardware protection).

One widely-used example of this is eBPF packet filters. The operating system allows users to load executable eBPF *bytecode* into the kernel to customize the kernel's network processing stack. The kernel restricts the packet filter language to make it easy to determine and permit only safe filters.

Another example is the use of JavaScript in modern web browsers. Browsers sandbox each website's JavaScript, preventing it from accessing other website data and the rest of the client machine. Yet another example is the Xerox Alto research machine, which exclusively used Mesa, and thus implemented all protection in software.

Language protection and garbage collection.

Garbage-collected languages must keep track of all valid pointers visible to the program. Thus, programs written in these languages cannot simply point to/jump to arbitrary memory locations; this would result in issues with implementing the garbage collector.

Thus, a garbage-collected language necessitates *software-level* protective restrictions on the language's capabilities.

This, of course, has practical limitations though. Thus, on modern systems, this is typically combined with hardware protection.

For instance, using a single interpreted language seems safe; however, this requires trusting an interpreter and its runtime libraries. Interpreted code is also extremely slow, which forces interpreters to compile code to machine code and run directly on the processor, increasing the attack surface.

Thus, for instance, the Windows operating system runs web browsers in a special process with restricted permissions, to prevent a vulnerability in an interpreter or browser from potentially causing a host machine takeover.

Writing all software in a single, safe language is no good either. Defensive programming is still required at the trust boundary between application code and the operating system, and the compiler and shared libraries must be trusted.

Language-Independent Software Fault Isolation

Perhaps the most clear-cut limitation of a single-language system is, of course, the fact that programmers are limited to a single programming language. Of course, it's not possible for the operating system to trust the compiler of *every* programming language out there; is there a better way to efficiently isolate application code at the software-level without hardware support, and independently of the programming language?

This is especially important for systems that need nested levels of protection within a process: web browsers executing JavaScript in sandboxes, restricting kernel device drivers, etc.

It turns out that this idea of sandboxing code is a popular approach; both Google and Microsoft have developed sandboxes that safely and efficiently run code in any programming language. By validating all addresses accessed are within a restricted memory region, and using control/data flow analysis to remove checks (injected by the compiler) not strictly necessary for the sandbox to be correct, the execution of the process remains both safely isolated and efficient.

The control/data flow analysis can help optimize:

- **Loop invariants.** If a loop strides through memory, replacing many runtime checks with just one at the loop's beginning is much more performant.

- **Return values.** If the program provably never modifies the saved RIP on the stack, the return values don't need to be validated.
- **Cross-procedure checks.** If a parameter is always checked before being passed to a subroutine, it need not be checked inside the subroutine.

*Note that indirect branching and indirect accesses definitely need *runtime* checks, and thus are part of the checks that are still necessary for the correctness of the sandbox.

Virtual machines without kernel support.

We can emulate a VM without any kernel support by modifying the machine code of the guest OS. This includes changing instructions to enable/disable interrupts to no-ops, changing instructions to execute a user program to first sandbox the program separately, etc.

Sandboxes via Intermediate Code

For portability reasons, both Microsoft and Google generate intermediate code with their compilers, which are then sandboxed. This makes it much easier to modify code and perform flow analysis to enforce guardrails.

Notably, the Java Virtual Machine (JVM) is one similar example of a sandboxed environment. While only for Java, it converts Java into JVM bytecode that can be more easily verified at runtime for safety.

Chapter 9: Caching and Virtual Memory

Almost all computer systems use caches in some way, shape, or form!

Chapter 8 included three widespread hardware caches:

- TLBs
- Virtually Addressed Caches
- Physically Addressed Caches

But there exist many more!

- DNS Resolution
- Web content
- Web search
- Email clients
- Incremental compilation (Makefiles)
- Just in time translation (typically for memory-constrained devices)
- Virtual memory (cache for disk)
- File systems (also use memory as a cache for disk!)
- Conditional branch prediction

All caches, though, face *3 design challenges*:

- **Locating the cached copy.** How does the system quickly determine if the cache contains a piece of data and retrieve it?
- **Replacement policy.** The cache must be kept small, relative to the backing store. So, how should the system decide which data is valuable to cache?
- **Coherence.** How do we keep cached data up to date? This is especially important for SMP and distributed systems.

9.1 Cache Concept

The simplest cache is a memory cache (e.g. VAC or PAC) that maps an address to the data stored at that address. A cache access either produces a *cache hit* (yay!) or a *cache miss* (sad :<).

For a cache to be useful, two properties must hold.

1. Cost of retrieving cached data must be significantly less than fetching data from the backing store.
2. The likelihood of a cache hit must be high enough to make the cache worth it.

For property 2, caches frequently have two sources of predictability: *temporal locality* (i.e., recently accessed data will likely be accessed again) and *spatial locality* (i.e. nearby data will likely be accessed). Memory cache design takes advantage of these two features: the choice of eviction algorithm improves temporal locality, while cache line size (e.g. 64 bytes) improves spatial locality. Memory caches even *prefetch* nearby data into the cache to improve spatial locality.

We can thus represent the expected latency of a read request to memory as *Average Memory Access Time (AMAT)*:

$$AMAT = P(\text{hit}) \cdot L(\text{hit}) + P(\text{miss}) \cdot L(\text{miss})$$

Where $P(x)$ represents the probability of event x and $L(x)$ represents the associated average latency of event x .

Write requests are a bit more complex. In particular, there are two possible cache modes for write requests: *write-through* and *write-back*. With write-through mode, write requests to the cache will propagate writes directly to the backing store. This can lead to significant extra overhead from write requests, especially if writes to the backing store are not buffered. Meanwhile, write-back caches only propagate updates to the backing store when the data that is "*dirty*" (written to) is *evicted* from the cache.

9.2 Memory Hierarchy

For hardware, there is a fundamental tradeoff between the speed, size, and cost of storage. Faster hardware is more expensive, and thus must be limited in size. Thus, there is a whole hierarchy of caches on a single chip. A computer in a data center might have

- First-level cache (L1). Frequently virtually addressed.
- Second-level cache (L2). Typically physically addressed.
- Third-level cache (L3, typically LLC=Last Level Cache)
- First- and second-level TLB
- Main memory (DRAM).
- Data center memory (DRAM). Facilitated by high-speed LAN connections. Going to nearby nodes' memory instead of disk is known as *cooperative caching*.
- Local disk or non-volatile memory (SSD).

- Data center disk.
- Remote data center disk.

9.3 When Caches Work and When They Do Not

Caching is influenced by many factors, ranging from the cache size, cache algorithm, and program behavior. Typically, the interaction between all these factors determines cache effectiveness—for instance, some cache algorithms are more suited to certain program behaviors.

Working Set Model

A program's *working set* is the size of a critical mass of program data such that, when included in the cache, receives good hit rate. On a graph of cache size versus hit rate, the working set is the "knee" of the curve. Of course, a cache should, ideally, fit in the working set of a program; not doing so can significantly impact performance, as the cache will experience a lot of *thrashing*.

As programs change, working sets will also change. Generally speaking, as long as the cache size is greater than the working set size, cache hit rates will remain high, though they may shift between different working sets. Moreover, when a processor switches from one program to another, the cache will experience a *burst* of miss rates. This is just an inevitable result of the *phase change behavior* inherent to these systems.

Zipf Model

Zipf was a linguist who developed a model, now named after himself, to describe the frequency of individual words in text. However, it turns out it's also great for describing common real-world distributions, particularly website access patterns.

The Zipf distribution describes a set of words ranked in order of popularity, and states that the frequency of each word is proportional to $\frac{1}{k^\alpha}$, where $1 \leq \alpha \leq 2$. More succinctly, $f_k \propto k^{-\alpha}$. An important characteristic of a Zipf distribution is that it is *heavy-tailed*. A significant proportion of the total words in a text are the most popular words; however, there is still a non-negligible proportion to less popular references.

In these situations, increasing the cache size improves cache hit rates, but at *diminishing returns*. This is contrary to the *threshold* behavior seen in the working set model, in which, once the cache size nears the working set size, the cache hit rates improves significantly.

9.4 Memory Cache Lookup

Caches higher up the systems stack, like page caches, have flexibility in their design—they can use a linked list, trees, hash table. However, hardware caches are constrained in their designs, since their lookup latency must be kept extremely low.

- **Fully Associative.** Any address can go anywhere in the cache. However, upon a cache access, it's necessary to check *every* cache line for the address. This is great for cache hit rates, but expensive, especially as caches grow larger and larger, as it requires even more comparators (and lookups take more time!).
- **Direct Mapped.** Each address has a specific cache line in which it can be stored. Think hash table! This is cheap and makes lookups extremely fast; however, it drastically reduces hit rates, especially if, e.g., RIP and RSP in x86-64 both map to the same cache line... oops ><
- **Set Associative.** This is a hybrid between the above approaches, and generally the most popular. With a set associative cache, every address directly maps to a *set of cache lines* (of size N for an N -associative cache), of which it is fully associative within this set. This avoids the thrashing of direct mapped caching, while not being as prohibitively expensive as a fully associative cache.

However, direct mapped and set associative caches can pose a design challenges for the operating system. These caches are much better if a program's working set is spread across the different "buckets"/"sets" in the cache. This makes factors like, e.g., the default page size, able to greatly impact this distribution.

This is particularly an issue if the OS assigns pages that fall into the same bucket to a single process—thus, operating systems typically leverage *page coloring* to prevent this. In this scheme, physical page frames are partitioned into sets based on the cache bucket they fall into, and the operating system will attempt to distribute a program's page frames across a variety of colors.

9.5 Replacement Policies

There is no one right replacement policy! Some policies are optimal for some workloads, and pessimal for others. In general, choosing the right policy depends heavily on (1) the problem's design constraints and (2) the distribution of requests this cache serves.

Random

Self-explanatory...

But it is occasionally a practical design, especially for systems constrained extremely heavily by latency on the critical path.

FIFO

First-In-First-Out is a caching strategy that operates exactly like *FCFS* from scheduling. Just think of a queue data structure! Unfortunately, it's also pessimal for several workloads. For instance, consider a program that scans through an array of memory addresses repeatedly; however, the entire array does not fit into the cache.

Belady's MIN

Belady's MIN is known as the *optimal* cache replacement policy, with regards to hit rate. Essentially, at each eviction, it will replace the block that is used *farthest in the future*. (Conversely, the worst possible strategy is to always replace the block that is used the soonest).

❓ **Is Belady's MIN really the best cache replacement policy?** >

Well, in terms of cache hit rate, yes. But is that really the *only* measure of cache effectiveness...? Some researchers *disagree*!

Unfortunately, like SJF with scheduling, MIN requires knowledge of the future, so it's not practical. However, cache algorithms are frequently designed to *approximate* MIN.

In fact, with knowledge of the future, we could technically do even better than Belady's MIN, by prefetching everything we need—reducing cache misses to 0.

LRU

Least Recently Used is a caching strategy that takes advantage of temporal locality. In essence, it maintains a queue (really, a *linked list*) of cache lines ordered by their most recent access. In particular, when a cache line is accessed, it is moved to the back of the queue (noting that it should be popped off the queue last, essentially). This is known as the **MRU**, most recently used, position.

However, in hardware, a linked list is difficult to implement with very low latencies; thus, it's typically necessary to *approximate* LRU.

LRU is a great approximation of Belady's MIN in many cases. Of course, however, it is still pessimal for some workloads—most prominently, the "memory scan" workload that was also adversarial for FIFO.

LFU

Least Frequently Used is an analogue of LRU that uses frequency, instead of recency, as its metric for cache value. It's typically used for distributions that closely follow a *Zipf* distribution (e.g., a CDN web cache), keeping popular pages in the cache.

Additional Factors? >

So far, we've assumed the cached objects are all the same size. But what about caching variable size objects? And what about parallel programs—e.g., for MapReduce, any file not being cached is as bad as if all the required files were not cached (since it is like a fork-join model). All of these factors can make cache eviction algorithms more complicated beyond what we cover here; however, we won't discuss this in much detail.

Belady's Anomaly

Intuitively, increasing cache size should improve the cache hit rate... right? Well, not quite. **Belady's anomaly** notes that, in some instances, adding space to a cache can negatively affect the cache hit rate.

However, this only impacts certain algorithms, and only in certain situations. *Stack-like* replacement policies like LRU and LFU, in which the blocks kept by a cache of size k are a *subset* of the blocks kept by a cache of size $k + 1$ at the same time t , do not suffer from Belady's anomaly. This is because any cache hit that occurred in the cache of size k will have certainly occurred in the cache of size $k + 1$. In contrast, the same property does not necessarily hold true for FIFO! Thus, there can be workloads such that a cache operating with FIFO increases in size and suffers a decrease in hit rate.

9.6 Memory-Mapped Files

Any modern operating system must implement some system for **demand paging**. Essentially, the operating system provides an illusion to applications of infinite memory; however, some of this memory actually be stored on disk, rather than in main memory! Thus, when a program tries to access unmapped memory, the operating system must go to disk and retrieve the

missing memory page. In short, the operating system uses main memory as a cache for the disk!

Memory-mapped files involve mapping a file contents into a program's address space, and allowing it to directly edit a file's contents that way. This essentially transforms this memory into a write-back cache for the disk. The application also does not need to worry about how this is done; the operating system uses demand paging to retrieve files from disk as they are referenced. This offers several advantages:

- **Transparency.** The program doesn't need to worry about if the file is in memory or not—again, the operating system handles this for it.
- **Zero copy I/O.** The operating system doesn't need to copy file data between disk and kernel buffers and user memory; rather, it just adds an entry to the program's page table, and only needs to worry about copying data between main memory and disk.
- **Pipelining.** The program can begin operating on data in the file once the page tables are setup; it doesn't need to wait for the entire file to load, as the operating system will handle any exceptions (unloaded segments) and provide an abstraction of the file already being there (demand paging)!
- **Interprocess communication (IPC).** Multiple processes can share information through a memory-mapped file mapped in both processes' address spaces, without needing to send data to kernel buffers/disk.
- **Large files.** For extremely large files, i.e. those that cannot fit in main memory (but can fit into the virtual address space), the application does not have to worry about managing which parts of the file are kept in memory, and which are on disk. The OS handles it all!

Implementation

Let's walk through the process of a program attempting to access an unmapped address.

1. TLB miss → page table walk
2. Page table exception → **page fault** hardware exception, traps into kernel
3. Conversion of virtual address to file offset (via segment table)
4. Disk block read—kernel allocates empty page frame and issues disk read request → thread yields processor (blocks on disk read)
5. Disk interrupt—disk interrupts kernel once request is serviced → kernel thread handling it is resumed
6. Page table update
7. Resume process
8. TLB miss → page table walk

9. Page table fetch → TLB update

There's one fairly important consideration that we glossed over in the above process, though—what if main memory is full? Then, when the kernel allocates an empty page frame, it must **evict** an existing page from main memory. More specifically, it must

1. Select a page to evict. How? We discuss in the next section :)
2. **Invalidate** all PTEs (page table entries) that point to evicted page. This is done via a **core map**, i.e. array of all physical page frames, including a list of PTEs pointing to this page. Note that the PTEs should not be removed, because, as far as the application should know, this is still in memory (abstraction of infinite memory!)
3. Write-back any changes to the evicted page to the disk. This is tracked via a **dirty bit** in the PTE metadata. This flag is also included in the PTEs in the TLB—whenever it's set (i.e. a write request generates a TLB hit), the change should be propagated to the page table by hardware. Also, note that the page must be copied back to disk if **any** of the PTEs pointing to this page have the dirty bit set.

Evicting a dirty page is more expensive than evicting a clean page; thus, the OS often **proactively cleans pages** in the background, to prevent dirty page eviction from affecting an instruction's critical path. Also, evicting a dirty bit requires a TLB shutdown, too—even more expensive!

Emulating a hardware dirty bit in software >

It's also possible to emulate a hardware dirty bit in software! The OS just marks a clean page as read-only; the first write will trigger a memory exception, upon which the OS can mark the page as dirty and upgrade the protection to R/W. Cleaning the page also requires resetting the protection to read-only.

Approximating LRU

As aforementioned, the exact LRU algorithm is far too expensive to implement for hardware caches, or even page caches, for that matter. The reason is, even if the operating system manages the page caching algorithm, it would require the hardware to maintain too much metadata (i.e. linked list). Instead, hardware typically chooses to provide a minimal set of metadata so the operating system may approximate LRU or LFU.

One commonly chosen algorithm for approximating LRU (used by the Linux kernel!) is the **clock** algorithm, or the **second-chance** algorithm. It leverages the fact that most processor architectures keep a **use bit** in each PTE, which is set in hardware whenever the PTE is

brought into the PTE. As with the dirty bit, a physical page is marked used if *any* of its PTEs have the bit set.

In essence, the clock algorithm attempts to approximate LRU by evicting *any old* page, rather than *the oldest* page. (Note: We will describe it according to CS 162's interpretation, which does not exactly follow the textbook). The algorithm maintains pointer into a queue of page frames (e.g., this may simply be the core map). Then, upon a...

- ...cache hit, the use bit of this page will be set (hardware already does this with TLB hit)
- ...cache miss, the pointer will be advanced until it finds a PTE that does not have a use bit set, and will evict this page. Along the way, it will clear the use bit of PTEs. Note that this can cycle through the entire queue, if necessary! Finally, the new page will have its use bit set to 1 (hardware already does this with TLB hit)

According to the textbook (not CS 162), the operating system can also this periodically, not just on every page fault. More frequent sweeps collect more fine-grained information about page usage; however, it increases software overhead.

A generalization of the clock algorithm is the k th chance algorithm. In essence, the kernel will pick a page that has not been used for the last k sweeps of the clock algorithm. The clock algorithm is extended to partition pages based on how recently they were used, with pages being evicted in FIFO order from the k th chance partition. This may be implemented by incrementing some counter every time a page is encountered with its use bit not set (and, if its use bit is set, reset the counter and use bit).

Note: FIFO is the just the first chance algorithm!

Emulating a hardware use bit in software >

Simply set a PTE to invalid even when the page is in memory; accessing the PTE will trigger a page fault, allowing the kernel to record usage statistics for the clock algorithm.

9.7 Virtual Memory

We can generalize the idea of memory-mapped files by backing *every memory segment* with a file on disk. This is the design philosophy taken by UNIX—if you're on a Linux system, for example, you can check out `/proc/<pid>/` for some files backing the program executable, memory, etc. Unlike memory-mapped files, though, this process memory is ephemeral; upon process exit, there is no need to write-back to disk.

The main advantage of virtual memory is **flexibility**. When out-of-memory (OOM), the system can "swap" some processes' pages to disk, and simply demand-page them into main memory when needed. However, this does introduce one critical complication: it's necessary to balance which processes' pages are paged to disk, and which are kept in main memory. After all, a modern disk can only handle so many page faults—there's several orders of magnitude difference between the throughput of handling page faults and the throughput of executing instructions.

Self-Paging

One consideration, for instance, is that the behavior of one process can hurt another process. Consider treating all pages equally, as in the clock algorithm. Then, if a process constantly accesses its pages over and over again, it'll make sure it keeps all of its pages in memory! This keeps its own latency down, but massively decreases the speed of other processes, to the point that it may seem they are making zero progress.

One popular solution to this is **self-paging**. In this scheme, each process/user is allocated its fair share of page frames, using max-min scheduling. Its page allocations can never exceed this fair share; extra pages will simply evict this process/user's own pages. Of course, this can detrimentally affect resource utilization; the question is whether this tradeoff is worth it! (Well, preventing malicious programs is generally worth it...)

Swapping

The simple scheme of swapping out lightly used pages to disk (i.e. outside of the working set of any process) works great for lightly loaded systems. However, under heavy loads, the union of the working sets of all active processes may no longer fit into main memory! In this instance, system performance will degrade dramatically, as page faults will increase significantly (due to the working set phenomenon described previously).

The solution is to prevent this overload condition from ever occurring by essentially **starving** a selected subset of tasks of their resources. Essentially, it's better to starve some tasks of their fair share if not doing so will essentially freeze the system.

This process is known as **swapping**. When too much paging activity occurs, the operating system prevents significant performance degradation by moving **all** of the page frames of some processes to disk. Eventually, when other tasks finish, the kernel can bring the swapped process's pages back into main memory. Unfair? Yes. Unreasonable? Quite the contrary.

Chapter 10: Advanced Memory Management

10.1 Zero-Copy I/O

How should the kernel transfer hardware device data to userspace application? A traditional approach would be for the device to copy data (e.g. from disk) into a kernel buffer, and then for the kernel to copy it from the buffer to userspace. Is there a way to avoid this level of indirection though, i.e. have the hardware device transfer data directly to the userspace application? In other words, can we achieve **zero-copy I/O**.

Moving all userspace applications into the kernel is obviously terrible for security reasons. Allowing applications to directly manipulate a kernel buffer via syscalls is a better idea, but it's not applicable to situations where the application may need to perform more complex work with the data.

There are, in fact, two popular, performant solutions for zero-copy I/O, that eliminate the copy across the kernel-user boundary for large blocks of data (for small blocks, it doesn't impact performance significantly).

Page Table Swap

- User-to-kernel space copy:
 - Kernel changes PTE to read-only
 - Kernel pins page to prevent eviction from main memory
 - Kernel then copies the userspace memory directly to the device
- Kernel-to-user space copy:
 - PTE initially points to empty user buffer
 - Kernel swaps PTE to point to full kernel buffer when data is ready

Virtual DMA

Modern hardware I/O devices have been granted the ability to transfer data to/from virtual addresses, not just physical addresses. Thus, the kernel can simply provide the virtual address of the user-level buffer to the hardware device, for it to copy to/from.

10.2 Virtual Machines

VM Page Tables

Virtual machines essentially have two sets of page tables!

- **GVA to GPA** (Guest Virtual Address to Guest Physical Address)
- **GPA to HPA** (Guest Physical Address to Host Physical Address)

The above configuration exists because the OS running inside the VM think it is translating to (and has access to) physical memory, when in reality it's an illusion provided behind the VM manager and host OS. Why? Because this VM is itself a user process, and must not be granted actual access to physical memory. Otherwise, the guest OS could take over the host OS! Thus, a GPA is effectively just a HVA (Host Virtual Address).

⚠ I didn't understand (or rather, didn't agree with) the textbook's explanation of this content at all. The following contents (until the end of the VM Page Tables section) are the culmination of my own research into this topic. So, take it with a grain of salt. >

Sources: [Reddit](#), [Rayanfam](#)

But, translating from GVA to GPA, and then GPA to HPA, would be slow on hardware that supports only one page table walk for address translation! Each memory access would require the *hypervisor* (VM manager) to *manually emulate* the page table walk. As aforementioned, this is much, much slower than a hardware page table walk.

There are two well-known solutions to this problem: **shadow page tables**, a legacy, software-assisted method and **extended page tables**, a modern, hardware-supported method.

Shadow Page Tables

Shadow page tables are essentially *direct mappings* from **GVA to HPA**. In essence, this is the page table that the hardware actually walks for address translation (e.g. in x86 the CR3 register is set to the shadow page table corresponding to a guest virtual address space/guest page table). So, a guest memory access doesn't have to trap into the hypervisor, but rather can directly use the MMU and let the hardware perform the page table walk for address translation.

Notably, the guest OS is *unaware* of the shadow page table. In its eyes, it's still managing the address translations and accesses for its or its process's page table. Thus, it's necessary to keep the shadow page table updated when the guest OS updates the its page table, i.e. the GVA to GPA page table.

In particular, if the guest OS updates a PTE in the guest page table (i.e. GVA now maps to a different GPA, which maps to a different HPA in the host page table), this update must trap

into the hypervisor so that it can update the shadow page table mapping for this GVA to point to the new HPA. This is typically done by setting the *pages containing* the guest page tables as *read-only* in the host page table. (To clarify, each guest page table lives at some GPA, which is equivalent to an HVA. The PTE for the HVA in the host page table is marked read-only, to allow trapping into the host OS / hypervisor upon modification).

It's also necessary to update the shadow table if the GPA to HPA mapping changes, i.e. the host page table of the hypervisor / backing the VM's address space.

However, there's one glaring issue with shadow page tables—maintaining *coherence/consistency* with the guest page tables is still slow, as each update requires a trap into the hypervisor, i.e. a **VM exit**. Each of these exits is very expensive (due to the context switch + associated TLB flush), and considering that OS's frequently update page tables or context switch (after all, the guest OS thinks it's directly modifying physical memory, which is fast).

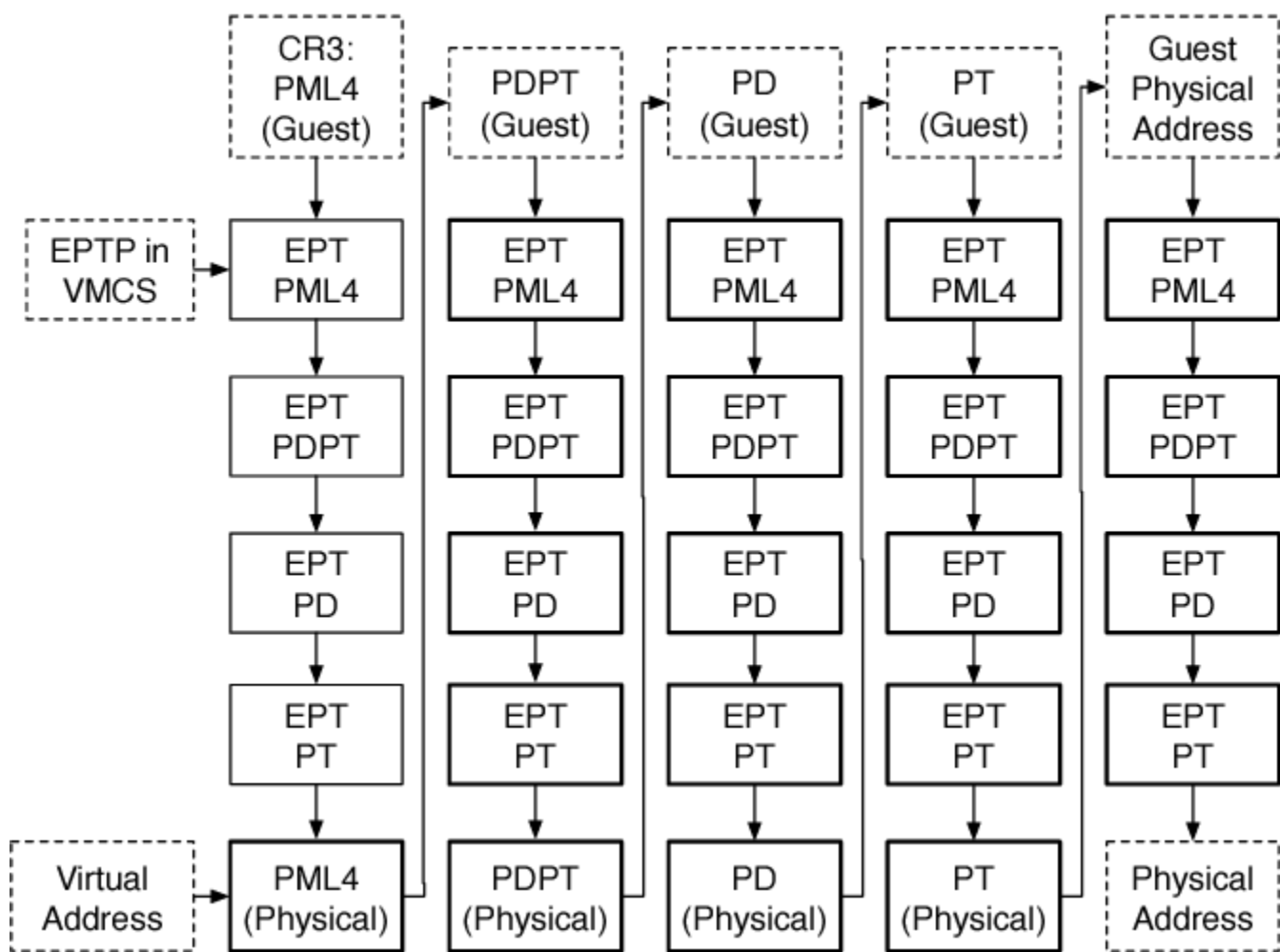
Thus, we turn to the modern, hardware-assisted solution: extended paging.

Extended Page Tables (EPT)

Extended Page Tables, also known as *Second-Level Address Translation* or even *Nested Page Tables* (AMD), is essentially an augmentation of the hardware MMU, in which it is provided the ability to perform a two-level address translation walk, i.e. GVA to GPA to HPA. In essence, we introduce another MMU, the EPT MMU, which will translate GVA to GPA using the guest page table, while the typical MMU continues to translate GPA to HPA.

It's not quite as simple as it seems though—while a typical (i.e. host process) memory access on x86-64 (assuming the default 5-level paging) requires only 5 memory accesses (HVA to HPA), a guest process memory access requires **25** memory accesses (GVA to HPA)! Why is this the case?

The following diagram succinctly illuminates the general idea.



Source

What's going on here??

Well, consider that the EPT MMU must walk the guest page table. But, the guest page table itself stores GVA to GPA mappings, and the EPT MMU cannot just directly access a memory address by GPA—that's a virtual address for the host! Thus, each time the EPT MMU retrieves the GPA of the next level of the guest page table, it must translate this GPA to an HPA—in other words, the regular MMU has to walk the host page table for *every step* of the guest page table walk. Thus, on x86-64, $5^2 = 25$ memory accesses are required.

This is certainly slower than memory accesses on the host, but remains much faster than shadow page tables. EPT takes at most $5 \times$ cycles of a regular memory access (or, $n \times$ for an n -level page table), but a shadow page table takes *several orders of magnitude* more cycles. Thus, practically all modern machines have EPT enabled to keep VMs performant.

Paravirtualization

Finally, there is actually one solution that I didn't mention before, because it isn't quite the same. **Paravirtualization** is a methodology where the guest OS is specifically an OS that is

aware it is operating in a VM. This is known as an *enlightened OS*, and are frequently much more performant than using a normal OS in a VM. However, it's not widely used because the guest OS must be entirely reimplemented to be enlightened, a highly non-trivial task.

Transparent Memory Compression

Consider the following situation: a host machine has 2 VMs running, both with their guest OS of the same version of Ubuntu. Then, would the host machine have to create separate pages for the distinct executable kernel images, despite the memory being the exact same.

The solution is *deduplication*, in which the hypervisor maps these executable images to the *same physical pages* in main memory, in order to avoid keeping the same exact data twice in memory. In fact, the hypervisor does this for all more than just the kernel images—periodically, a *scavenger* runs in the background to identify VM pages that can be deduplicated, and modifies the guest page table such that these addresses point to the same GPA (effectively sharing the same host physical page).

There are actually two different cases to consider, though, for memory deduplication.

- **Multiple copies of the same page.** This is precisely the design described above—these guests' PTEs point to the same page, with the same content. This can be used for read-only pages, or read-write pages via copy-on-write. This is especially useful for kernel images, shared libraries, and zeroed pages.
- **Compression of unused pages.** There also exist memory deduplication strategies for a VM's pages that are close, but not quite the same, as another VM's pages. In particular, for a fairly infrequently used page that is a *delta* of (close to) an existing page in another VM, the OS can, instead of evicting the page, can replace the page with a delta that indicates the difference between the similar pages. Then, it can mark the full page that's kept (not replaced with a delta) as read-only, and mark the delta as invalid. Then, any access to the delta can quickly regenerate the page using the full page already in main memory + the difference, while any write to the full page will either update (re-compress) the delta or evict it.

10.3 Fault Tolerance

Checkpoint and Restart

A simple approach to fault tolerance is to have the OS periodically save the process's state to disk, to allow restoring the process state from this *checkpoint* or *snapshot* if the process

crashes. This involves first suspending each thread, saving each thread's registers to memory, and then copying the process's memory to disk.

Note that we must pause the threads when saving its memory; otherwise, we'll have a race condition in which different portions of the state are in different sections of the program. However, we can optimize this a bit to minimize process stalling—we can mark the process's pages as COW, and then reset each to read-only once copying to disk is finished.

This is also helpful for **process migration**, i.e. moving a running program from one machine to another. This is commonly used to load balance VMs across machines in a datacenter.

Recoverable Virtual Memory

Checkpoints are expensive, though. Can we do better in saving virtual memory?

A naive solution is to *log changes*, rather than saving the entirety of virtual memory. However, this would require trapping on *every* write to main memory—a prohibitively expensive operation. We can do something similar, though, at the *page granularity*. Every time the system takes a snapshot, only the pages that have been *modified* are saved to disk. These are called **incremental checkpoint**.

Deterministic Debugging

Debugging a sequential program is easy. Debugging a parallel program is harder, due to non-determinism. Debugging an operating system is even harder, not only due to extensive concurrency but also a result of the difficulty in differentiating what's "input" and "output."

Virtual machines, though, provide a nice abstraction for a deterministic debugging environment of an operating system (and thus, by extension, concurrent programs). Essentially, the hypervisor records every action taken by the guest operating system, and then replays the precise actions. We can also record the exact timing of interrupts to simulate these exactly too!

10.4 Security

TL;DR: we can use VMs as honeypots and use it for defense in depth, since taking over the host OS from a VM requires a privilege escalation followed by a VM escape followed by another privilege escalation.

10.5 User-Level Memory Management

The key idea of this section is that applications may have better visibility into how to manage its own pages, i.e. which pages to evict, page access control, etc. Thus, some operating systems provide some user-level control over memory, commonly in the following two forms:

- **Pinned pages.** Programs can *pin* virtual memory pages to physical pages, preventing eviction unless absolutely necessary.
- **User-level pages.** On a page fault/protection violation, the kernel fault handler will allow the application to manage the fault. (Note: the user-level page handler must be stored in pinned memory, to avoid infinite recursion).

Chapter 11: File Systems

Goals

- Reliability
- Large capacity, low cost
- High performance
- Named data
- Controlled sharing

Nonvolatile storage and file systems.

Main DRAM memory is *volatile*, i.e. it can be lost upon crashes/power failure. *Nonvolatile* storage, like hard disks and flash storage, is persistent/durable, retaining state across crashes/power outages.

One critically important aspect of nonvolatile storage, though, is that data access can be much, much slower (several orders of magnitude) than DRAM access. This plays a significant role in I/O interaction with these devices.

11.1 The File System Abstraction

Formally, a *file system* is an operating system abstraction that provides *persistent, named data*. Persistent meaning stored until explicitly deleted, and named meaning accessible via a human-readable identifier.

There are three major parts to the file system abstraction

- **File.** A named collection of data in a file system. It is typically composed of two parts: metadata and data.
- **Directory.** Effectively a folder that contains files and other folders. Formally, a list of names and a mapping from each name to a file/directory.
- **Volume.** A volume is a collection of *physical* storage resources that form one logical storage device. Typically, a single disk corresponds to one volume, or a single disk is partitioned into multiple volumes. Less commonly, multiple disks can correspond to a single volume.

Typically, the file system is constructed in a "tree-like" fashion. The all-encompassing directory is the *root directory* `/`, and other files/directories form other nodes. Notably, files must be leaf nodes. A string that identifies a file/directory is a *path*, e.g. `/etc/passwd`, and represents the traversal one must take to reach that file from the root directory. A path

starting with `/` is *absolute*, and are interpreted relative to the root directory, while all other paths are *relative*, and are interpreted relative to the *current working directory (cwd)*.

File systems, however, are not exactly trees. They are, in fact, DAGs—directed acyclic graphs. This is due to the possibility of multiple **hard links** to an underlying file, i.e. a mapping between a name and an underlying file. In other words, it's possible for multiple paths to point to the same file. Note that these hard links are typically restricted to avoid cycles, and hence file systems are usually DAGs.

❓ Why no cycles? >

Consider the path `/a/b`. If `b` maps to `/`, then we could have infinitely long paths...

`/a/b/a/b/a/b...`

Many systems also support **symbolic** or **soft links**. This is a directory mapping that, instead of mapping a file name to an underlying file, maps a file name to *another file name*. Though moving the other file name may result in dangling links, symbolic links have two key advantages: (1) systems typically allow symbolic links to directories (because these won't have cycles... traversing a symbolically linked directory will just bring you to the other directory, in contrast to a hard link) and (2) a symbolic link can refer to a file in a different file system or volume (since it just maps to the file name, rather than the underlying file which may have different structure).

Others, namely Windows, support **shortcuts**, too, which are interpreted by a "windowing system." We won't discuss this in much detail.

A single computer, notably, can make use of multiple file systems stored on multiple volumes by **mounting** volumes in a single logical hierarchy.

11.2 API

```
create(path)
link(existing_name, new_name)
mkdir(path)
rmdir(path)

open(path)
close(fd)
```

```
read(fd, buf, len)
write(fd, buf, len)
seek(fd, offset)
mmap(fd, off, len)
munmap(ptr, len)
fsync(fd)
```

11.3 Software Layers

Broadly,

- **API and Performance.** The top levels are responsible for providing a file system API and keeping storage accesses fast.
- **Device access.** Device drivers abstract the details of I/O hardware away to provide a unified interface for operating systems.

API and Performance

System calls and libraries provide the necessary API for file system usage.

The **block cache** keeps recently read/write blocks in main memory, keeping programs performant.

Prefetching, i.e. speculatively retrieving blocks (e.g. during large sequential access), reduces future I/O requests' latency, reduces storage device overheads (replace many small requests with one large request), and improves parallelism for RAID and flash drives. Notably, though, too aggressively prefetching has downsides: cache pressure, I/O contention (with active requests), and wasted effort on inaccurate guesses.

Device Drivers

Translate between abstraction layers of OS and I/O hardware devices. Typically, there at least exist **block device** (read data in fixed block sizes) and **character devices** (read data in stream) interfaces.

Device Access

Memory-mapped I/O provides a way for the OS to communicate with an I/O device by mapping a device's control registers to physical memory addresses on the memory bus. This is the modern solution.

Port-mapped I/O is the legacy solution, and involves specific architecture support (distinct I/O instructions) to communicate with special "ports" that map to device registers.

Direct Memory Access (DMA) allows a storage device to *bypass* the CPU and write data directly from its own memory to the system's main memory. This reduces the CPU's workload (otherwise it would need to copy all that data), and the operating system facilitates this by (1) providing a target physical address to the device and (2) *pinning* the target pages in memory until the device is done, i.e. ensuring they cannot be swapped to disk until explicitly unpinned.

Advanced DMA

Modern systems now frequently use an IOMMU (I/O memory management unit) that translates device virtual addresses to main memory physical addresses, like the CPU's MMU. This provides better protection and abstractions.

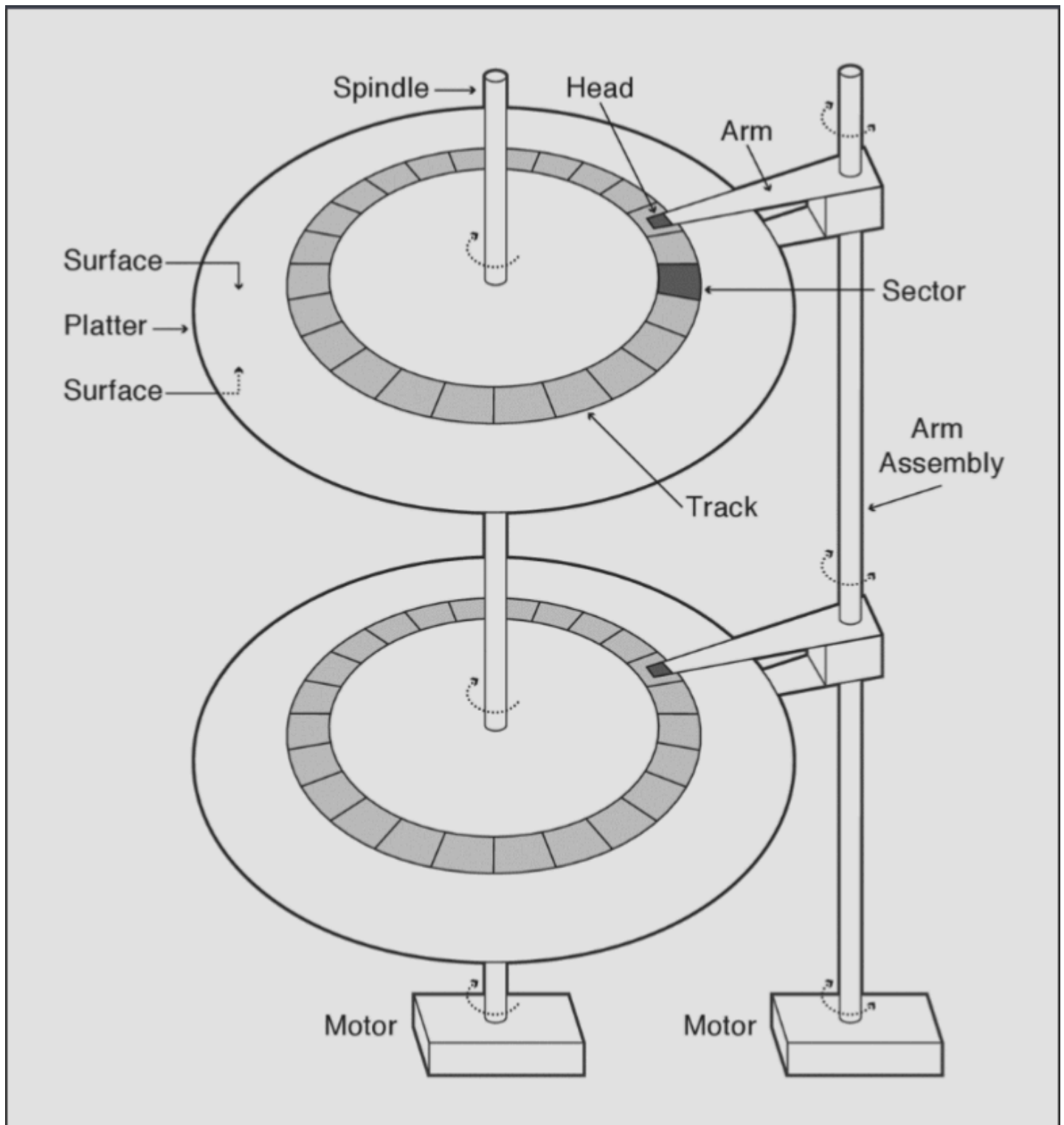
Interrupts are an efficient solution to inform the CPU once an I/O device is done handling a request / received a new external input. This is practically always a better option than **polling**

Chapter 12: Storage Devices

This chapter, we discuss the characteristics of *persistent storage devices*, and how to design our system to be performant in context of the underlying hardware.

12.1 Magnetic Disk

The below diagram showcases the structure of a magnetic disk, also known as a **hard drive**.



A magnetic disk drive is composed of multiple magnetic disks, known as *platters*, stacked on top of each other. A *spindle* connects all the magnetic disks through the middle, and a motor continuously spins the disks when the drive is on. On the side, there is another rod known as the *arm assembly*, which is connected to a motor that rotates it across a 180 degree range on demand (depending on requests) and holds an *arm* for each platter. At the end of each arm is a *head*, which is able to read/write data by sensing/introducing a magnetic field on the corresponding *surface* of its platter. Each platter has two surfaces, one on each side. typically, the disk head will "float" on a layer of rapidly spinning air created by the spinning of the disk platter—this allows the disk head to remain extremely close to the platter.

⚠️ **A head crash occurs when the disk head literally hits the platter. This can irreversibly damage part of the disk, and result in a loss of data.**

Each surface is divided into concentric circles of *tracks*, which are each divided into *sectors*, typically 512 bytes. The arm's rotation allows switching tracks, and the disk's spinning allows reaching the desired sector. Notably, disk hardware cannot read/write individual bytes/words—it must read/write an entire sector.

❓ **Why is this the case? >**

One reason for this is that disks encode each sector with error correction code data.

Logical sector 0 subsequent tracks are also *staggered* by the amount of time it takes the disk to move the head to a different track or switch from to the head on the opposite surface to improve sequential access speed; this is known as *track skewing*. Each disk also includes *spare sectors* distributed across surfaces to account for broken sectors; *slip sparing* helps preserve good sequential access performance by remapping all sectors between the bad sector and then next spare sector.

Disk drives also frequently include a few MB of *buffer memory*, which is used by the disk controller for

1. Buffering data being read from / written to the disk
2. *Track buffering*, i.e. buffering sectors that have been read by the disk head (i.e. while rotating to the desired sector) but were not requested by the operating system (but may be requested in the near future).
3. *Write acceleration*, i.e. the disk stores data to be written in the buffer and acknowledges the writes to the OS before data is actually written to the platter; later, the disk controller flushes the writes.

⚠ Write acceleration can be risky if power is lost before buffered data is safely stored.

Server drives implementing SCSI/Fibre Channel interfaces and commodity drives with SATA interfaces use a *safer* write acceleration technique known as **tagged command queueing** (TCQ) or, for SATA, **native command queueing**. This allows an OS to issue concurrent requests and for the disk to process them out-of-order; it may be configured to only acknowledge writes when blocks are written to the platter.

Disk Access and Performance

An OS request would ask the disk to read/write one or more consecutive sectors. A disk's sectors are identified by *logical block addresses* (LBAs), which specify the surface, track, and sector.

Servicing a request requires three steps: (1) **seeking** to the right track, (2) waiting for the desired sector to **rotate** to the head, and (3) **transferring** the blocks. That is, disk access time is

$$T_{\text{access}} = T_{\text{seek}} + T_{\text{rotation}} + T_{\text{transfer}}$$

1. Seek

A seek involves a disk moving its arm to the desired track. A disk's minimum switch time is the time required to move the head to an adjacent track, while its maximum switch time is the time required to move between the innermost/outermost tracks. A disk's average seek time is frequently approximated as the time required to seek *1/3* of the entire way across the disk. Also, the *head switch time* is the time it would take to switch to a head on a different surface, and has a cost similar to a minimum seek, due to slight misalignments between surfaces.

⚠ Average Seek Time? ▾

Average seek time is *not* typically a good metric, because it assumes zero locality in a workload—which is typically not true. The *minimum* seek time is typically more representative.

⚠ Cylinder? ▾

A *cylinder* is a set of tracks on different surfaces with the same track index. Early file systems placed related data on different surfaces but within the same cylinder; however, this has largely fallen out of popularity since switching disk heads is effectively a 1-track

seek now; thus, most choose to locate such data in adjacent tracks on the same platter instead.

2. Rotate

As you may recall, the disk is always spinning when on. However, after the disk seeks to the correct track, it may not be at the right sector! So, it must wait for a bit—this is called *rotational latency*.

3. Transfer

Once the disk reaches the desired sector, the data must be transferred to/from disk from/to buffer memory. In order to amortize seek and rotation time, which are typically the dominating factors in latency, disk requests are typically batched into requests for multiple sequential sectors. The time to transfer one or more sequential sectors once the disk head finds the first sector is the *surface transfer time*.

The time to transfer data between the host's memory and the disk's buffer is the *host transfer time*. This is on the critical path for reads and possibly writes if the disk does not use write acceleration. Latency, though, is not typically affected by this much since most disks overlap host latency with the other aforementioned processes and this metric is typically not a bottleneck.

? Random Access? >

We've talked a lot about prioritizing sequential access performance. But what about workloads with largely random accesses? Well, as alluded to previously, disks typically deliver bad performance with random access workloads—this is because the seek and rotation latencies are *not amortized*.

Disk Scheduling

As aforementioned, seek and rotate latencies are *very expensive*. So, it begs the question—how can we optimally schedule/order the disk requests we must serve to optimize for low latency and high throughput?

The simplest scheduling algorithm is just **FIFO**, but this can, of course, result in very poor performance depending on the workload. The worst case involves requests that alternate between the outer and inner tracks of a disk.

SPTF (shortest positioning time first) or **SSTF** (shortest seek time first) may seem effective, given that SJF was optimal for throughput for CPU scheduling. In essence, given the current

position of the disk head and platter, service the quickest pending request. However, not only do these algorithms suffer from starvation, like SJF, these algorithms' scheduling does not necessarily minimize the *aggregate* seek and rotation times across an entire workload. This is because choosing to serve one request actually affects the seek/rotation times of other requests, in contrast to SJF!

Thus, some entirely disk-specific algorithms have been developed: *elevator*-based algorithms.

In essence, an elevator-based algorithm functions similarly to how an actual, real-life elevator services requests: when going up, it continues until all pending requests to go to above floors are serviced; then, when going down, it continues until all pending requests to go to below floors are serviced.

SCAN functions in the exact same way; the disk arm sweeps from the inner to outer tracks, servicing all requests between the current position and the outer edge. Then, it does the equivalent when sweeping from the outer to the inner tracks.

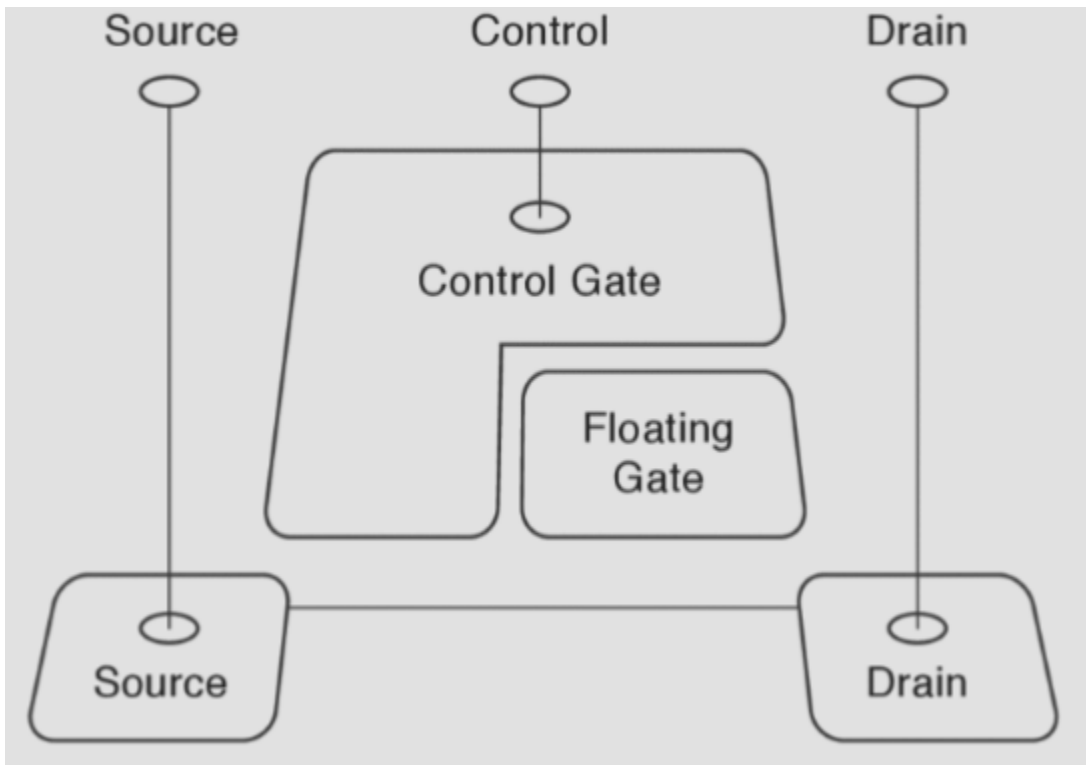
CSCAN, i.e. *circular scan*, functions similarly; except, the disk only services requests when the head is traveling in one, predetermined direction (inner to outer, or outer to inner). When the last request in this direction is service, the disk seeks to where it started without servicing requests. The advantage of *CSCAN* over *SCAN* is that, after a pass in one direction, the disk head would currently be in a region of disk with sparse requests, since it was just serviced; *CSCAN* moves the head to an area with (probably) greater request density. Also, *CSCAN*, by the same idea, is fairer.

R-SCAN and *R-CSCAN* are variants that also account for *rotation time*. (*SCAN* and *CSCAN* primarily attempt to amortize seek time). In essence, the R variants allow small seeks in the "wrong direction" if a sector on a previous track is very close with regards to the rotation proximity.

12.2 Flash Storage

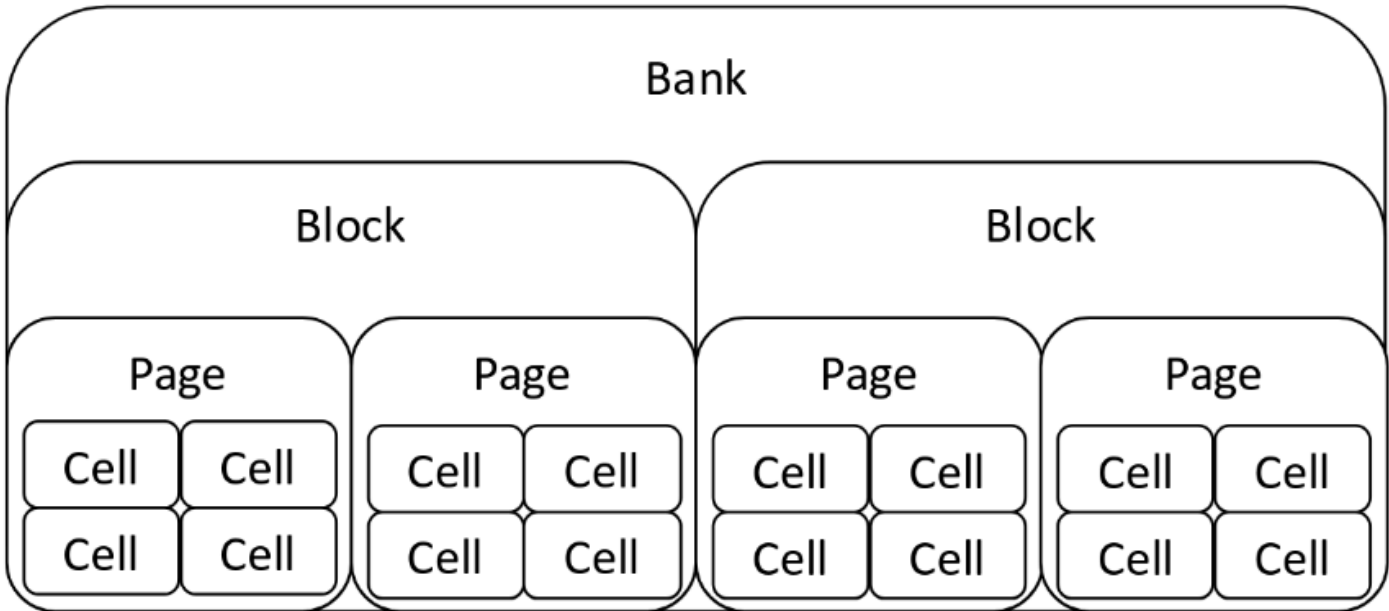
Flash storage is a type of *solid state storage*—it has no mechanical components and stores data via its circuitry and electricity. With its lack of mechanical parts, flash storage has much better random I/O performance than disks, can use less power, and are less susceptible to physical damage. At present, though, it remains more expensive.

The minimal unit of storage in flash is a *floating gate transistor*, known as a *cell*. It typically stores a single bit (single-level flash storage) but can also store multiple (multi-level flash storage).



There are multiple types of flash storage, depending on the control gate used to store a bit. **NOR** flash allows individual words to be read/written, and is useful for storing device firmware since it can be executed in place. **NAND** flash must be read/written on a page granularity, but is denser and is thus more commonly seen.

Here is the structure of a **bank** of flash storage—each flash chip has multiple banks, which can be accessed in parallel.



There are **3** primary operations associated with flash storage

- **Erase erasure block.** Before flash memory can be written, it must be erased by setting each cell to a logical 1. Moreover, flash memory can only be erased in units called *erasure blocks*, which are often quite large. Erasure is also *slow*, on the order of several milliseconds. (Erasure is also what gives flash memory its name!)
- **Write page.** Once erased, NAND flash can be written page-by-page, an operation on the order of tens of microseconds.
- **Read page.** NAND flash can be read similarly on a page-by-page basis, with the same latency as writing a page.

The necessity of *erasing* an entire erasure block before writing is the biggest challenge facing flash storage. Most flash storage devices implement a **flash translation layer** (FTL) that maps logical flash pages to different physical pages on the flash device. Then, when a logical page is overwritten, the new version is written to a free, already-erased physical page, and the logical page is *remapped* to the physical page. This is known as **write remapping**, and significantly improves flash performance (since erasure is no longer necessary for every single write).

It's also necessary to discuss the **durability** of flash storage. Flash memory can retain its persistently stored data for years without power; however, flashing and writing memory involves high current loads, which results in circuit degradation—consequently, a cell can *wear out* and become unreliable! (Also, multi-level cells wear out faster).

Also, reading a cell many times can disturb nearby cells' charges, inducing a *read disturb error* that changes a cell's value due to excessive reads to nearby cells.

Flash devices implement several techniques to account for their durability issues.

- **Error correcting codes.** Each page has a few extra bytes for extra correcting codes, to resolve any bit errors.
- **Managing bad page and bad erasure block.** The device firmware will mark a failing page/erasure block as bad.
- **Wear leveling.** As aforementioned, FTL uses write remapping to keep write latency low. This has the added benefit of ensuring that a frequently used hot page does not wear out a physical page on the flash device. Some devices extend this by migrating unmodified pages to prevent read disturb errors.
- **Spare pages and erasure blocks.** Flash devices frequently possess spare pages and spare erasure blocks. This serves 2 purposes: (1) extra space for device wear leveling management, even if the device is logically full and (2) enables bad page/erasure block management to function without shrinking the device's logical capacity

Wear also affects a flash device's performance: (1) accesses may require additional retries (due to errors) and (2) as spare pages and erasure blocks are consumed by bad ones, the wear leveling algorithms have less spare space and must frequently *garbage collect* live pages.

Chapter 13: Files and Directories

Four major challenges

- Performance
- Flexibility
- Persistence
- Reliability

We will discuss the first three in this section.

13.1 Implementation Overview

A file system's primary job is to map a file name and offset to a persistent physical storage block. It does this in two steps

1. It maps a file name and offset to a file number, typically traversing through several **directories** to retrieve the file number. The directories are generally arranged in a tree-like structure (though, technically a DAG)
2. The file number is associated with an **index structure**, which is a persistent data structure that maps the file number and offset to a specific storage block. This is frequently some sort of tree.

File systems also use **free space maps** to track free storage blocks, which allow file systems to allocate and free blocks as files grow and shrink. This is typically implemented as a bitmap, and frequently allows finding free blocks around a desired location, to support *spatial locality*.

File systems further utilize several **locality heuristics** to group data for optimizing performance; for instance, magnetic disk drives are, at times, *defragmented* to improve performance of future accesses.

13.2 Directories

Directories are simply a way for file systems to group related data together, to make it easier for users. They're typically arranged in a tree-like structure, and each directory is actually stored as a file itself, known as a directory file!

Directory API

In order to prevent corruption of the special directory file structure, there exists a specialized API for modifying directory files. In particular, there are `mkdir`, `link`, `unlink`, and `rmdir` syscalls.

But how is a directory file actually formatted? Early implementations simply had linear lists of (file name, file number) pairs. For instance, Linux's original ext2 file system was a linked list of these pairs. (Linked list instead of an array because directory entries could be removed). This is fine for directories with few entries, but as systems grew, some workloads generated thousands of files in a directory—meaning access times slowed down ($O(n)$, so scales linearly with number of entries). Thus, more recent file systems organize directory entry contents as a tree, typically some sort of balanced binary search tree.

Also, as mentioned in [Chapter 11](#), there exists two types of links that can "duplicate" files. We include the same explanation here.

File systems, however, are not exactly trees. They are, in fact, DAGs—directed acyclic graphs. This is due to the possibility of multiple **hard links** to an underlying file, i.e. a mapping between a name and an underlying file. In other words, it's possible for multiple paths to point to the same file. Note that these hard links are typically restricted to avoid cycles, and hence file systems are usually DAGs.

Many systems also support **symbolic** or **soft links**. This is a directory mapping that, instead of mapping a file name to an underlying file, maps a file name to *another file name*. Though moving the other file name may result in dangling links, symbolic links have two key advantages: (1) systems typically allow symbolic links to directories (because these won't have cycles... traversing a symbolically linked directory will just bring you to the other directory, in contrast to a hard link) and (2) a symbolic link can refer to a file in a different file system or volume (since it just maps to the file name, rather than the underlying file which may have different structure).

Now, you may understand how exactly this works. Any hard links to a file map to the same underlying file number, while a symbolic link just maps to another file name itself (which eventually resolves to a file number).

❓ **Most file systems store file metadata in a file header that is found with the file number. Why?** ▼

Hard links make storing file metadata in directory entries difficult, as it would require duplicate information and updates.

13.3 Files

Now, we discuss the more complex part of file systems: how to organize the data associated with a file number. This part of the file system is complex, and sees many different designs,

because it must attempt to achieve several goals.

- Support sequential locality
- Provide efficient random access
- Limit overheads for small files, but facilitate very large files
- Enable the storage of per-file metadata

This section, we'll discuss several popular designs.

- **FAT** (Microsoft File Allocation Table) is the oldest (late 1970s), and uses a simple *linked list* index structure that supports granularity at the block level. It uses a FAT array for managing free space, and the only method for supporting sequential locality is periodically defragmenting the file system. While largely a legacy choice for a computer's primary file system, it remains prevalent for small USB drives, embedded devices, and boot drives where file systems are expected to be kept relatively compact.
- **FFS** (Unix Fast File System) was released in the mid 1980s, and inspired the ext2 and ext3 file systems that are default on most Linux systems. It uses a *tree-based multi-level index* for its index structure (explained soon :-), which can be described as fixed and asymmetric, uses a fixed bitmap for managing free space, supports block-level granularity, and uses block groups and reserved space to improve sequential locality.
- **NTFS** (Microsoft New Technology File System) was introduced in the early 1990s to replace FAT, and is the default today on Windows systems. It uses a *dynamic* tree-based structure, and supports *extent*-level granularity, effectively variable-sized blocks (extents are to blocks as segments are to pages). It also uses a bitmap for tracking free space, and uses best fit and defragmentation as its locality heuristics. Its use of a flexible tree and extents has inspired Linux's ext4, XFS, Apple's Hierarchical File Systems (HFS/HFS+), and more.
- **ZFS** (Zettabyte File System) was designed by Sun Microsystems in the early 2000s. It featured a dynamic tree, block-level granularity, a *log-structured space map* for free space management, and locality improvements via block groups and a *write-anywhere* policy (explained later). However, its main innovation is that it's a **COW** (copy-on-write) file system, which provides strong integrity and optimizes write performance.

In summary,

	FAT	FFS	NTFS	ZFS
Index structure	linked list	tree (fixed, assymmetric)	tree (dynamic)	tree (COW, dynamic)
Index structure granularity	block	block	extent	block
Free space management	FAT array	bitmap (fixed)	bitmap in file (file)	space map (log-structured)
Locality heuristics	defrag.	block groups reserve space	best fit defrag.	write-anywhere block groups

Figure 13.8: Summary of key ideas discussed for four common file systems approaches.

⚠ Warning

FAT and NTFS refer to blocks as clusters, NTFS refers to extents as runs (proprietary Microsoft as usual...). The textbook uses block and extent, which are more commonly used, and these notes will follow this.

FAT

FAT is quite simply named after its index structure, the **file allocation table**. Its effectively an array of 32 bit entries in a **reserved area** of the volume. Each file is backed by a linked list of FAT entries; each entry consists of just a pointer to the next FAT entry (really, its index in the FAT, or a special EOF value). Meanwhile, the FAT itself had a 1-to-1 correspondence with the backing device; the first FAT entry corresponded to the first block, etc.

The file number found by traversing directories was the index of the **head** FAT entry for the file. Free space was simple; unused FAT entries are marked 0, and so it sufficed to linearly scan the FAT array for a free entry.

Generally, FAT's locality heuristics were simplistic and resulted in significant fragmentation (e.g. next fit algorithm, start from last allocated entry and return next free entry). Hence, the only way to improve FAT's locality was to periodically defragment.

In summary, the FAT system, while simple and portable to practically all operating systems, has several limitations.

- **Poor locality**

- **Poor random access**
- **Limited file metadata and access control**

No metadata like file owner/group id

- **No support for hard links**

File metadata is stored in the directory entry since it cannot be stored in the FAT, so no hard links—how would you find all the other links when modifying metadata?

- **Volume and file size limitations**

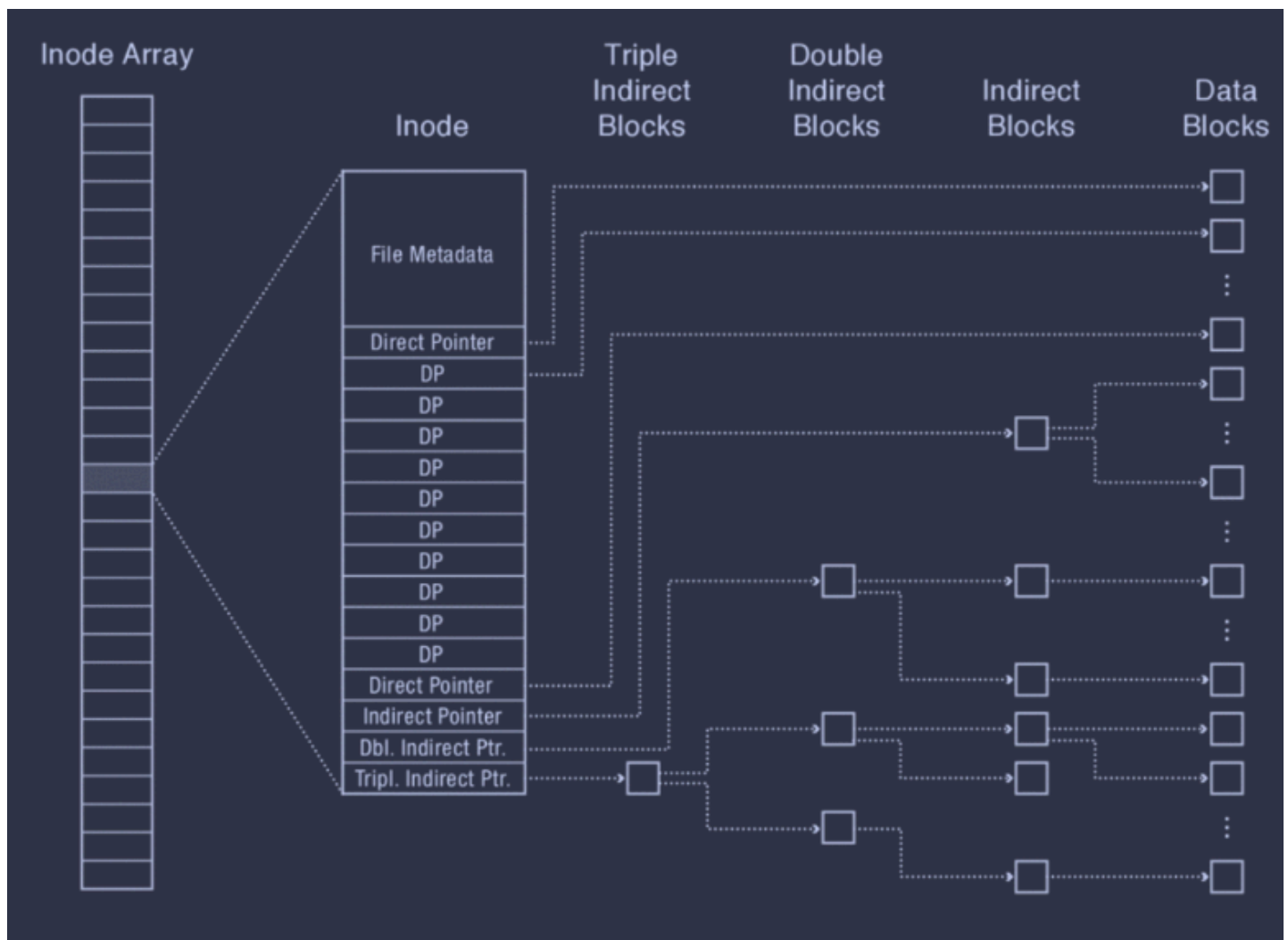
FAT-32, the most commonly used version today(?) supports at most 2^{28} blocks, i.e. 1 TB with 4 KB block sizes, and $2^{32} - 1$ byte file sizes

- **No support for modern reliability techniques**

i.e., those discussed in Chapter 14.

FFS

FFS's index structure is a *multi-level index*, an asymmetric tree structure that provides good efficiency for small and large files. It can be succinctly summarized by the following diagram.

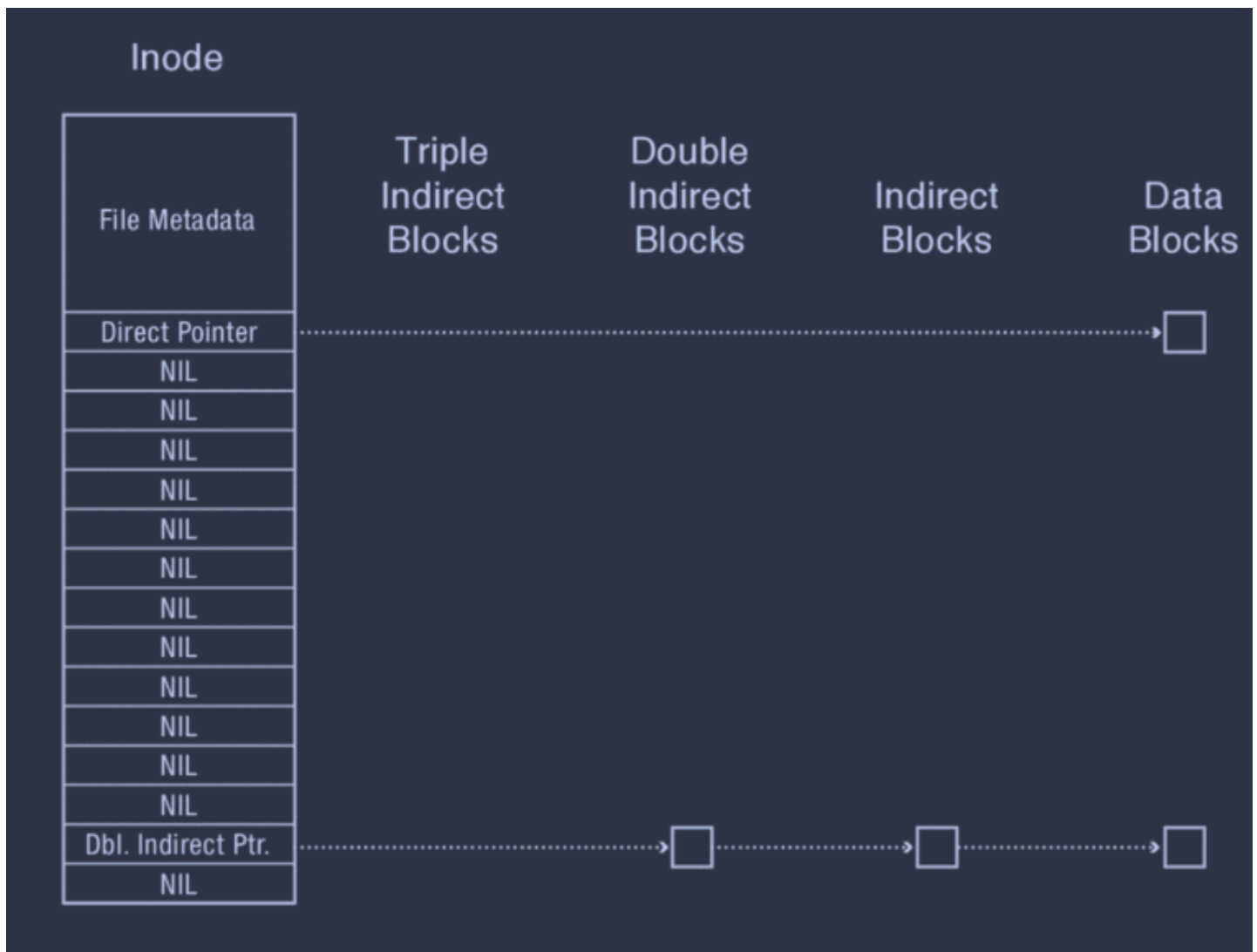


In essence, each **inode** represents a file. It begins with file metadata, several direct pointers, and then several different indirect pointers. The direct pointers directly store the block numbers of their corresponding data blocks. However, the indirect pointers have some level of indirection; the double indirect pointer, for instance, points to a double indirect block first, which itself contains pointers to indirect blocks, each of which contains pointers to the actual data blocks.

Also, note that all inodes are stored in an **inode array** that lives in a fixed location on disk, and a file number is called an **inumber** in FFS, and corresponds to that file's index in the inode array.

This asymmetric structure allows storing large, extensible files without significantly increasing the space or runtime overhead for smaller files. Small files only require one access per data block (using the direct pointers only) while larger files are able to support both efficient random accesses (since it's a tree structure) and good sequential accesses, since the degree of the indirect nodes are extremely large (i.e. reading an indirect block means hundreds of data blocks can be read before the next indirect block must be read).

FFS's tree also has a fixed structure, which is much easier to implement. Additionally, FFS can effectively support **sparse files**, i.e. files that are largely composed of empty space (null bytes), as these ranges of empty space can be represented as null pointers in FFS. In other words, FFS can perform **lazy allocation** of disk blocks.



Sparse files do have some limitations, though. Not all file systems support them, so they're not very portable. Also, programs may not handle sparse files correctly, i.e. reading a sparse file and writing each byte to another file results in a non-sparse file with many null bytes.

Regarding *free space management*, FFS just uses a **bitmap** in a fixed, known location. Regarding locality heuristics, FFS has two: block group placement and reserved space.

Block Group Placement

This heuristic involves:

1. Dividing a disk into *block groups* (where blocks within the same group typically have good locality/low intra-group seek times)
2. Distributing *metadata* (inode array, free space bitmap) across block groups, where the metadata tracked within a block group typically corresponds to its block group
3. *Placing files* in a block group according to spatial locality, e.g. files within the same block group as its directory. Note that subdirectories are typically placed *separately* from their parent directories; otherwise, the block group may fill quickly.

4. *Placing data blocks* for the same file within the same block group. FFS uses a seemingly counterintuitive strategy of writing a block to the **first free** block in the file's block group. This heuristic sacrifices short term locality, as it fills up various holes in the start of a block group; however, it benefits long term locality, as this will reduce fragmentation and typically result in a long run of free space at the end.

The idea is that, for small files, its blocks will end up distributed across several holes—an acceptable sacrifice for smaller files. But, for large files, the majority of its blocks will end up in the consecutive free space at the end, providing the good sequential locality critical for larger files.

Reserved Space

This heuristic is simple. Since the block group heuristic is only really effective when there is enough free space on disk, FFS reserves some fraction of the disk space. If the actual disk free space falls below this fraction, FFS considers the disk full for all but a super user's processes. This allows the super user to reduce disk usage / increase storage capacity, while keeping the file system efficient.

Rotational Delay

Consider the following situation: the disk reads a block, does some processing, and then tries to read the next block. But, in the meantime, the disk has continued turning (as a magnetic drive constantly rotates), and the next block is already gone! How can we solve this?

Solution 1: **Skip Sector Positioning** (Interleaving). Essentially, just place blocks of a file on every other block of a track, i.e. leave blocks in between.

Solution 2: **Read-Ahead**. Predict that the application will require the next block, and read the block anyways. Many modern disk controllers have enough internal RAM to hold a complete track in memory!

Limitations

FFS, despite efficiently storing both small and large files and requiring zero defragmentation, has some limitations.

- It's inefficient for tiny files (e.g. files smaller than a couple data blocks)
- It's inefficient for files with large contiguous segments on disk (must break it up into blocks anyways)
- It needs to reserve free space

NTFS

NTFS uses a tree like FFS, but this tree is *flexible* and tracks dynamically-sized *extents*, not blocks.

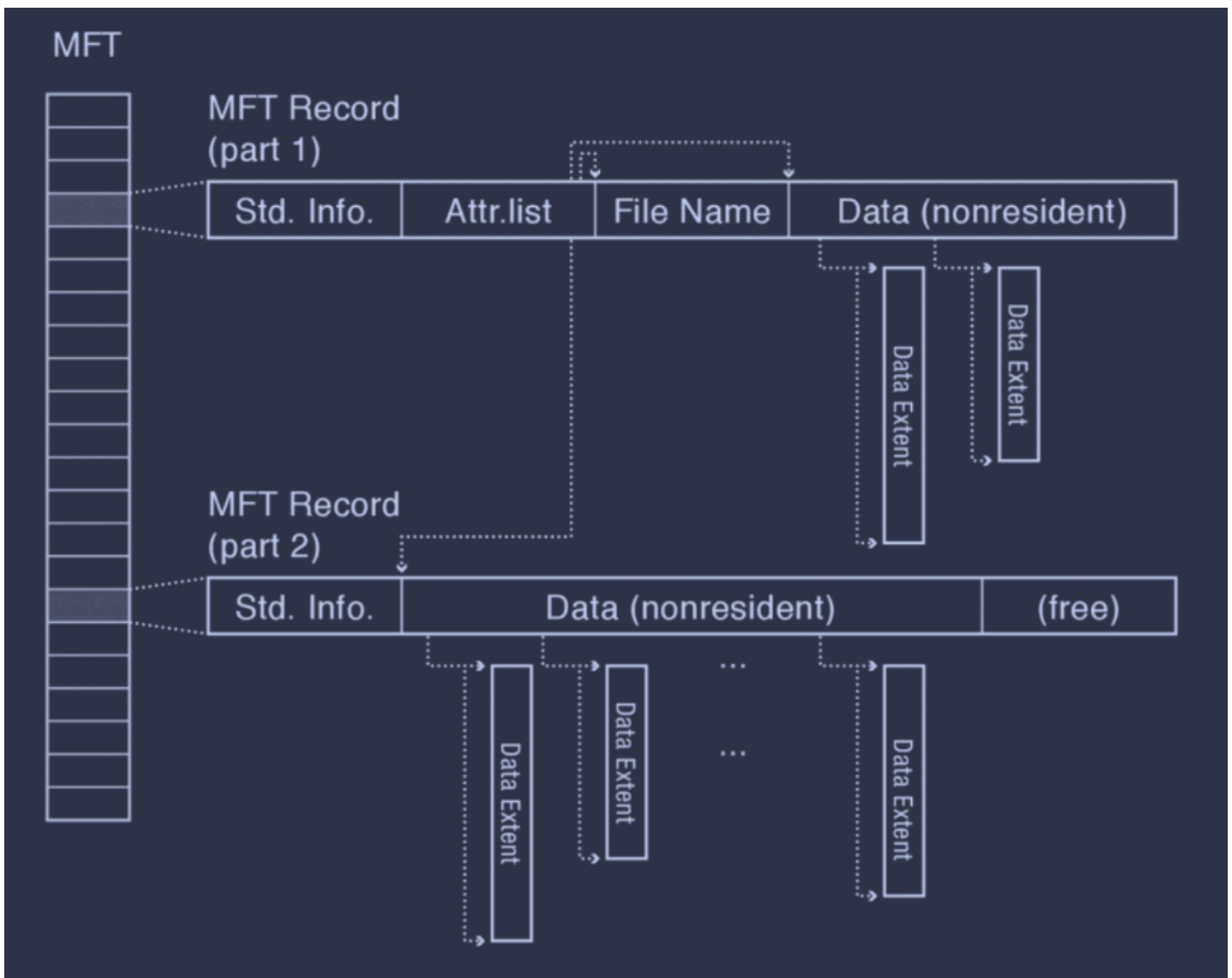
The root of the trees is stored in a **master file table (MFT)** similar to FFS's inode array. Each element in the MFT is a 1 KB MFT record, which stores a sequence of variable-sized **attribute records**, which store data or metadata.

Attributes can be too large to fit in a record. These are known as *nonresident* attributes (while those that can fit are known as *resident* attributes), and a non-resident attribute stores extent pointers in its MFT record and its contents in those extents.

Beyond simple data attributes, there are a few common metadata attributes.

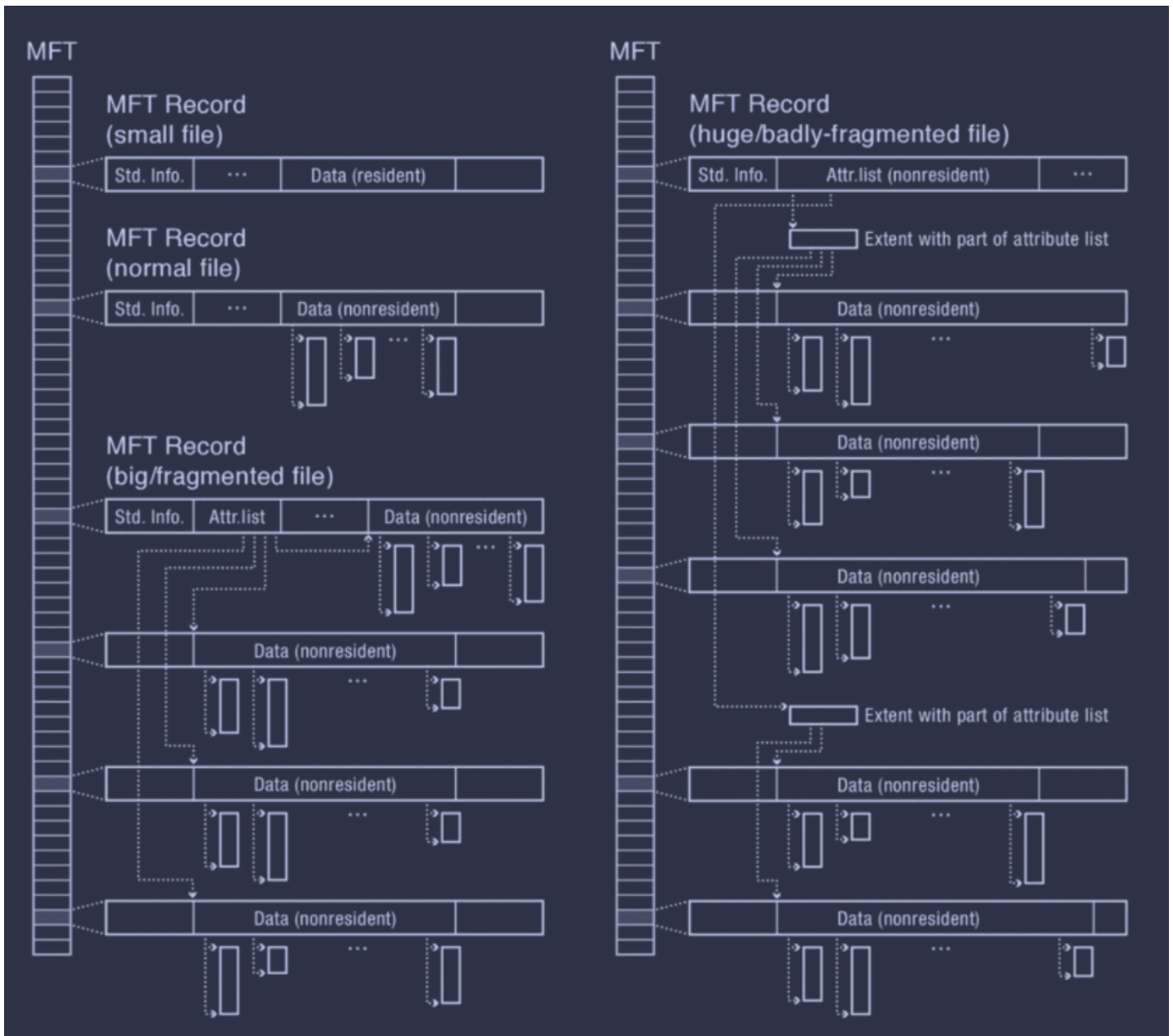
- **Standard Information.** This holds file creation time, owner ID, etc.
- **File Name.** An MFT can have multiple file name attributes, if there are multiple hard links to this file.
- **Attribute List.** A file's metadata may have many variable-sized attributes; thus, it may require more MFT records. In this case, NTFS stores the attributes in multiple MFT records, and the attribute list of the parent MFT record will include pointers to these MFTs.

The below diagram shows an example of a file with an attribute list and both resident and non-resident attributes.



A file in NTFS may go through four stages of growth, depending on size and fragmentation.

1. Small file has all contents included in MFT record, with a resident data attribute.
2. File has a small number of extents tracked by a non-resident data attribute.
3. File becomes fragmented, i.e. too many extents, and has an attribute list in the root MFT record pointing to other MFT records with non-resident data attributes.
4. File becomes huge or has extreme fragmentation, in which case the attribute list becomes non-resident.



Metadata Files

NTFS also stores all of its metadata, e.g. free space bitmap, root directory, volume bad blocks, etc., in a few files with well-known file numbers. Even the MFT is stored as a file, with file number 0. Though, to locate the MFT, the first sector of an NTFS volume includes a pointer to the first entry of the MFT (it's not actually possible to locate the MFT based off the file number... the file number is the index in the MFT!). However, storing the MFT in a file has its benefits; it makes it possible to expand and shrink the MFT as needed.

Locality Heuristics

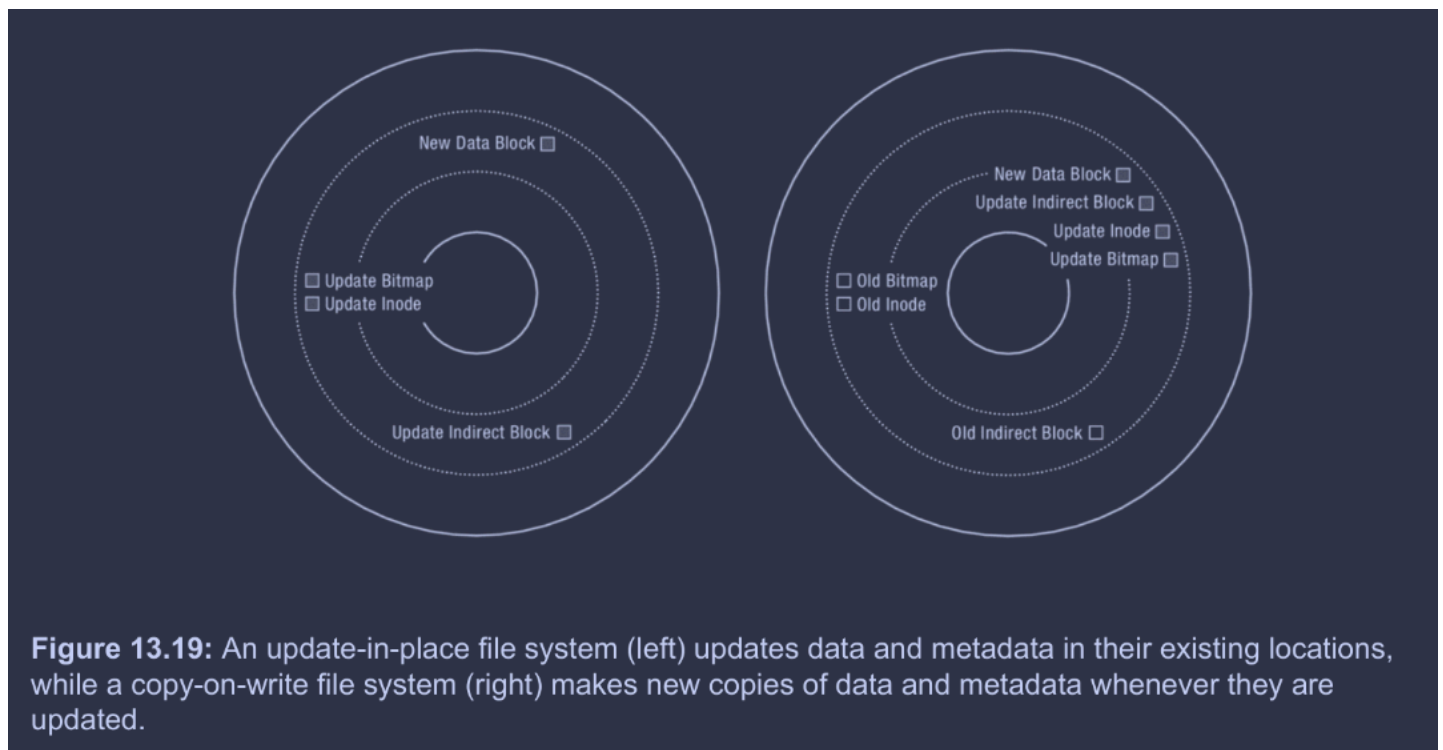
Most NTFS implementations use some variation of *best fit*. To assist this, there's even a `SetEndOfFile()` API that allows specifying a file's expected size at creation time.

Also, to avoid MFT fragmentation, NTFS reserves part of the volume's start for MFT expansion. If the non-reserved area becomes, full, however, it halves the size of the MFT reserve area and continues. This is repeated until this is not possible (MFT takes up more than 50% of the reserve area).

Finally, Microsoft provides a defragmentation utility for its NTFS file systems.

ZFS (COW)

Here's a quick visualization of update-in-place vs. copy-on-write systems.



Benefits of COW

- **Small writes are expensive.** Large sequential writes are simply much better than small random writes, and COW turns update-in-place's small random writes into large sequential writes.
- **RAID. Redundant arrays of inexpensive disks** (RAID) is a topic we'll discuss more in Chapter 14. However, point is, is that RAID is becoming increasingly popular, and it makes small writes even more expensive.
- **Caches filter reads.** Large DRAM caches handle most reads, but cannot entirely buffer writes—otherwise, this sacrifices durability/reliability. Therefore, write performance dominates.
- **Flash storage.** Flash storage's lifetime is improved by *wear leveling*, i.e. spreading writes evenly across cells. The FTL (flash translation layer) already uses COW—so COW file

systems are, by nature, more compatible with flash.

- **Versioning**. COW file systems enable versioning, e.g. Git commit histories. As storage capacities grow larger and larger, versioning becomes more appealing.

Implementation Principles

We'll discuss ZFS, a COW file system that uses an index structure like FFS.

First, ZFS replaces the typical inode array that serves as the root of FFS with an **uberblock**. Whereas FFS's inode array stored the inode data itself in the the inode array, ZFS, as COW system, cannot do this, since the inode data should not be updated in place. Instead, each entry of the uberblock (essentially) contains a pointer to the **root dnode**, which resolves to a dnode array (which is essentially the inode array).

The root dnode is decoupled from the uberblock in order to keep the root dnode COW; a new root dnode can be created first, and then the pointer to the root dnode can be updated in the uberblock.

Notably, the uberblock itself is not in a fixed location; instead, an **array** of 256 uberblocks are kept in a fixed location, each with an associated checksum. On a root dnode update, the **next** uberblock is actually updated with a pointer to the new root dnode. This helps ZFS makes the uberblock COW too—the idea is that this will protect the uberblock from errors after system crashes. On restarts, ZFS scans the uberblock array for the uberblock with a valid checksum and the highest sequence number.

Note that the dnode essentially just replaces the inode in functionality. Each **dnode** has space for three block pointers, and a field for the depth of the tree rooted at this dnode. If the depth is...

0. The dnode stores the file's data itself.
1. The dnode's block pointers are direct pointers.
2. The dnode's block pointers are indirect pointers.
3. The dnode's block pointers are double indirect pointers.
4. etc.
5. etc.
6. etc.
7. etc.

for up to six levels of indirection!

Also, data block and indirect block sizes are variable, actually; their sizes are specified in a file's dnode.

The root dnode itself is the root of a tree that eventually resolves to leaves of dnode arrays. Together, these dnode arrays represent the inode array of FFS. Each dnode in a dnode array represents a file, and is the root of a tree that resolves to leaves of data blocks. An example (though simplified from ZFS's full structure) is displayed below.

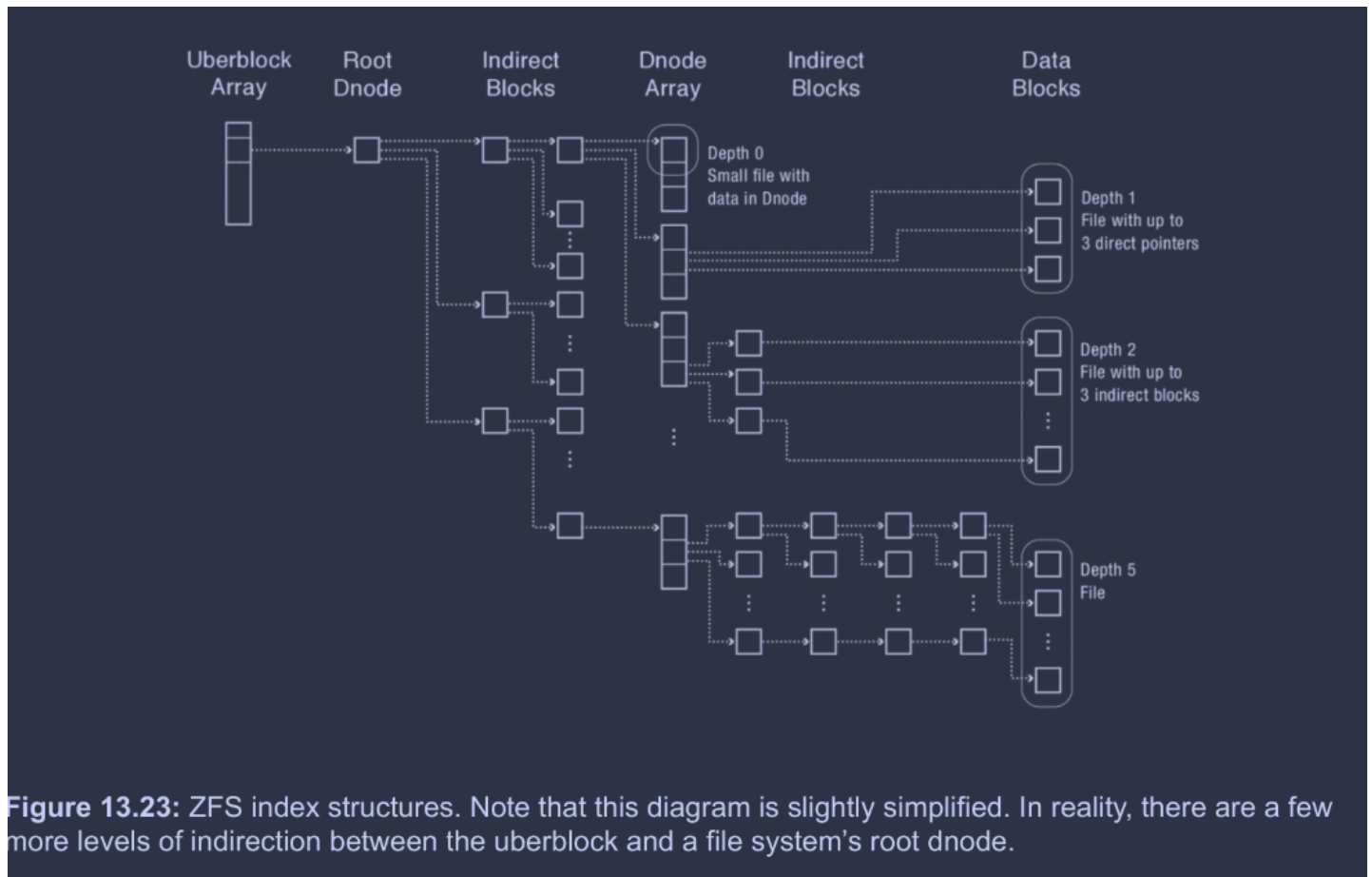


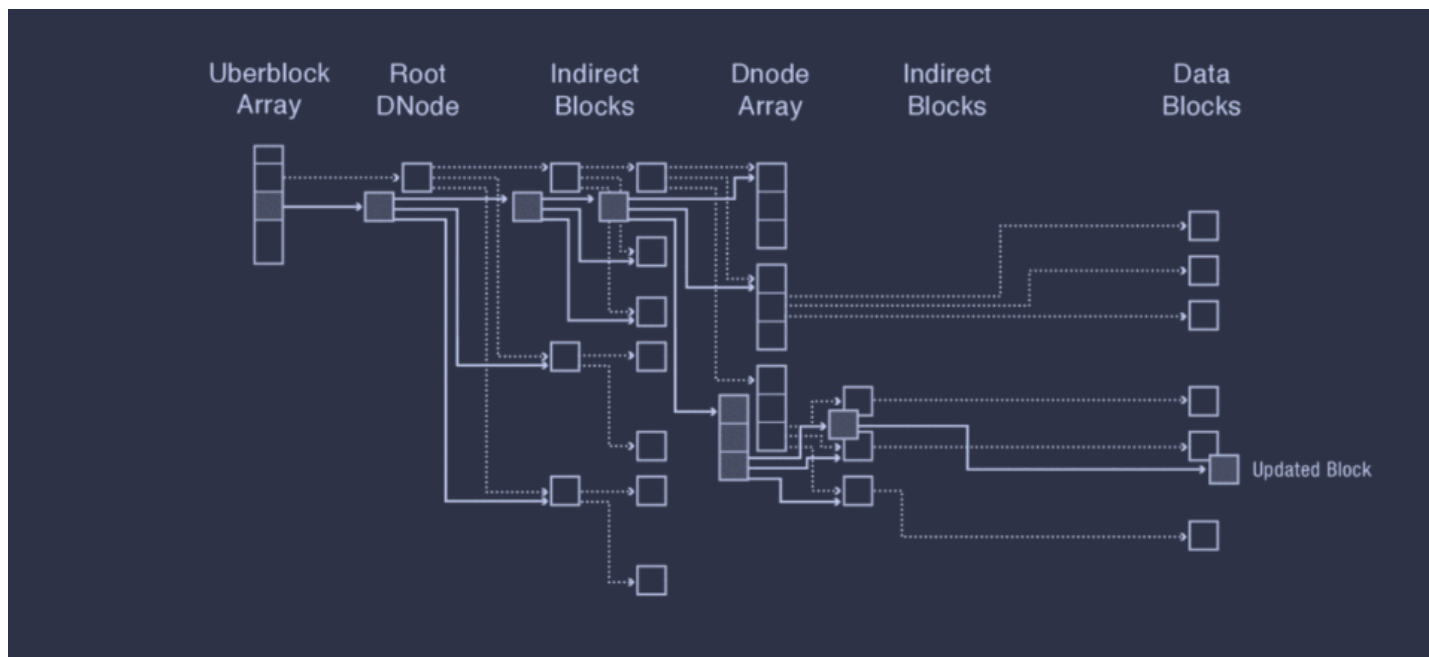
Figure 13.23: ZFS index structures. Note that this diagram is slightly simplified. In reality, there are a few more levels of indirection between the uberblock and a file system's root dnode.

⚠ **Make sure you understand this before moving on! It took me a while to understand what the textbook was saying in this section, so I'll reiterate here if you're still confused.** >

The uberblock is the root of the whole file system. The uberblock array just allows for the uberblock to be COW, with checksums to denote which uberblocks are valid after crashes. The uberblock contains a pointer to a root dnode; this ensures the root dnode is COW as well. The root dnode, through some number of indirect blocks in a tree structure, resolves to leaves, each representing a subset of the dnode array. The motivation for turning the dnode array into a tree structure is that it's more efficient for COW (see the diagram below for visual clarification). Each element in the dnode array represents a file,

and, through 0 or more indirect blocks in a tree structure, resolves to leaves of data blocks (or even data within the dnode itself).

Finally, here's a diagram showing the process of updating the last block in a 2-level ZFS file.



Critically, note how the tree structures for storing the dnode array and the data blocks for each file is incredibly helpful for making COW efficient, as it reduces the amount of content that must be copied.

ZFS Space Map

ZFS's space maps are designed to efficiently scale with extremely large storage systems. They use three key ideas.

- **Per block group space maps.** ZFS has a space map for each block group, and restricts new block allocation to a subset of block groups at any given time, and keeps those block groups' space maps in memory (reducing allocation overheads).
- **Tree of extents.** Each block group's free space map is represented as an *AVL tree of extents*. This allows efficient resolution of a free extent of desired space as both time and space scale logarithmically with the block group size, rather than linearly as it would with bitmaps. (Technically, worst case space scaling is linear; however, on expected space scaling is logarithmic).
- **Log-structured updates.** Caching a space map in memory (i.e. first bullet point) effectively speeds up allocations, but not frees. Thus, rather than updating the on-disk space map, ZFS appends space map updates to a log. When a block group is activated for

allocation, ZFS performs all the logged updates at once, effectively batching all updates until the next use of the space map.

ZFS Locality Heuristics

COW systems like ZFS, though, must perform a lot of work just to update a block. ZFS includes some nice heuristics to optimize write behavior.

- **Sequential writes.** Aslogarithmically alluded to in the [beginning of this section](#), the updates can frequently be grouped into a single write of a free range of sequential blocks on disk. (Excluding the uberblock). Sequential writes are much faster than random writes, and thus make up for the increased quantity of metadata updates.
- **Batched updates.** ZFS also batches file writes. Rather than writing each update to disk one-by-one, ZFS batches several seconds of updates and writes the entire update to disk, effectively amortizing the updates to all the dnodes and indirect blocks.

Additionally, ZFS cleverly chooses where to write new block versions in three steps.

1. **Choose a device.** ZFS uses a WRR (weighted round robin) policy based on the amount of free space left in each device. ZFS also places ~512 KB on one device before switching to maintain good locality for future reads—almost like a quanta in scheduling RR!
2. **Choose a block group.** ZFS's first choice is to use the most recent block group. If it's too full or fragmented, though, ZFS selects a new block group, biasing towards those with more free space, closer to the outer edge of the disk (for improved bandwidth), and that have recently been used (to limit seek distance).
3. **Choose a block within the group.** First fit allocation until almost full, then ZFS switches to best fit.

Chapter 14: Reliable Storage

"The central question of this chapter is: How can we make a storage system more reliable than the physical device out of which it is built?"

The question of reliability actually revolves around **3 principles**.

- **Reliability** - the ability of a system or component to perform its required functions under stated conditions for a specified period of time (IEEE definition)
- **Availability** - the probability that the system can accept and process requests (the proportion of time for which a system works as intended, with regards to access)
- **Durability** - the ability of a system to recover data despite faults

Persistent storage systems, in particular, must deal with two threats to reliability.

- **Operation Interruption** - some sort of failure or crash midway through an operation can leave data in an *inconsistent* state, which the system cannot make sense of.
- **Loss of Stored Data** - non-volatile storage media can fail, causing data to be lost.

To address this, there are two mainstream solutions, one for each issue.

- **Transactions** - essentially, a way to make a sequence of non-atomic operations function as an *atomic unit*. The system's state either reflects the entire sequence's changes, or none.
- **Redundancy** - checksums, error correcting codes, or any other way of cleverly "duplicating" data to detect or recover from lost or corrupted storage devices.

14.1 Transactions

Pre-Transactional Approaches

Many older file systems used ad-hoc approaches to solve the problem of interrupted operations.

FFS, for instance, would simply ensure that any changes to the disk were always completed in a specific order. Then, if a crash occurred during a group of updates, a disk scan could identify the inconsistency and repair the data structures. This disk scan, for FFS, was the `fsck` utility.

However, this approach has largely been abandoned during modern times for a few reasons.

1. **Complex Reasoning**. Like multi-threaded concurrency, this approach quickly becomes complicated to reason about as systems grow more complex.
2. **Slow Updates**. File systems are forced to insert barriers between dependent operations, reducing pipelining and parallelism.
3. **Slow Recovery**. After a crash, a machine must scan *all* of its disks for inconsistencies. As persistent storage rapidly grew, the resulting overhead grew larger and larger.

Transaction Abstraction

As aforementioned, a **transaction** abstracts a sequence of several operations as a single, atomic unit. A transaction finishes in one of two ways, it...

- *commits*, meaning the effects of its underlying operations are visible in the system.
- *rolls back*, meaning the effects are not reflected by the system.

A transaction, at its core, aims to fulfill the four **ACID** properties. (These are most commonly referenced in database system design, but really apply to *any* system desiring reliability).

- **Atomicity** - updates are "all or nothing." Transactions that *commit* see all updates take effect, while those that *roll back* see zero updates take effect.
- **Consistency** - the system is always in a legal state, i.e. a transaction brings the system from one legal state to another. More formally, a transaction guarantees that the system's invariants that hold at the start of the transaction also hold at the end of the transaction.
- **Isolation** - transactions appear to execute on their own. In other words, concurrently executing transactions do not affect each other.
- **Durability** - once a transaction is successfully committed, its changes are permanent and will not be lost, even after system failures.

Transaction Implementation

A transactional file system must deal, of course, with the inherent fact of persistent storage devices—their only atomic operation is writing a single sector or page. Typically, they do this by, before executing a transaction itself, recording a transaction's *intentions* in persistent storage, i.e. the operations it intends to perform. Only once these are all safely stored, the transaction should begin to perform its operations; any interruption can be recovered by the system re-executing the stored intentions.

Redo Logging is one way of doing exactly this. It consists of four stages.

1. **Prepare**. Append all planned updates to the log.

2. **Commit**. Append a commit record to the log, indicating that the transaction has committed. (Or, it may be rolled back. In this case, simply not writing a commit record may suffice.)
3. **Write-back**. The transaction's updates are actually carried out.
4. **Garbage collect**. Finally, the transaction's updates in the log are removed.

Note that step 2 is known as the *atomic commit*. Before that moment, the transaction may safely be rolled back; after step 2 completes, the transaction must take effect. This becomes clear after understanding the recovery routine: for each record in the log (scanning sequentially), if it is...

1. an *update* record for a transaction, then add this record to a sequence of planned updates for this transaction.
2. a *commit* record for a transaction, then write-back this transaction's logged updates (i.e. the sequence of planned updates for this transaction)
3. a rollback record (if rollback records are supported), then discard the sequence of planned updates.

When the end of the log is reached, any remaining sequences of planned updates are discarded, as these transactions did not have commit records in the log!

There are, however, a few key details that must be observed.

- *Concurrent transactions* - if multiple transactions are being executed at the same time, each record must identify the transaction it belongs to.
- *Asynchronous write-back* - write-back may be delayed as far back as desired; what matters is the commit record. This allows flexibility for minimizing latency between a commit and the function itself returning, as well as optimizing throughput by batching writes together.
- *Repeated write-back* - since an update record in a transaction may be repeated/written-back multiple times, each operation must be **idempotent**—that is, it should have the same exact effect regardless of how many times it is executed.
- *Recovery crashes* - the system must be able to recover from crashes during a recovery. This is largely guaranteed by the redo log structure and idempotence already.
- *Garbage collection implementation* - it's evidently necessary to reclaim space for transactions that have been fully written back. This is usually done by keeping several **persistent** pointers—one for the most recent record that has been garbage collected, one for the most record that can be garbage collected, and one for the most recently added record—and using a circular log.

- *Ordering is important* - reordering a transaction's steps (i.e. doing write-back before committing) can lose the guarantees of atomicity.

Undo Logging

Transactions can also be implemented via undo logging, which essentially just replaces logging the transaction's intentions with logging the *old version* of the object. In the event of a failure, the system simply writes the old version of the object back into place.

However, undo logging frequently requires more random I/Os (and just more I/O in general, especially if objects are large) and gives up the flexibility (and performance) of asynchronous write-backs. Thus, redo logging is more commonly used.

Undo/redo logging involves logging both the old and new versions of an object. This allows redoing the updates of committed transactions and undoing the updates of uncommitted transactions.

⚠ Concurrent Transactions and Shared State ▾

What happens if concurrently executing transactions operate on the same shared state? Then, these transactions may not actually appear atomic. To address this, one popular method is **two-phase locking (2PL)**. This divides a transaction into two phases: during *expanding* (pre-commit), locks are only acquired; during *contracting* (post-commit), locks are only released. 2PL helps ensure a strong form of isolation known as *serializability*, i.e. the results appear as if the transactions are processed sequentially, one at a time.

Notably, 2PL has the risk of introducing deadlock. Transactions, though, provide a simple solution themselves! If a set of transactions deadlocks, one or more transactions are rolled back, release their locks, and are queued for execution later.

📄 Multiversion Concurrency Control (MVCC)

MVCC is an alternative to locks for enforcing transaction isolation. It is, essentially, the concept of **RCU Locks** abstracted and applied to transactions.

📄 Isolation Relaxation >

The strict requirements of serializability inevitably impact performance, as it forces transactions to, at times, block or roll back. In some cases, it may be beneficial to relax isolation requirements to improve performance. For instance, **snapshot isolation** only requires that each transaction's reads appear to come from a "snapshot" of the system's

committed data when the transaction starts. Transactions are buffered until they commit; then, the system checks for **write-write** conflict, essentially two transactions attempting to modify the same object with overlapping transaction times. If there exists a write-write conflict, the transaction being committed is rolled back. Note that snapshot isolation is weaker than serializability because a transaction's reads and writes happen at logically separate times (i.e., you can imagine that each transaction is split into one operation of reads and one operation of writes, and serializability is only guaranteed after this split). This allows **write skew anomalies**, where a transaction reads object x and updates object y , while a concurrent transaction reads object y and updates object x .

Finally, let's discuss the **performance** of redo logging. Surprisingly, it's not too bad, despite seemingly duplicating every single write (once to the log, once to the actual data). This is for several reasons.

- **Sequential Log updates** - this translates to a very fast log, as sequential writes are, as frequently discussed, much better than random writes.
- **Asynchronous Write-Back** - since write-back can be delayed until some time after the system "returns," (1) the latency is largely masked and (2) the updates may be batched, improving throughput.
- **Fewer Barriers/Synchronous Writes** - in the end, transactions serve as a replacement to the ad-hoc reliability solutions like FFS's careful ordering, which we know to require barriers and synchronous write operations, which ultimately reduce pipelining and parallelism. Transactions do need barriers too, but only a few per transaction.
- **Group Commit** - group commit is an optimization that combines a set of transaction commits into one log write to amortize the costs of writing commit records to disk, and effectively reduces some of the disk I/O overhead.

Transactional File Systems

Finally, we discuss the actual details of transactions in file systems. Traditional, update-in-place file systems were one of two types.

- **Journaled** - Only updates to a system's metadata are included in transactions, while updates to user's files are simply done in-place. This ensures consistency of the file system, but has fewer guarantees regarding the state of contents of regular (non-directory) files. However, it does have the benefit of better performance, as not logging the contents of regular files avoids the overhead of large writes.
- **Log-Structured** - All updates to disk (both metadata and data) are included in transactions. In other words, all updates are logged, and then at some point in the future

the system performs all the updates at once, essentially batching all the requests.

Many modern file systems, though, are copy-on-write. They offer several performance optimizations over journaled and log-structured file systems.

- **Batched Updates** - this transforms small, random writes into a few large, sequential writes and actually reduces how many blocks must be written by coalescing multiple updates of the same indirect blocks and inodes.
- **Intent Log** - essentially, an optimized redo log to support the batched updates. By default, writes are only written to the intent log if they are forced to disk, and the intent log may also use a separate, dedicated logging device that is much faster than the main storage device.

14.2 Error Detection and Correction

Storage systems include multiple defenses against device failures.

- Storage devices themselves detect failures with checksums and correct small corruption with error correcting codes
- Storage systems include redundancy using RAID architectures to reconstruct corrupted data
- File systems/software include end-to-end correctness checks

We'll discuss each of these in turn, separately.

Storage Device Failures and Mitigations

Individual storage devices—whether magnetic disk or flash—can fail in two distinct ways.

1. An isolated disk sector or flash page can lose existing data or degrade to the point where it cannot consistently store new data
2. An entire device can fail

Sector/Page Failures

Storage devices use two mitigation techniques.

Error Correcting Codes

Error correcting codes are a clever way of encoding data such that an error in a small number of bits within a page can be fixed automatically by the device hardware.

Remapping

Disks and flash devices typically include some number of spare sectors/pages. When a sector or page permanently fails, the OS or device firmware can *remap* the failed sectors/pages to good, spare ones.

Device Failures

When a whole storage device fails, the host computer will detect this failure, and any reads/writes will return error codes. Failure rates are typically described by an *annual failure rate* or the *mean time to failure (MTTF)*. In general, "the number of failure events in a fixed time period can be mathematically modeled as a Poisson process, as the interarrival time between failure events follows an exponential distribution."

RAID

RAID, or *Redundant Array of Inexpensive Disks*, is a system that cleverly distributes data redundantly across several disks for the purpose of recovering data in the event of an individual disk failure. The two most commonly discussed RAID architectures are

- **RAID 1 (Mirroring)** - quite simply, the data in one disk is always duplicated to another disk.
- **RAID 5 (Rotating Parity)** - essentially, given n disks, $n - 1$ data blocks are distributed across $n - 1$ of the disks, and a parity (redundancy) block is written to the last disk. Typically, this parity block is simply the XOR of all the other blocks; thus, if any one disk fails, each block within it can be recovered using the $n - 2$ other blocks and the 1 parity block (from the $n - 1$ remaining disks).

Notably, there are a couple performance considerations to be had with the organization of rotating parity redundancy.

- *Rotating the parity block* - the parity block must be written to every time a block within its set of blocks is updated. Therefore, to balance load, it's best to rotate the parity block of the sets between disks, i.e. evenly distribute parity blocks across the disks. (Note that having parity blocks without rotating parity is **RAID 4**).
- *Striping data* - typically, a **strip** of sequential blocks is placed on a disk before shifting to another disk in order to preserve the performance gains of sequential access (i.e. future reads/writes are likely to be sequential on a single disk, rather than having to switch disks between every block). Thus, rather than creating a set consisting of $n - 1$ blocks and 1 parity block, a set consists of $n - 1$ data strips and 1 parity strip! In other words, you can imagine that data striping treats the smallest unit as a sequential run of multiple blocks, known as a strip. Meanwhile, the set of $n - 1$ data strips and 1 parity strip is actually called a **stripe**.

Below is a diagram of RAID 5 in action.



⚠ Atomic Updates ✓

What if the system crashes while we're updating the parity block? Then, it'll seem like we have corrupted data, when really our update was just interrupted midway through. To deal with this, it's necessary to make our RAID updates atomic. There are a few solutions.

Non-volatile write buffer

Hardware RAID systems can often include a battery-backed write buffer, which simply includes the planned/in-progress writes.

Transactions

Transactions provide atomicity by nature.

Recovery scan

After a crash, the system scans the blocks and updates any inconsistent parity blocks.

ℹ RAID: Extended ✓

Though we describe in detail just RAID 1 and RAID 5 above, there are actually many more RAID levels! The original paper described RAID 0 – RAID 5; three of these variants see wide use today.

RAID 0: JBOD (Just a Bunch Of Disks)

This system doesn't actually add any redundancy; it just spreads data across multiple disks.

RAID 1: Mirroring (described previously)

RAID 5: Rotating Parity (described previously)

More variants have also been developed.

RAID 6: Dual Redundancy

RAID 5 but with two parity blocks, generated using erasure codes (e.g. Reed-Solomon codes) that tolerate up to two disk failures.

RAID 10, RAID 50: Nested Raid

These were originally called RAID 1+0 and RAID 5+0. Quite simply, they just combine RAID 1 with RAID 0 or RAID 5 with RAID 0, respectively.

RAID Reliability?

So, how reliable are RAID 1 and RAID 5? They can actually lose data in 3 distinct ways.

- 2 full disk failures
- 1 full disk failure and 1+ sector failures on other disks
- Overlapping sector failures (same stripe) on multiple disks

The expected time until one such event occurs is the *mean time to data loss (MTTDL)*.

But we can, of course, further improve RAID reliability. Broadly, there are three techniques.

1. **Increasing redundancy** - double or even triple redundancy, i.e. mirroring data on more than one disk for RAID 1. A *dual redundancy array* allows data to be reconstructed tolerating at most two disk failures, i.e. this is just RAID 6, which is double redundancy for RAID 5.

2. **Reducing non-recoverable read error rates**

Scrubbing - periodically reading all disk sectors, detecting sectors with unrecoverable read errors, reconstructing their contents with RAID, and remapping them to a spare.

More reliable disks - yeah... idk why this is listed in the textbook but yup

3. **Reducing mean time to repair (MTTR)**

Some systems include a *hot spare* disk that is idle, but can quickly replace a failing disk. MTTR is, though, inevitably limited by the time to write all the reconstructed data to the new disk too.

Some systems also *decluster* data, splitting the reconstruction across multiple disks to allow parallel reconstruction.

Software Integrity Checks

These are integrity checks implemented at the software/file system level, to detect hardware failures themselves. We discuss two common techniques.

- **Block Integrity Metadata**

This is commonly some sort of checksum included with each data block. For instance, the Write Anywhere File Layout (WAFL) file system stored a 64 byte *data integrity segment* (DIS) for each 4 KB block, which contained a checksum, the data block's identity, and a checksum of the DIS itself.

- **File System Fingerprints**

File systems like ZFS include fingerprints that provide a checksum for the *entire* file system via a **Merkle tree**. Understanding a Merkle tree is left as an exercise to the reader... bonus points for studying its use in cryptography :)